



OWASP
Smart Contract Security

scs.owasp.org ↗

Security Handbook : Web3 Operational Security

11

Security Handbook Series

GRANT SUPPORT • 2025-2026



ecosystem
support
program

Supported by Ethereum Foundation's
Ecosystem Support Program (ESP)



Contents

Frontispiece	4
About This Handbook	4
Copyright and License	5
Author and Creative Credits	5
Purpose and Scope	6
Target Audience	6
How to Use This Handbook	6
Relationship to SCSVS, SCSTG, and SCWE	7
Part I: Foundations	8
1. Introduction to Operational Security	8
2. Alignment with Standards and Frameworks	11
3. Core Principles	19
4. Operational Implementation Process	28
Part II: Web3-Specific Considerations	36
5. Web3 OpSec Challenges	36
6. Endpoint and Device Security	42
7. Workspace and Productivity Security	47
8. Authentication and Access	49
9. Communication and Collaboration Security	51
10. Travel and Remote Work	54
Part III: Control Domains	58
11. Control Domains Overview	58
12. Control Selection and Implementation	69
Part IV: Integration and Continuous Improvement	74
13. Integration with Other Frameworks	74
14. Continuous Improvement	80
Part V: Appendices	87
15. Appendix A: Glossary	87
16. Appendix B: Policy and Template Library	91
17. Appendix C: Case Studies and Tabletop Exercises	93
18. Appendix D: Checklists (Quick Reference)	99
19. Appendix E: References and Further Reading	105

Appendix C: References and Further Reading 107

Standards and Frameworks 107

Incidents and Case Studies..... 108

Tools and Documentation..... 108

Further Reading..... 109

Frontispiece

OWASP SMART CONTRACT SECURITY PROJECT

Web3 Operational Security

About This Handbook

Web3 Operational Security is part of the OWASP Smart Contract Security Handbook Series. The series extends smart contract security beyond contract code into the people, process, application, infrastructure, and operational layers surrounding Web3 systems.

Each handbook is designed as a defensive field reference for architects, engineers, security practitioners, incident responders, auditors, and organizational leaders. It complements the OWASP Smart Contract Security Verification Standard (SCSVS), Smart Contract Security Testing Guide (SCSTG), Smart Contract Weakness Enumeration (SCWE), Smart Contract Top 10, and SCS Checklist without replacing those resources.

SERIES EDITION **2026**

Copyright and License

Copyright © 2026 The OWASP Foundation and contributors.



This document is released under the [Creative Commons Attribution-ShareAlike 4.0 International License](#). You may share and adapt the material with appropriate attribution and under the same license. For reuse or distribution, make the license terms clear and identify material changes.

OWASP publications are community-developed educational resources. References to organizations, products, or services do not constitute endorsement, and the material is provided without warranty.

Author and Creative Credits

PROJECT LEAD

Shashank | CredShields

CEO and Co-Founder

AUTHOR | PROJECT MAINTAINER

Pratik Lagaskar

Security Researcher

COVER PAGE DESIGNS

Siddharth Neekher

Lead UI Designer @ CredShields

OWASP Smart Contract Security Project

Purpose and Scope

This handbook operationalizes security for Web3 teams: identifying critical information, analyzing threat actors, assessing vulnerabilities, weighing risk in business context, and deploying targeted countermeasures. It is anchored in two current U.S. National Institute of Standards and Technology (NIST) references: Zero Trust Architecture (ZTA) under [NIST SP 800-207](#) and its companion implementation guide [NIST SP 1800-35](#), plus the [NIST Cybersecurity Framework \(CSF\) 2.0](#) functions of Govern, Identify, Protect, Detect, Respond, and Recover. Both frameworks assume a perimeter-less, “never trust, always verify” model, a natural fit for Web3 organizations that never had a network perimeter to defend in the first place.

A protocol’s smart contracts can pass every audit while the team behind them loses everything to a phished admin, an unlocked laptop, or a hot wallet compartmentalized in name only. Web3 operations run on distributed teams, pseudonymous collaborators, and Web2 SaaS tooling that sits next to keys controlling real value, so operational security (OpSec) here carries the same discipline the term originally described in intelligence and military doctrine: protect the information and access that, if exposed, lets an adversary act. This handbook applies that discipline to key management, endpoint hardening, communications, travel, and the control domains a Web3 team runs day to day.

Target Audience

Security specialists, operations and strategy leads, DevOps and Site Reliability Engineering (SRE) teams, protocol and dApp engineering teams, and anyone who carries day-to-day responsibility for the operational security of a Web3 organization.

How to Use This Handbook

Start with Part I to fix vocabulary and the standards baseline: what OpSec means in a Web3 context, how NIST ZTA and CSF 2.0 map onto it, and the five-step operational process (asset identification, threat analysis, vulnerability assessment, risk evaluation, control deployment) that structures every chapter after it. Part II is Web3-specific reference material, covering the challenges unique to transparent, decentralized, key-custodying teams before moving through endpoint, workspace, authentication, communication, and travel controls in turn. Part III organizes controls by domain (organizational, personnel, physical, technical) and walks through selecting and deploying them. Part IV integrates OpSec with the wider security program and sets

a cadence for continuous improvement. Part V holds the glossary, a policy and template library, case studies with tabletop exercise scripts, and quick-reference checklists; a reader under time pressure can jump straight to Appendix D and work outward from there.

Relationship to SCSVS, SCSTG, and SCWE

This handbook sits beside, not inside, the Smart Contract Security Verification Standard (SCSVS): SCSVS verifies that a deployed system satisfies defined security requirements, while the controls here protect the humans, devices, and processes that build, sign, and operate that system. The Smart Contract Security Testing Guide (SCSTG) tests code and infrastructure; the operational controls in Parts II and III are what keep the environment the SCSTG tests from being undermined by a compromised laptop or a phished operator before code ever ships. The Smart Contract Weakness Enumeration (SCWE) catalogs weaknesses in contract code; this handbook covers the class of weakness SCWE does not enumerate, in people and process. Two sibling handbooks share the most surface area: the Hiring, Remote Work, and Insider Threat Handbook (05) goes deep on the personnel-side controls introduced in Chapters 6 through 10, and the Incident Response Handbook (06) owns the escalation and forensics process that Chapter 13's integration section hands off to.

Part I: Foundations

This part defines operational security (OpSec) for a Web3 organization, distinguishes it from smart contract security and from generic enterprise IT security, and grounds it in the frameworks that give it verifiable structure. It sets out five core OpSec principles used throughout the handbook: defense in depth, least privilege, need-to-know, compartmentalization, and continuous monitoring. It closes with the five-step operational implementation process that later parts apply to specific Web3 domains: identify assets, analyze threats, assess vulnerabilities, evaluate risk, deploy controls.

1. Introduction to Operational Security

Before a team can defend key custody, employee onboarding, or incident response, it needs a shared definition of operational security and a reason to prioritize it above the smart contract audit that usually consumes the security budget. This chapter supplies both.

1.1 What is Operational Security

Operational security (OpSec) began as a US military discipline, not an IT one. During the Vietnam War, a team investigating why enemy forces anticipated American operations found no single classified leak: the adversary was piecing together publicly observable indicators (flight schedules, radio traffic patterns, logistics movements) into an accurate picture of intent. The Department of Defense named the discipline of denying that kind of inference Operations Security, and the United States formalized it government-wide in [National Security Decision Directive 298 \(1988\)](#), which still defines the five-step OPSEC process: identify critical information, analyze threats, analyze vulnerabilities, assess risk, and apply countermeasures.

OpSec is not synonymous with information security. Information security protects the confidentiality, integrity, and availability of specific systems and data; OpSec protects against an adversary correlating fragments (public GitHub activity, a conference talk, an out-of-office reply, a wallet's

on-chain history) into an operational picture that no single fragment reveals on its own. A firewall rule is an information-security control. A policy against announcing a treasury rebalance before it executes is an OpSec control.

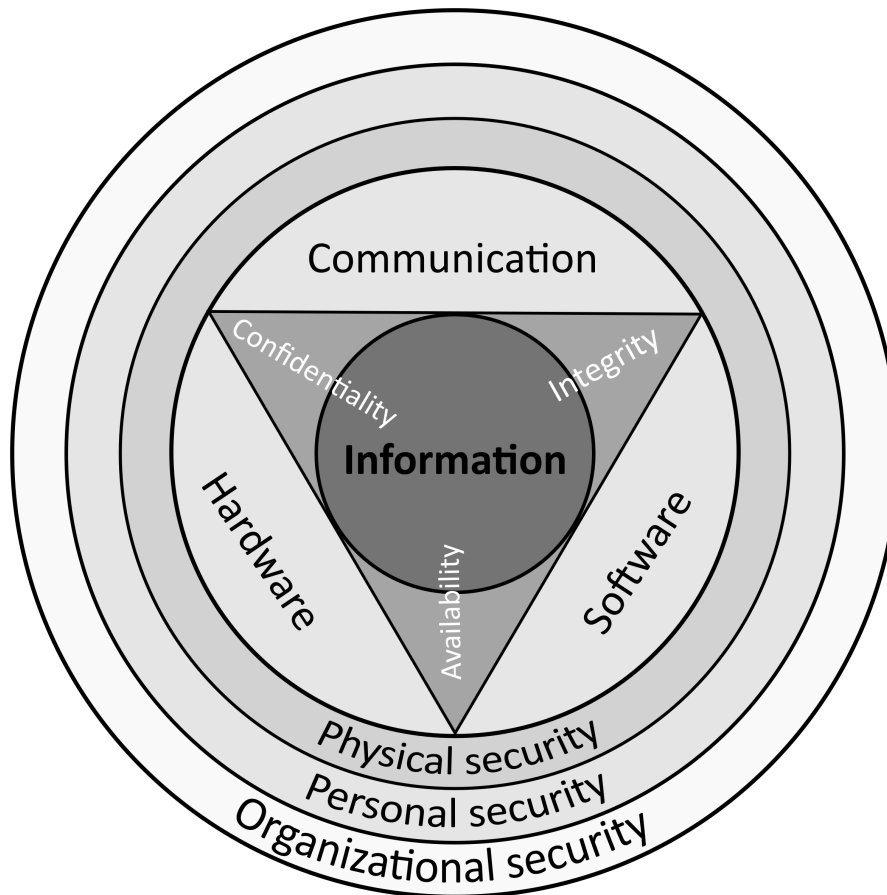


Figure. The CIA triad: confidentiality, integrity, and availability, the three goals information security protects. OpSec is the discipline that sits alongside it, protecting against inference from fragments no single one of these three properties covers. Source: [Wikimedia Commons](#), CC BY-SA 4.0.

The five-step process in NSDD-298 reappears, adapted, as Chapter 4 of this Part: critical asset identification, practical threat analysis, actionable vulnerability assessment, contextual risk evaluation, and targeted control deployment. That structure, not a list of tools, is what makes OpSec a repeatable discipline rather than a vague call to “be careful.”

1.2 Why OpSec Matters in Web3

Two properties make OpSec disproportionately important in Web3 compared with a typical enterprise. First, the ledger is public and permanent: a wallet’s balance, its transaction graph, and often the ENS name or social account linked to it are queryable by anyone, giving an adversary a head start on the open-source intelligence (OSINT) reconnaissance that used to take weeks of corporate research. Second, execution is irreversible: a signed transaction that moves funds to an attacker has no chargeback and, absent a rare and contested social-layer intervention, no recovery path.

The largest crypto losses on record bear this out, and most of them are OpSec failures, not smart contract bugs. In March 2022, attackers the FBI attributed to North Korea's [Lazarus Group and APT38](#) compromised five of nine validator keys on the Ronin sidechain behind Axie Infinity, reportedly through a fake job offer that delivered a malicious payload to a senior engineer, and drained roughly \$620 million ([Ronin Network incident summary](#)). In February 2025, an attacker compromised infrastructure at Safe{Wallet}, the multisig provider Bybit used, and manipulated the transaction Bybit's signers reviewed before they approved it, converting a routine cold-wallet transfer into a theft of roughly \$1.4 to 1.5 billion in Ether, again attributed by the FBI to North Korean state-linked actors ([Bybit hack summary](#)). Neither attack exploited a line of Solidity. Both fall under [WA05 \(Fake Interview and Video Call Social Engineering\)](#) and [WA01 \(Multisig Hijacking\)](#) of the OWASP Web3 Attack Vectors Top 15, the companion catalog covered in full in Handbook 10.

1.3 How to Use This Handbook

Read this Part once, start to finish, to fix the vocabulary and the two frameworks (Zero Trust Architecture and the Cybersecurity Framework) that later parts assume. After that, treat the handbook as a reference to dip into by task. Part II, Web3-Specific Considerations, adapts the general principles here to constructs that only exist in this industry: multisig custody, validator operation, DAO governance, and pseudonymous contributor management. Part III, Control Domains, is the operational core, with domain-by-domain controls for identity, endpoints, communications, and physical security. Part IV, Integration and Continuous Operations, covers how OpSec fits into a program that already runs incident response and monitoring, and how to keep it current as tactics change. Part V holds the appendices: templates, checklists, and the worksheets referenced throughout.

If your immediate need is narrower, for example hardening a key ceremony or writing an off-boarding checklist, skip to the relevant chapter directly. Each chapter is written to stand on its own once you carry forward the five OpSec principles introduced in Chapter 3.

1.4 Relationship to SCSVS, SCSTG, and SCWE

The [OWASP Smart Contract Security \(SCS\)](#) project's existing standards are deliberately scoped to the contract layer. The Smart Contract Security Verification Standard (SCSVS) states verifiable requirements for the code that runs on-chain. The Smart Contract Security Testing Guide (SCSTG) describes how to test that code. The Smart Contract Weakness Enumeration (SCWE) catalogs the ways that code goes wrong. None of the three asks who held the deployer key, whether the person who approved a Safe transaction verified it on a hardware device, or whether a departed employee still has console access. This handbook answers those questions.

The [OWASP Web3 Attack Vectors Top 15](#) makes the gap explicit: of its fifteen categories, at least six (WA01 Multisig Hijacking, WA03 Private Key Compromise, WA05 Fake Interview and Video Call Social Engineering, WA11 Wrench Attacks and Physical Coercion, WA12 Insider Threats and

Collusive Abuse, and WA15 Nation-State Infiltration via Fake Hiring) are OpSec failures with no corresponding SCWE entry, because they never touch the contract. This handbook operationalizes those categories in detail. It also cross-references sibling handbooks that go deeper on adjacent surfaces the Web3 Attack Vectors list flags but this one only summarizes: Employee Lifecycle Security (03), Hiring, Remote Work, and Insider Threat (05), Incident Response (06), and Infrastructure Security (07).

2. Alignment with Standards and Frameworks

Web3-specific advice is more durable when it maps onto frameworks an enterprise security program, an auditor, or a regulator already recognizes. This chapter grounds the handbook in two publications from the US National Institute of Standards and Technology (NIST) and the zero trust components both reference.

2.1 NIST Zero Trust Architecture (ZTA)

Zero trust architecture (ZTA) replaces the assumption that anything inside a network boundary is safe with a requirement that every access request is evaluated on its own merits. [NIST Special Publication \(SP\) 800-207](#) defines the model abstractly; a companion practice guide translates it into deployable reference designs. The four subsections below cover the standard, its stated boundaries, its central tenet, and why it fits Web3 organizations better than most enterprises realize.

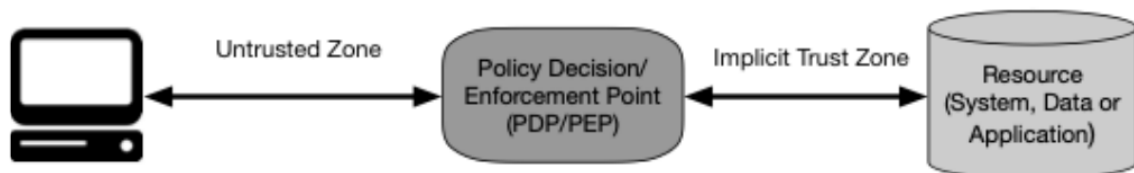


Figure 1: Zero Trust Access

Figure. Zero trust moves the decision boundary from “inside the network” to access to each resource. For Web3 operations, a source repository, RPC console, signer-management interface, and treasury workflow are separate resources whose authorization decisions should not be inherited from one successful login. Source: [NIST SP 800-207](#), U.S. government work, public domain.

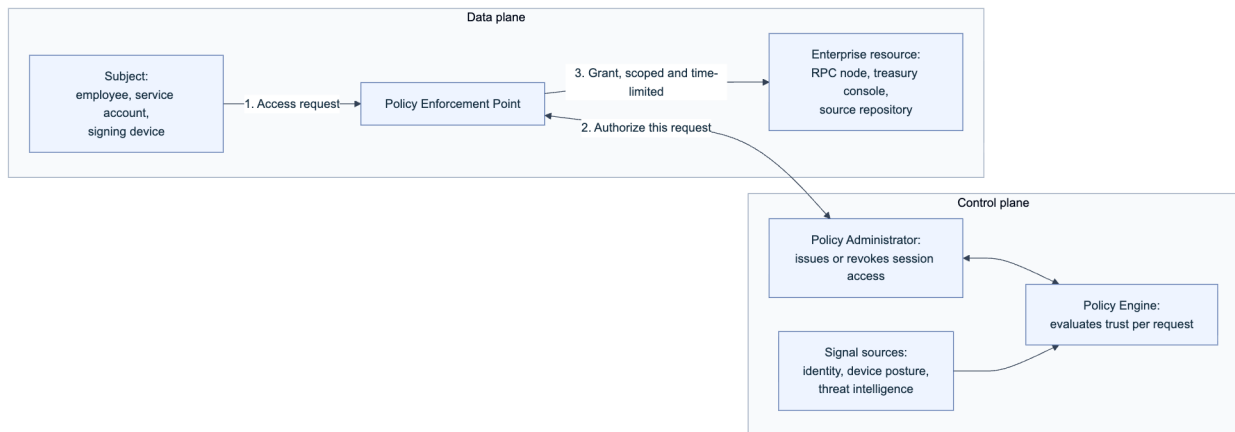


Figure 1. NIST SP 800-207's logical Zero Trust Architecture components, redrawn for a Web3 resource. No standing trust exists between subject and resource; the Policy Engine evaluates fresh signal on every request.

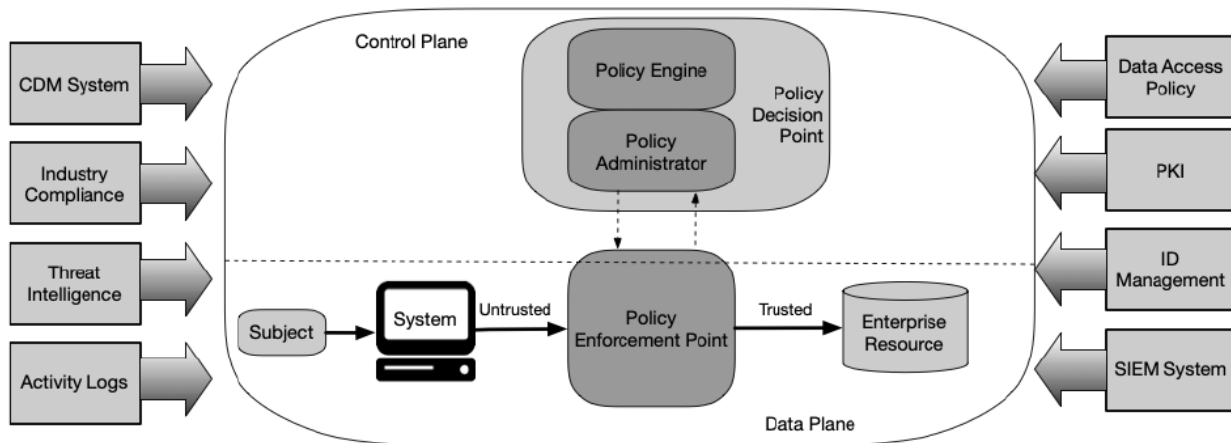


Figure 2: Core Zero Trust Logical Components

Figure. The authoritative NIST component model distinguishes the policy decision point from the enforcement point in the data path. Keeping those roles separate is what lets policy use identity, device, threat, and telemetry signals without placing every signal source directly in the resource path. Source: *NIST SP 800-207*, U.S. government work, public domain.

2.1.1 NIST SP 800-207 and SP 1800-35 Practice Guide

NIST published SP 800-207, *Zero Trust Architecture*, in **August 2020**. It defines zero trust as a set of principles rather than a product category: no implicit trust is granted based on network location or asset ownership, every access request is authenticated and authorized before it is granted, and access is scoped to the minimum necessary and continuously reevaluated. The document is deliberately abstract so it applies across cloud, on-premises, and hybrid environments without prescribing a specific vendor architecture.

That abstraction left a gap between principle and deployment, which the National Cybersecurity Center of Excellence (NCCoE) closed with SP 1800-35, *Implementing a Zero Trust Architecture*, finalized in **June 2025**. The NCCoE built nineteen example implementations with twenty-four industry collaborators, covering identity governance, microsegmentation, software-defined perimeters, and secure access service edge, and published the lessons learned from each. For a Web3

team, SP 1800-35 is the more useful document day to day: it shows concretely how ZTA maps to identity providers, endpoint agents, and network controls a security engineer can actually buy or build, where SP 800-207 supplies the vocabulary to argue for doing so.

2.1.2 Scope: Enterprise IT; Excludes ICS, OT, IoT (See NIST for Boundaries)

SP 800-207 states its own boundary: it targets enterprise IT infrastructure and explicitly excludes industrial control systems (ICS), operational technology (OT), and most Internet of Things (IoT) deployments, which have different latency, availability, and failure-mode requirements that a per-request authentication model can violate. NIST points readers to separate ICS/OT guidance rather than stretching 800-207 to cover them.

That boundary matters for Web3, because some infrastructure a protocol depends on sits closer to OT than to enterprise IT: a hardware security module (HSM) signing a validator's blocks, a bare-metal node with strict uptime requirements, or a physically gated safe cannot always tolerate a policy engine that adds latency or fails closed on a network blip during a live signing ceremony. Apply ZTA fully to the corporate identity, endpoint, and application layers this handbook covers, and treat signing infrastructure and physical custody hardware as a distinct domain with its own availability-first controls, covered in Part III. Conflating the two produces either an unsafe ZTA rollout on custody hardware or an under-protected corporate perimeter left in the legacy model.

2.1.3 "Never Trust, Always Verify" and Identity-Centric Security

"Never trust, always verify" is the popular shorthand for zero trust, though NIST's own phrasing is more precise: no implicit trust based on physical or network location. In practice this shifts the primary security boundary from the network perimeter to identity. Under a perimeter model, a user who reached the corporate virtual private network (VPN) was largely trusted for the session that followed. Under an identity-centric model, every request, not every session, carries its own authentication and authorization decision, informed by who is asking, from what device, in what posture, and against what resource.

For Web3 organizations this is a natural fit rather than a retrofit. A multisig co-signer's identity (which hardware key, which device, whether that device shows the correct signing screen) matters more than which network the co-signer connected from; a compromised laptop on the correct corporate VPN is exactly the scenario that drained Bybit's cold wallet through Safe{Wallet} in February 2025. Binding access decisions to verified identity and device posture, re-evaluated per request, closes the gap that a one-time network-perimeter check leaves open.

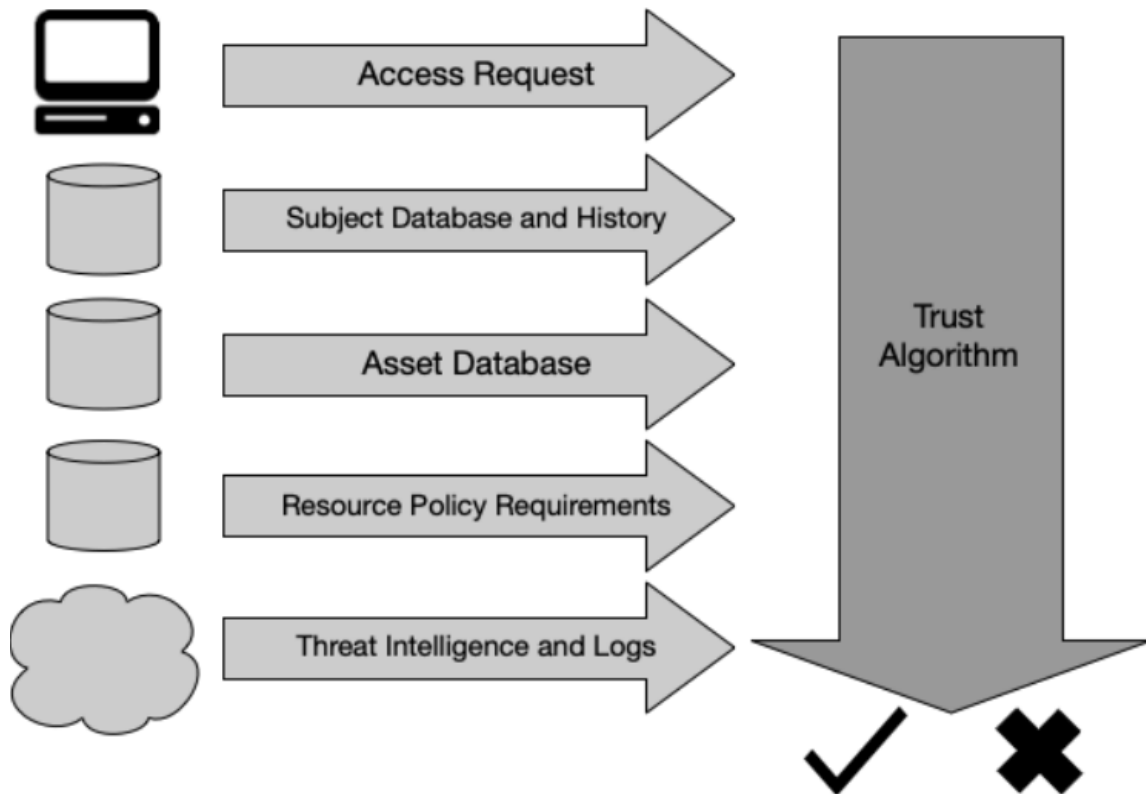


Figure 7: Trust Algorithm Input

Figure. A zero-trust decision is only as trustworthy as its inputs. Identity alone is insufficient: resource policy, asset posture, threat intelligence, historical behavior, and current telemetry must converge on the policy engine, with stale or unavailable signals handled by an explicit fail-closed rule for critical assets. Source: [NIST SP 800-207](#), U.S. government work, public domain.

2.1.4 Applicability to Web3 (No Traditional Perimeter)

Most enterprises adopt zero trust to dismantle a perimeter they already have: office networks, VPNs, and on-premises data centers built over decades. Web3 organizations rarely have one to dismantle. Teams are commonly remote-first and distributed across time zones and jurisdictions from day one; infrastructure spans self-hosted validator nodes, multiple cloud providers, hardware wallets, and third-party RPC services; contributors range from full-time employees to pseudonymous DAO participants paid in tokens. There was never a castle, so there is no moat to defend and no legacy VPN culture to unwind.

That absence is an advantage only if a team designs for it deliberately, and a liability if it does not. A validator operator with signers in four countries cannot assume any network location is trustworthy, so access to signing infrastructure must be authenticated and authorized on a per-session, per-device basis regardless of where the request originates: this is the ZTA model applied by default rather than by migration. The risk is that “no perimeter to defend” gets quietly read as “no perimeter to build,” leaving identity as the only control and skipping the device posture, continuous monitoring, and policy engine layers that make identity-centric security actually work rather than merely sound modern.

2.2 NIST Cybersecurity Framework (CSF) 2.0

Where ZTA governs access decisions, the **NIST Cybersecurity Framework (CSF) 2.0**, released in **February 2024**, gives a vocabulary for the security program as a whole. It organizes a program into six functions and, unlike its 2018 predecessor (CSF 1.1), treats governance as a first-class function rather than an implicit assumption. The two subsections below define the functions and map them to OpSec controls.

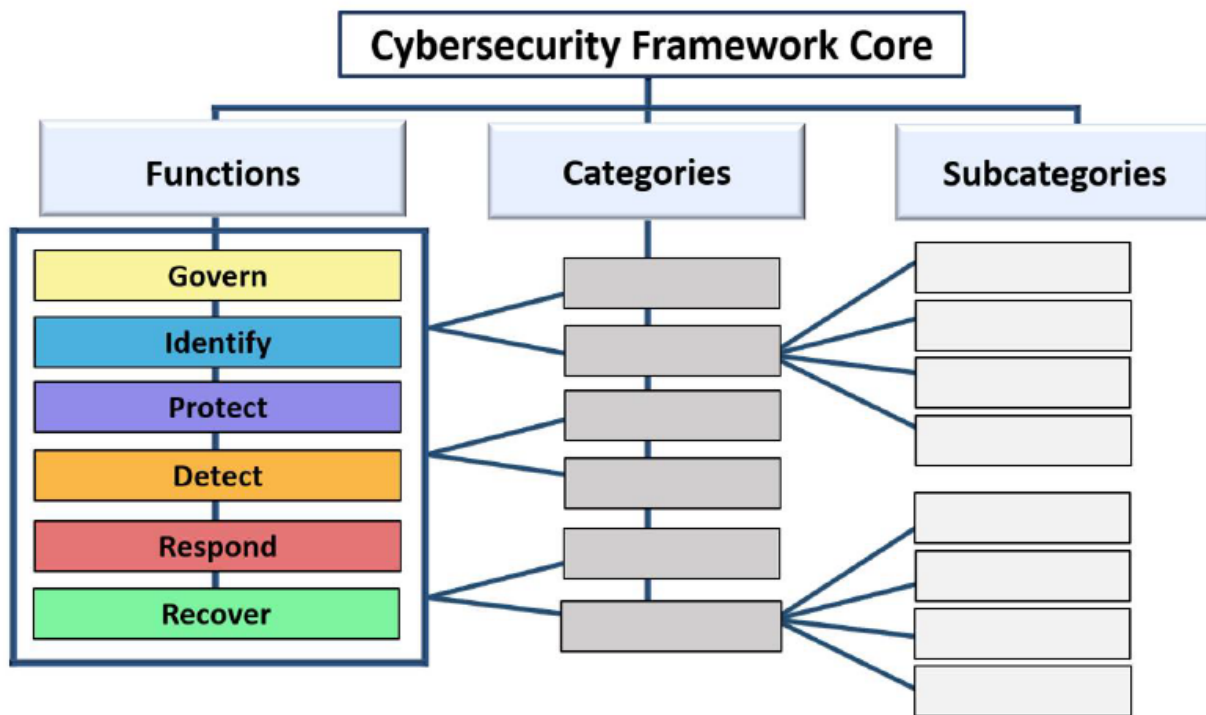


Fig. 1. CSF Core structure

Figure. CSF 2.0 is an outcomes hierarchy, not a product checklist: Functions organize Categories, Categories contain measurable Subcategories, and Informative References map outcomes to implementation guidance. Use that structure to connect an OpSec policy objective to evidence an assessor can verify. Source: *NIST Cybersecurity Framework 2.0*, U.S. government work, public domain.

2.2.1 Govern, Identify, Protect, Detect, Respond, Recover

CSF 2.0 organizes cybersecurity outcomes into six functions, listed here with the one-line purpose NIST assigns each:

Function	Purpose
Govern (GV)	Establish and monitor the organization's cybersecurity risk management strategy, expectations, and policy (new in 2.0)
Identify (ID)	Understand the organization's assets, risks, and current security posture
Protect (PR)	Implement safeguards to limit or contain the impact of a potential event

Function	Purpose
Detect (DE)	Find and analyze anomalies and indicators of compromise
Respond (RS)	Take action once an incident is confirmed
Recover (RC)	Restore capabilities and services impaired by an incident

Govern is the headline addition in the 2.0 revision. CSF 1.1 buried governance inside Identify; CSF 2.0 elevates it to its own function, explicit that cybersecurity risk is enterprise risk and belongs on the same reporting line as financial or legal risk, with a dedicated Supply Chain Risk Management (GVSC) category. For a Web3 organization, that elevation matters directly: a treasury multisig threshold, a key-holder travel policy, or a decision to concentrate signing authority in one geography are governance decisions with security consequences, and CSF 2.0 gives them a home in the framework rather than leaving them as engineering-team footnotes.

The remaining five functions map cleanly onto the five-step OpSec process from Section 1.1 and Chapter 4. Identify and Protect correspond to asset identification and control deployment, Detect and Respond correspond to continuous monitoring, and Recover closes the loop back to Govern for lessons learned.

2.2.2 Mapping OpSec Controls to CSF Functions

The table below gives one representative Web3 OpSec control per function, each expanded in later parts of this handbook.

CSF Function	Representative Web3 OpSec control
Govern	Board-level key-custody policy defining multisig thresholds and signer geographic distribution
Identify	Living asset register of every wallet, deployer key, admin role, and infrastructure credential (Section 4.1)
Protect	Hardware-backed signing, least privilege, and just-in-time access for privileged roles (Sections 2.3.4, 3.2)
Detect	On-chain transaction monitoring and off-chain security information and event management (SIEM) alerting on privileged account activity
Respond	Incident response runbook with a pause or freeze path, covered fully in Handbook 06
Recover	Tested key regeneration and cold-backup restoration procedure after a compromise

Use the table as a gap-finding tool, not a compliance checkbox. A team that can point to strong Protect controls but has no Govern-level policy stating who is authorized to change a multisig threshold has a documented function without an accountable owner, precisely the gap CSF 2.0

was revised to close. Run this mapping exercise during the asset identification step in Chapter 4, so that control deployment in Section 4.5 has a known function, and therefore a known owner and reporting line, for every control it proposes.

2.3 Zero Trust Components Relevant to OpSec

SP 1800-35 groups zero trust into implementable components rather than leaving it at the principle level of SP 800-207. Four of those components recur throughout this handbook's control chapters: identity governance, network microsegmentation, secure remote access, and time-bound credentials. Each is introduced here and applied concretely in Part III.

2.3.1 Enhanced Identity Governance (EIG) and ICAM

Enhanced Identity Governance (EIG) is [CISA's](#) term for a zero trust approach in which identity is the primary basis for granting access, rather than one factor among several. It depends on mature Identity, Credential, and Access Management (ICAM), the US federal government's umbrella term for the systems and processes that establish who a digital identity belongs to, issue it a credential, and govern what that credential can access (idmanagement.gov).

Web3 organizations run two identity systems that rarely talk to each other: corporate identity (a workspace suite, a single sign-on provider, GitHub organization membership) and cryptographic identity (which key controls which multisig signer slot, which address holds a validator's operator role). EIG done well links them: revoking an employee's corporate identity should trigger a review of every cryptographic role tied to that person, not leave a signer key active because it lives in a hardware wallet the offboarding checklist never touches. Bot and relay service accounts need the same governance discipline as human ones; an unreviewed automation key with standing signing rights is functionally an ungoverned identity, regardless of whether a human sits behind it.

2.3.2 Microsegmentation and Software-Defined Perimeter (SDP)

Microsegmentation divides a network into small, independently controlled zones so that reaching one workload does not imply reaching its neighbors. Where a traditional flat network lets a compromised marketing laptop route toward a database server, microsegmentation requires a policy decision at every hop. Software-Defined Perimeter (SDP), a related architecture formalized by the [Cloud Security Alliance](#), goes further by making protected resources invisible until a client authenticates: no open port, no listening service visible to an unauthenticated scan, an approach sometimes summarized as “dark” infrastructure.

Apply both to signing and node infrastructure specifically. A validator's signing key and the machine that holds it belong in a segment unreachable from the general corporate network and unreachable from the public internet outright. A remote procedure call (RPC) endpoint or admin console that would otherwise need a public IP address and an allowlist can instead sit behind an

SDP gateway that never advertises the service to an unauthenticated client. The practical payoff is that a phished laptop or a leaked engineer credential no longer implies a path to signing infrastructure, because that infrastructure was never reachable from the segment the laptop lives in.

2.3.3 Secure Access Service Edge (SASE) and Remote Access

Secure Access Service Edge (SASE, commonly pronounced “sassy”), a term [Gartner](#) introduced in a 2019 report on the convergence of networking and security, describes delivering SD-WAN connectivity together with a bundle of cloud-delivered security services (secure web gateway, cloud access security broker, zero trust network access, and firewall-as-a-service) from edge points of presence close to the user, rather than backhauling traffic to a corporate data center.

SASE fits a Web3 team’s default topology better than a legacy site-to-site VPN does. A VPN typically grants network-level access once connected, reintroducing the implicit trust ZTA is meant to remove, and it assumes a hub of corporate offices that most Web3 teams never build. A SASE deployment enforces identity-aware policy at the edge nearest each remote contributor, whether an engineer in one country or a contractor in another, without ever placing them “inside” a network segment they do not need. The result is consistent policy enforcement for a distributed workforce, with zero trust network access replacing standing VPN tunnels as the default remote-access model in SP 1800-35’s example implementations.

2.3.4 Just-in-Time and Ephemeral Credentials

Just-in-time (JIT) access grants a privileged permission only for the window it is needed, then automatically revokes it, replacing the standing admin role that sits active, and attackable, around the clock for the sake of the ten minutes a month it is actually used. Ephemeral credentials extend the same idea to secrets themselves: a short-lived, auto-expiring token or key rather than a long-lived password or API key that, once leaked, remains valid indefinitely.

Web3 operations have concrete places to apply both. A multisig co-signer role can be time-boxed to the window around a scheduled transaction rather than left standing. Cloud infrastructure roles used to deploy a contract or update a node can be issued through short-lived session tokens (AWS Security Token Service, or a dynamic-secrets engine such as HashiCorp Vault) rather than long-lived Identity and Access Management (IAM) keys committed to a build variable:

```
# Example: request a 15-minute scoped AWS session instead of using a static key
aws sts assume-role \
  --role-arn arn:aws:iam::ACCOUNT_ID:role/deploy-readonly \
  --role-session-name ci-deploy-$(date +%s) \
  --duration-seconds 900
```

The security payoff compounds with the identity governance in Section 2.3.1: even if EIG fails to catch a stale identity during offboarding, a credential that expired eight hours ago closes the gap anyway. Standing credentials are a single point of failure that JIT access is designed to remove, a theme Chapter 3 returns to directly.

3. Core Principles

Frameworks describe what a mature program looks like from the outside; principles are what an engineer or an operator actually applies when making a decision at two in the morning. This chapter sets out the five principles that recur, named explicitly, throughout every later part of this handbook.

3.1 Defense in Depth

Defense in depth layers independent controls so that the failure of any single one does not expose the protected asset directly. The idea predates computing (it is a military doctrine for fortification), and its computing form is described in classic references such as [NIST SP 800-53](#) and [CISA's layered-security guidance](#). The three subsections below define it, show how the layers stack for a Web3 asset, and explain why single points of failure defeat the whole idea.

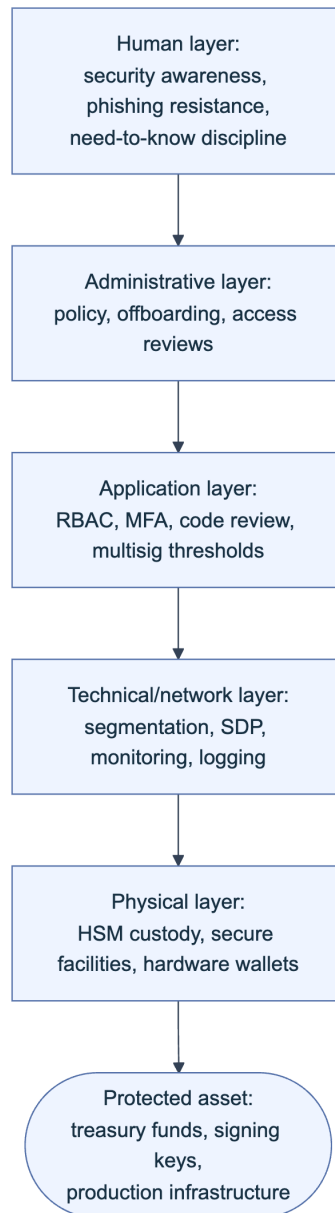


Figure 2. Defense in depth as five independent layers around a Web3 asset. An attacker who defeats one layer, for example a phished credential at the human layer, still faces every layer beneath it.

3.1.1 Definition and Rationale

Defense in depth accepts, as a starting assumption, that any single control will eventually fail: a patch will be late, a phishing email will land in the right inbox on the wrong day, a password will be reused. Rather than trying to build one perfect control, the strategy stacks multiple independent controls so that an adversary who defeats one still has to defeat the next, and the next, before reaching the protected asset. Independence matters as much as quantity: five controls that all depend on the same credential store are not five layers, they are one layer wearing five names.

The rationale is economic as much as technical. Each additional layer raises an attacker's cost (time, tooling, and the chance of detection) even when it does not raise that cost to infinity, and the goal of most Web3 defenses is not absolute prevention but making an attack too slow, too

expensive, or too visible to complete before the target's own controls (monitoring, rate limits, multisig thresholds) intervene. A single high-quality control, a strong password policy, for instance, is still worth having, but treating it as sufficient on its own is the specific mistake defense in depth exists to correct.

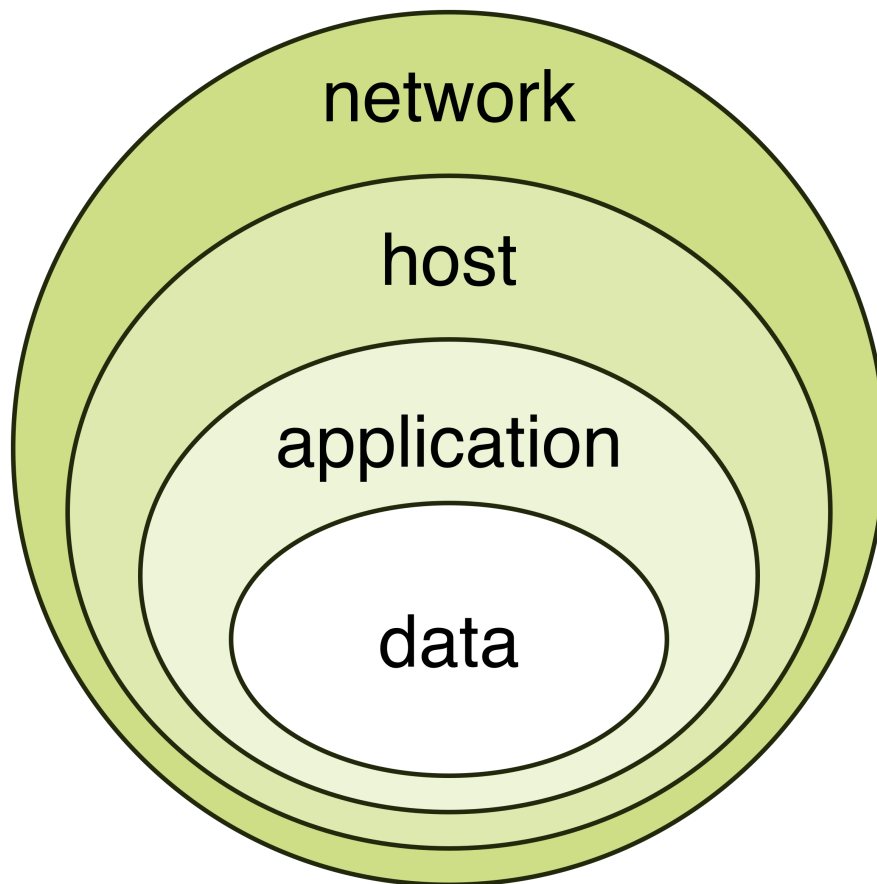


Figure. Defense in depth drawn as an onion: the protected asset sits at the center, and an attacker must peel back each independent layer in turn to reach it. Source: [Wikimedia Commons](#), CC BY-SA 3.0.

3.1.2 Layering Controls Across Physical, Technical, Administrative, and Human

Classic defense-in-depth models group controls into four categories, and Web3 operations have concrete instances of each:

Layer	Traditional example	Web3 instance
Physical	Badge-controlled data center access	HSM custody, hardware wallet storage, secure key-ceremony facilities
Technical	Firewalls, network segmentation	Microsegmented signing infrastructure, on-chain monitoring, SDP-gated RPC access
Administrative	Written policy, access review cadence	Multisig signer policy, offboarding runbooks, change-management approval

Layer	Traditional example	Web3 instance
Human	Security awareness training	Phishing-resistant multi-factor authentication (MFA) culture, fake-recruiter awareness training tied to WA05/WA15

No single layer is sufficient, because Web3-specific attacks are engineered to skip straight to whichever layer is weakest. A drainer contract (WA04) defeats technical controls entirely by relying on a user's own signature; a fake interview lure (WA05) defeats every technical control by targeting the human layer directly, exactly why the Ronin Bridge and Bybit incidents in Section 1.2 both succeeded despite the target organizations running conventional endpoint security. Effective defense in depth for a Web3 team therefore means building administrative and human-layer controls with the same rigor normally reserved for the technical layer, not treating awareness training as a compliance formality bolted onto a network diagram.

3.1.3 Ensuring No Single Point of Failure

A single point of failure (SPOF) is any component whose compromise alone defeats the entire defense stack, regardless of how many layers surround it. The clearest Web3 example is a single private key with unilateral control over an upgradeable contract's admin function or a treasury's entire balance: no matter how well that key is physically secured, its compromise is total and immediate, and every layer in Section 3.1.2 becomes irrelevant the moment it happens.

The structural fix is to require quorum rather than a single actor for any high-value action: a multisig wallet requiring M-of-N signatures, an HSM cluster that needs multiple key-shard holders (a Shamir's Secret Sharing scheme, for instance), or a timelock that forces a delay between a proposed action and its execution, giving monitoring and human review a window to intervene. Quorum requirements have their own failure mode if misconfigured; a two-of-three multisig where two signers share an office, a laptop, or a cloud account is not meaningfully more resilient than a single key. Validate that the parties satisfying a threshold are actually independent across every layer in the table above, not merely independent in name on a governance document.

3.2 Principle of Least Privilege

Least privilege grants each user, process, or system exactly the access it needs to perform its function, and nothing beyond that, for exactly as long as the function requires it. The three subsections below cover minimum rights, how roles and just-in-time access implement them at scale, and the offboarding discipline that keeps privilege from silently accumulating.

3.2.1 Minimum Access Rights for Users, Systems, and Processes

Least privilege applies identically to a human engineer, a continuous integration and deployment (CI/CD) service account, and a smart contract's own admin function: each should hold the narrowest set of permissions its actual job requires, evaluated against what it does today, not what it

might plausibly need someday. A developer who occasionally needs production database access for debugging does not need standing production access; a deployment pipeline that only ever pushes to a staging environment does not need a mainnet deployer key sitting in its secrets store, even unused.

The habit that erodes least privilege is convenience-driven over-provisioning: granting broad access once to avoid a second request later, or reusing a single highly privileged service account across unrelated pipelines because creating a scoped one takes more setup time. Audit privileged accounts, human and non-human alike, on a fixed cadence against the question “what did this identity actually use in the last review period,” and revoke anything unused. A permission nobody has exercised in ninety days is not a convenience, it is unmonitored attack surface waiting for a credential leak to activate it.

3.2.2 Role-Based Access and Just-in-Time Privilege

Role-based access control (RBAC) assigns permissions to defined roles rather than to individuals, so access changes when someone moves roles instead of accumulating on top of whatever they already had. Combined with the just-in-time access from Section 2.3.4, RBAC becomes the mechanism least privilege is actually enforced through at scale: a role defines the maximum a function could ever need, and JIT grants the specific instance of that access only for the window it is used.

For a Web3 organization, define roles around the actual privileged actions in the environment rather than around job titles. “Multisig co-signer for treasury operations,” “mainnet deployer,” “production node administrator,” and “customer support with read access to identity-verification records” are roles with clearly bounded blast radii if compromised. A job title like “senior engineer” is not a role in this sense, because it implies nothing about which of those specific privileged actions the person actually performs. Map every standing credential in the environment back to one of these bounded roles during the asset identification step in Section 4.1; any credential that does not map cleanly to a role is a candidate for the offboarding review in the next section.

3.2.3 Offboarding and Credential Revocation

Offboarding is where least privilege most commonly fails in practice, because access accumulates gradually and continuously while revocation happens once, at a single point in time, and is easy to do incompletely. A departing employee’s corporate email might be disabled within the hour, while their name remains a signer on a Gnosis Safe, their key remains authorized on a validator host, or their personal device retains a copy of a shared password vault, none of which a standard HR-triggered deprovisioning workflow touches.

Build the offboarding checklist around the role mapping from Section 3.2.2, not around a generic IT list. For every role the departing person held, walk through every credential, key share, and standing session tied to that role, and confirm each is revoked, rotated, or reassigned before the person’s last working day, not after. Treat any multisig or key-shard role as requiring active

remediation (a signer swap, a re-key ceremony) rather than passive account disablement, since a departed co-signer's key material does not stop functioning just because their corporate account is deactivated. This handbook covers offboarding at a summary level; the Employee Lifecycle Security Handbook (03) provides the full checklist, and the Hiring, Remote Work, and Insider Threat Handbook (05) covers the insider-risk dimension of a departure gone wrong.

3.3 Need-to-Know Basis

Need-to-know restricts information, not just system access, to the people whose function actually requires it, independent of whether they hold a role that would technically permit broader access. The three subsections cover classifying information, handling and sharing it, and choosing channels that do not undermine the classification the moment sensitive content is typed into them.

3.3.1 Information Classification and Compartmentalization

Classification gives need-to-know a concrete boundary instead of leaving it to individual judgment about what feels sensitive. A simple four-tier scheme covers most Web3 organizations:

Tier	Definition	Web3 example
Public	Safe for anyone, published deliberately	Audited contract source, published tokenomics
Internal	Safe for all employees and contributors, not for the public	Internal roadmap, non-sensitive team discussion
Confidential	Restricted to a named team or role	Pre-disclosure audit findings, unreleased partnership terms
Restricted	Restricted to a named individual list, logged access	Key ceremony logistics, treasury signer identities, incident forensic detail

Compartmentalization applies the same classification horizontally, not only vertically: two people at the same seniority and the same clearance tier may still not need the same restricted information if their functions do not overlap. A security engineer investigating an active incident and a marketing lead both hold Confidential-tier clearance in the scheme above, but the marketing lead has no need to know the specific vulnerability under investigation before a coordinated disclosure. Classifying information without compartmentalizing it by function reduces to “trust everyone at this level with everything,” which defeats the purpose.

3.3.2 Data Handling and Sharing Policies

A classification scheme is only as strong as the handling rules attached to each tier: where it can be stored, how it can be transmitted, how long it is retained, and who can approve an exception. Write these rules to be checkable, not aspirational. “Restricted-tier documents are stored only in the encrypted vault, never in a general-purpose cloud drive, and any exception requires sign-off from the security lead, logged with a reason and an expiry date” is checkable; “handle sensitive data appropriately” is not.

Sharing policy deserves particular attention around pre-disclosure security findings and key-ceremony logistics, the two categories most likely to leak through convenience. An auditor's draft report shared over email to a personal address, a key-ceremony date mentioned in a public calendar invite, or an incident bridge call number posted to a semi-public chat channel are all violations of a data-handling policy even when no malicious intent is involved. Pair the written policy with periodic spot checks, confirming the actual location of last quarter's audit drafts matches the policy, rather than assuming a signed acknowledgment form guarantees compliance.

3.3.3 Secure Communication Channels for Sensitive Information

The channel a message travels through can undermine its classification regardless of how carefully the content itself was written. Consumer messaging apps and public chat servers, both common in Web3 team culture, are frequently unencrypted at the server side, retained indefinitely by the platform, and one misdirected message away from reaching an unintended audience; none of that is appropriate for Confidential- or Restricted-tier content as defined in Section 3.3.1.

The channel choice is also an attack surface in its own right. Nation-state actors tracked under [WA05, Fake Interview and Video Call Social Engineering](#), have repeatedly used fabricated recruiting processes to induce crypto-industry targets into installing malicious "video call" or "coding assessment" software, turning the act of joining a legitimate-seeming communication channel into the initial compromise (see the [FBI, CISA, and Treasury joint advisory on North Korean cyber actors targeting blockchain companies, AA22-108A](#)). Standardize on a small set of vetted, end-to-end encrypted channels for Confidential and Restricted communication, require any external call software to be installed only from an organization-managed source rather than a link sent by an external party regardless of how legitimate the sender appears, and treat "please install this to join the call" as a standing red flag requiring verification through a second channel.

3.4 Compartmentalization

Compartmentalization is need-to-know applied structurally, at the level of networks, environments, and accounts, so that a breach in one compartment does not automatically grant reach into the next. The three subsections cover network segmentation, environment isolation, and separating the accounts that access each domain.

3.4.1 Network and System Segmentation

Segmentation, introduced technically in Section 2.3.2 as microsegmentation, is a compartmentalization control at the network layer: dividing infrastructure into zones such that reaching one does not imply reaching another. Apply it along the boundaries that actually matter for a Web3 organization, not along a generic office-network diagram. Corporate productivity systems (email, chat, ticketing), engineering infrastructure (build pipelines, source control, staging environments), and signing or custody infrastructure (HSMs, validator hosts, multisig-adjacent tooling)

belong in at least three separate zones with explicit, minimal, and logged pathways between them, rather than one flat network where a phished laptop in the first zone has a plausible route to the third.

Segmentation earns its value from being tested, not merely diagrammed. A network diagram showing three zones is not evidence that a compromised host in zone one cannot reach zone three, only that it should not be able to. Periodic penetration testing or red-team exercises (Section 4.3.2) that specifically attempt lateral movement across the drawn boundaries convert an architectural intent into a verified control, and any successful crossing found during such a test is a finding to remediate before the next quarterly review, not a footnote.

3.4.2 Isolation of Critical Assets; Dev / Test / Prod Separation

Development, test, and production environments should be isolated to the point that a compromise or a mistake in one cannot reach another, and Web3 teams have a failure mode enterprise software teams rarely face: a testnet key or a staging deployer credential reused, deliberately or by copy-paste habit, in a mainnet context. A key generated “just for testing” and later promoted to a production role carries none of the custody controls a production key was supposed to have from creation, because it was never treated as sensitive when it was made.

Enforce isolation with structural, not merely procedural, separation: distinct cloud accounts or projects per environment (not just distinct namespaces within one account), distinct secrets stores, distinct build pipelines with no shared credentials between a staging deploy job and a mainnet deploy job, and a hard rule that no key or credential generated for test or staging purposes is ever promoted to production use, full stop, with a freshly generated and separately custodied credential required for the production role instead. Verify this rule technically where possible, for example automated scanning that flags any address seen on both testnet and mainnet under a production role, rather than relying on a developer to remember it under deadline pressure.

3.4.3 Separate Accounts and Access Methods per Security Domain

The same person should authenticate differently, and often through entirely separate accounts, depending on which security domain they are acting in. An engineer who has both a general corporate workspace account and a role requiring access to production infrastructure should not use the same account, the same password manager vault, or in many cases the same physical device for both, because a single compromised session, a malicious browser extension or a phished corporate login, should not cascade into the higher-value domain.

This is most visible, and most often skipped for convenience, in cloud and code hosting: one source-control organization spanning both a public marketing website repository and the private smart-contract repository, or one cloud account holding both a low-value marketing analytics workload and the treasury infrastructure, collapses two very different risk profiles into one blast radius. Split by account boundary, not just by access-policy configuration within a single account:

a misconfigured policy is a bug to fix, but a genuinely separate account is a compartment an attacker cannot cross through a policy misconfiguration alone, only through a second, independent compromise.

3.5 Continuous Monitoring and Improvement

The four principles above describe a defensive posture at a point in time; this one keeps that posture from decaying as the environment, the team, and the threat landscape change. The three subsections cover measuring control effectiveness, learning from what goes wrong (including what almost goes wrong), and deliberately updating controls rather than letting them fossilize.

3.5.1 Security Metrics and Effectiveness of Controls

A control that is deployed but never measured is a hypothesis, not a defense. Track metrics that show whether a control is actually working, not just whether it exists:

- Mean time to detect (MTTD) and mean time to respond (MTTR) for privileged-account anomalies
- Percentage of privileged accounts, human and service, with hardware-backed MFA enrolled and enforced
- Percentage of privileged access reviewed and re-justified in the last quarter, against the RBAC mapping in Section 3.2.2
- Click and credential-entry rate on internal phishing simulations, trended over time rather than compared once to an industry benchmark
- Time from an offboarding trigger to full credential revocation across every system in Section 3.2.3's checklist

Report these to the governance function from Section 2.2.1 (CSF's Govern), not only to the engineering team that implements the controls, so a declining metric becomes a resourcing conversation rather than unnoticed drift. A metric that never moves is often as informative as one that moves badly: a hardware-MFA-enrollment figure that has read 100 percent for two years without a single new hire or offboarding event logged against it usually indicates the metric stopped being measured, not that the control became permanently perfect.

3.5.2 Learning from Incidents and Near-Misses

An incident that reaches the point of financial loss is the expensive way to learn a lesson; a near-miss (a phishing email reported before anyone clicked it, a JIT access request denied because it looked anomalous, an alert that fired correctly and stopped an attack in progress) teaches the same lesson for free, if the organization captures it. Most teams have a process for the former and none for the latter, which means they only ever learn from their worst days.

Run every confirmed incident, and a sampled subset of near-misses, through a blameless post-mortem: what happened, what control caught or failed to catch it, and what would need to change for the same pattern to be caught earlier next time, with the explicit goal of fixing the

process rather than assigning fault to the individual who reported a phishing email late or nearly fell for it. Blame-oriented postmortems reliably suppress future reporting, the opposite of what a near-miss program needs to function; teams that punish the person who almost clicked the link train employees to stop reporting near-misses at all. Handbook 06, Incident Response, details the full postmortem process this section summarizes.

3.5.3 Updating Controls for New Threats and Technologies

Static controls age against a moving threat landscape. Two shifts illustrate why this section closes the chapter rather than standing alone. Generative artificial intelligence (AI) has made voice- and video-based impersonation, a deepfake call from a “CEO” or “investor” requesting an urgent wire or key action, cheap enough to attempt at scale, a pattern multiple threat-intelligence vendors documented targeting crypto and fintech firms through 2024 and 2025. Authentication technology has moved as well, with FIDO2 and passkey-based phishing-resistant authentication increasingly available to replace the SMS and app-based one-time codes that MFA-fatigue and SIM-swap attacks were designed against.

Treat control review as scheduled, not reactive: revisit the threat-actor and tactic list in Section 4.2.1 on a fixed cadence (quarterly is reasonable for a fast-moving industry) and ask explicitly whether any control assumes a threat model that no longer holds, for example a verbal-confirmation callback procedure that assumed a human voice could not be convincingly cloned. A phishing-resistant MFA rollout, a callback-verification policy updated to require a pre-shared challenge phrase rather than voice recognition alone, and a periodic review of which authentication methods remain phishable are all concrete outputs of this discipline, and each closes the loop back to the asset-and-threat cycle in Chapter 4.

4. Operational Implementation Process

The five-step OPSEC process traced back to NSDD-298 in Section 1.1 becomes, in this chapter, a repeatable operational cycle: identify what matters, understand who threatens it and how, find where it is weak, evaluate what that means for the business, and deploy controls sized to the answer. Run the cycle deliberately rather than once at program launch, because the continuous monitoring principle from Section 3.5 exists specifically to feed its results back into step one.

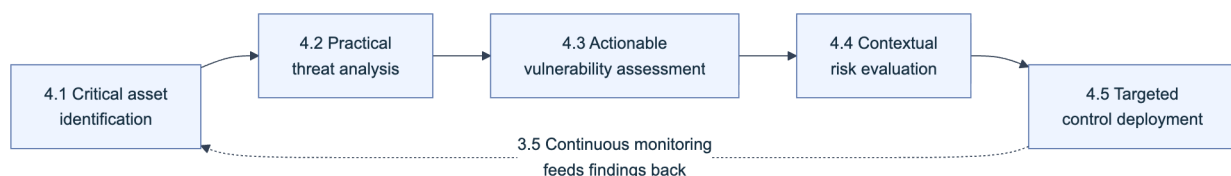


Figure 3. The operational implementation process as a continuous cycle, not a one-time project. Each pass narrows toward the highest-priority risk given the current asset inventory and threat landscape.

4.1 Critical Asset Identification

A control deployed against an asset nobody documented protects nothing measurable. This step builds the inventory every later step in the cycle, and every chapter in Part III, assumes already exists.

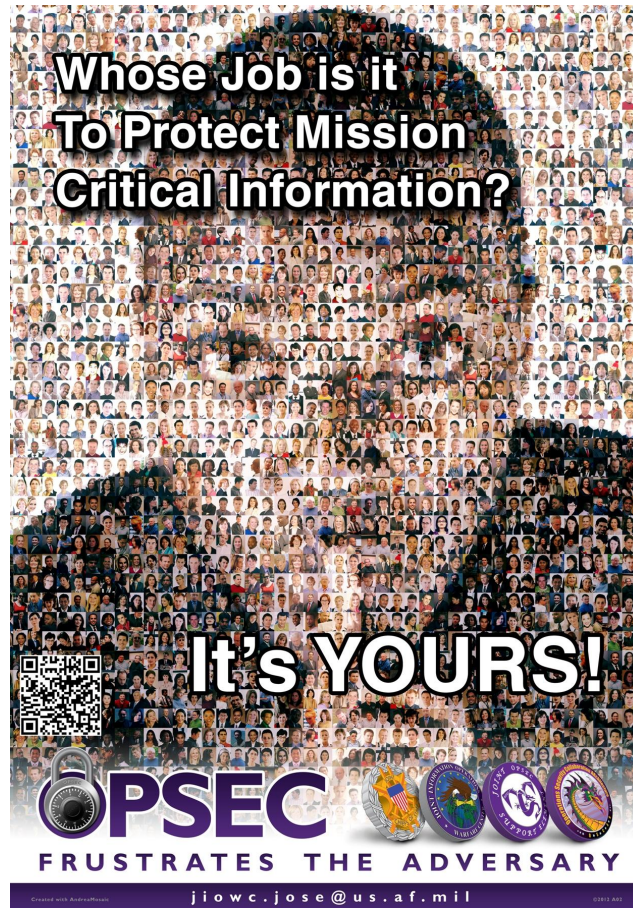


Figure. A real U.S. Air Force operational-security awareness artifact, not a decorative stock image. It distills this step to one question every contributor must answer: which mission-critical information would help an adversary if disclosed? Source: U.S. Air Force Medicine, OPSEC poster, public domain; credit requested by the source.

4.1.1 Mapping and Documenting Assets

Build an asset register that covers both traditional IT assets and the Web3-specific ones a generic template omits:

Asset class	Examples	Owner field
Cryptographic	Multisig signer keys, validator keys, deployer keys, HSM-held key shards	Named individual or role
On-chain	Deployed contracts, admin/upgrade roles, treasury addresses	Governance body or team
Infrastructure	RPC endpoints, validator hosts, cloud accounts, build pipelines	Engineering lead

Asset class	Examples	Owner field
Identity	Single sign-on accounts, source-control org membership, service accounts	IT/security
Information	Pre-disclosure audit reports, key-ceremony records, incident forensics	Classification owner (Section 3.3.1)

The register only earns its keep if it is actually current, which means assigning an update trigger to each class (a new deployment, a new hire, a signer rotation) rather than relying on an annual audit to catch drift. Cross-reference every entry against the RBAC role mapping from Section 3.2.2: an asset with no role and no accountable owner listed is itself a finding, not an administrative gap to fix later. Store the register at the Restricted classification tier from Section 3.3.1, since a complete asset map is exactly the reconnaissance an attacker would otherwise have to assemble themselves.

4.1.2 Prioritization by Impact of Compromise

Not every asset in the register deserves equal attention, and treating them as equal dilutes scarce security effort across low-value targets while high-value ones wait their turn. Prioritize by realistic impact of compromise, not by how technically interesting an asset is to defend. A treasury multisig holding the majority of a protocol's funds, a contract's upgrade-admin role, the organization's primary domain registrar account, and the credentials with publish rights to a package registry entry consumed by thousands of downstream projects are "crown jewel" assets whose compromise is catastrophic and often irreversible, and they should receive disproportionate attention relative to their count in the register.

A useful discipline is to ask, for each asset, "if this specific credential or key were compromised right now, what is the maximum realistic loss before anyone could intervene," and rank by that figure rather than by a generic severity label. This framing naturally surfaces assets a team underrated: a marketing team's social media account might not hold funds directly, but if it can post a "verified" malicious contract address to hundreds of thousands of followers, its realistic impact belongs near the top of the list, not the bottom.

4.2 Practical Threat Analysis

With assets ranked, the next step asks who actually wants each one and how they would try to get it, grounded in real observed tactics rather than generic "hackers" as an undifferentiated category.

4.2.1 Identifying Relevant Threat Actors and Tactics

Ground threat-actor identification in structured references rather than intuition. [MITRE ATT&CK](#) catalogs adversary tactics and techniques across an Enterprise matrix built from real-world observations, useful for the corporate-IT surfaces this handbook covers (initial access, credential

access, lateral movement). MITRE's newer framework, **AADAPT (Adversarial Actions in Digital Asset Payment Technologies)**, extends the same structured approach specifically to digital-asset and blockchain-payment adversary behavior, filling the gap ATT&CK's enterprise focus leaves for on-chain-specific tactics.

Four actor profiles cover most Web3 incidents, echoing the pattern used across this handbook series. Financially motivated criminals run phishing, drainer, and social-engineering campaigns at scale for direct profit. Nation-state actors, most visibly North Korea's Lazarus Group and its sub-clusters (tracked by various vendors under names such as TraderTraitor or Contagious Interview), pursue sanctioned-regime revenue through patient, well-resourced campaigns, including the fake job offer behind Ronin and the developer-infrastructure compromise behind the Bybit incident. Malicious or negligent insiders retain legitimate access that outlives its purpose (WA12). Opportunistic actors run mass scans for exposed secrets, unpinned credentials, or reused keys with no specific target in mind. Map each threat to the specific asset classes from Section 4.1 it realistically targets, since a nation-state campaign and an opportunistic scanner pose very different threats to the same treasury multisig.

4.2.2 Organization-Specific Threat Profiles

The generic actor list in Section 4.2.1 becomes useful only after it is weighted for your organization specifically. A custodial exchange, a DeFi protocol with a large public treasury, a DAO with pseudonymous contributors, and an infrastructure provider running validator nodes for other protocols each attract a different mix of the four actor profiles, and applying a one-size threat model across all four wastes effort on threats that do not apply while missing ones that do.

Build the profile from concrete, checkable factors: the total value the organization's assets could realistically expose (a rough proxy for financially motivated and nation-state attention), how publicly visible key holders and signers are (professional networking presence, conference speaking, source-control history tied to a real name, all of which feed the reconnaissance that made the Ronin fake job offer lure possible), employee headcount and hiring velocity (a fast-growing team is a wider surface for WA15-style fake-hiring infiltration), and the jurisdictions the team and its key holders operate in. Revisit the profile whenever any of these factors changes materially, not on a fixed calendar alone, since a funding round or a public treasury milestone can shift the threat profile faster than an annual review cycle catches.

4.3 Actionable Vulnerability Assessment

Threat analysis identifies who and how; this step finds where the organization is actually exposed to that how, validated rather than assumed.

4.3.1 Systematic Identification and Validation of Weaknesses

Systematic assessment combines self-directed reconnaissance with structured exercises, and both matter because they surface different classes of weakness. Run an OSINT self-assessment periodically: what can an outside party learn about your key holders, your infrastructure, and your

operational schedule from public sources alone? A professional profile naming a “Head of Treasury Operations,” a conference talk that names the cloud provider hosting production, or a source-control history that reveals a deployment cadence are exactly the reconnaissance an attacker like the one behind Ronin Bridge performs before ever sending a fake job offer. Finding it yourself first means you can decide deliberately what stays public.

Pair OSINT self-assessment with tabletop exercises that walk a specific scenario, a compromised co-signer’s laptop, a fake recruiter contact, a leaked internal message, through the organization’s actual controls step by step, asking at each point whether a real control would have caught it or whether the walkthrough is relying on an assumption nobody has tested. Validated weaknesses, meaning ones confirmed through an exercise rather than merely hypothesized, are the only ones that belong in the risk register in Section 4.4; an untested hypothesis is a good question, not yet a finding.

4.3.2 Practical Testing and Validation

Move from tabletop hypotheticals to executed tests wherever the organization’s risk tolerance and legal scope allow. Authorized phishing simulations measure the human-layer control from Section 3.1.2 directly, and are trended over time (per the metrics in Section 3.5.1) rather than treated as a one-time score. Physical security walkthroughs validate the physical layer: can an unbadged visitor reach a workstation with access to signing infrastructure, is a hardware wallet left unlocked in an unattended office. Access review audits validate least privilege directly: pull the actual current permission set for a sample of accounts and compare it against the RBAC role mapping from Section 3.2.2, rather than trusting that the mapping document and reality have stayed in sync.

Purple-team exercises, where an internal or contracted red team executes specific MITRE ATT&CK or AADAPT techniques while the defensive team observes what does and does not trigger a detection, validate the monitoring layer with the most rigor of any method here, because they test whether Section 3.5’s metrics would actually catch a real attempt rather than only a simulated phishing email. Schedule each test type on a cadence proportional to the asset criticality it validates: crown-jewel assets from Section 4.1.2 warrant more frequent and more adversarial testing than a low-priority marketing account.

4.4 Contextual Risk Evaluation

A validated weakness against a low-value asset and the same weakness against a crown-jewel asset are not the same risk. This step weighs each finding against the business context established in Sections 4.1 and 4.2 before anything moves to remediation.

4.4.1 Business-Context Risk Analysis

Risk, in the formulation this handbook uses throughout, is likelihood weighed against impact, and both terms need to be evaluated in the specific business context of the finding, not in the abstract. A weak password policy on a modest marketing operations budget account and the same

weak password policy on the infrastructure supporting a nine-figure treasury multisig are technically identical findings and completely different risks. Treating them with the same urgency either wastes effort on the former or, more dangerously, under-reacts to the latter because it reads as “just a password policy gap” on a generic severity scale.

Evaluate likelihood using the threat-actor weighting from Section 4.2.2 (does the profile that would realistically exploit this weakness actually target organizations like yours) and evaluate impact using the prioritization from Section 4.1.2 (what is the realistic maximum loss). A finding that scores high on a generic vulnerability scanner but low on both actual likelihood and actual impact for your specific organization is a lower priority than a finding a scanner would never flag but that maps directly onto the fake job offer pattern behind a \$620 million loss at Ronin.

4.4.2 Informed Decision-Making and Prioritization

Formalize the output of risk evaluation into a risk register that supports a real decision, not just a list of findings:

Risk	Likelihood	Impact	Priority	Owner	Decision
Standing admin key for treasury multisig	Medium	Critical	1	Security lead	Mitigate: move to JIT plus hardware signing
Unreviewed contractor access to staging	Low	Medium	3	Engineering manager	Accept, with quarterly review
Public professional profile naming key holders	High	High	2	Communications/HR	Mitigate: policy update

Four decisions are available for any risk: accept it explicitly (documented, with an owner and a review date, never silently), mitigate it (deploy a control that reduces likelihood or impact), transfer it (insurance, a third-party custody provider assuming some operational burden), or avoid it (stop doing the thing that creates the risk, such as retiring an unused legacy admin key entirely). The register’s value comes from forcing an explicit decision on every entry, rather than letting unranked findings accumulate in a backlog that never gets prioritized against the crown-jewel assets identified earlier in the cycle.

4.5 Targeted Control Deployment

The cycle's final step turns a prioritized risk register into deployed controls, and the two subsections address the two ways this step most commonly goes wrong: deploying generically instead of targeting the actual prioritized risk, and deploying so much friction that people route around the control entirely.

4.5.1 Implementing Controls for Prioritized Risks

Deploy controls against the risk register in the priority order it establishes, mapped back to the CSF Protect function from Section 2.2.2, and resist the temptation to deploy the easiest control first regardless of its rank; a quick win on a Priority-3 finding does not substitute for slow progress on Priority-1. For each control deployed, state explicitly which specific finding it addresses and how, so the deployment can be verified against that finding rather than checked off generically: “enabled hardware-backed MFA for the treasury multisig co-signer role” is verifiable against the finding it targets, while “improved authentication security” is not.

Verify effectiveness against the actual threat scenario the finding described, not only against a checklist item. If the prioritized risk was “a fake job offer could compromise a co-signer’s device,” echoing the Ronin pattern, the control is not complete once MFA is enabled; it is complete once the organization can show that a device compromised through that specific vector would still be blocked from initiating a signing action, which likely requires device attestation or a hardware-isolated signing flow in addition to MFA. Tie every deployed control back to the metric in Section 3.5.1 that will show whether it is working.

4.5.2 Balancing Security and Operational Friction

Every control this handbook recommends imposes some cost on the people who work within it, and a control that imposes more friction than the asset’s criticality justifies gets worked around rather than followed, worse than not deploying it at all because it creates a false sense of coverage. The clearest documented example of friction defeating a control is MFA fatigue: Uber’s [September 2022 breach](#), attributed to a Lapsus\$-affiliated actor, succeeded not because MFA was absent but because the attacker bombarded a contractor with repeated push-approval requests until, worn down, the contractor approved one, at which point a control that should have stopped the intrusion instead rubber-stamped it.

Right-size friction to the asset-criticality tiers established in Section 4.1.2. A crown-jewel asset (the treasury multisig, the production deployer role) justifies phishing-resistant, number-matching or hardware-token MFA and a deliberately, slightly inconvenient JIT approval flow, because the cost of that friction is small relative to the asset’s value, while a low-criticality internal tool does not justify the same friction and will simply drive shadow-IT workarounds if it receives it anyway. Ask, for every control under consideration, whether the friction it adds is proportional to the specific risk it mitigates, not whether more friction is generically safer; past a certain point, it measurably is not.

Key controls for Part 1

- Ground every OpSec decision in NIST SP 800-207/1800-35 (zero trust) and CSF 2.0 (Govern through Recover), so the program maps onto frameworks an auditor already recognizes.
- Apply the five principles, defense in depth, least privilege, need-to-know, compartmentalization, and continuous monitoring, as a checklist for every new control, not as background theory.
- Build and maintain a living critical-asset register (Section 4.1) that includes cryptographic and on-chain assets a generic IT template omits.
- Run the five-step implementation cycle continuously: identify, analyze threats, assess vulnerabilities, evaluate risk, deploy controls, then feed monitoring results back to step one.
- Size every control's friction to the asset-criticality tier it protects; both under-protecting crown-jewel assets and over-friction on low-value ones defeat the program.

Part II: Web3-Specific Considerations

Part 1 established operational security (OpSec) vocabulary and the Zero Trust baseline. This Part applies that baseline to what actually differs in Web3: a public ledger that erases the line between transparency and privacy, keys that are bearer instruments with no chargeback, and organizations that run on distributed contributors, SaaS tooling, and community channels an adversary can watch as easily as an employee can. Every chapter below pairs a Web3-specific challenge with the endpoint, workspace, authentication, communication, and travel controls a team uses to hold the line against it.

5. Web3 OpSec Challenges

This chapter names the structural properties of Web3 systems and teams that break assumptions a conventional OpSec program takes for granted, before later chapters turn each one into a control.

5.1 Transparency vs. Privacy

A public blockchain is a permanent, queryable record of every transaction a wallet has ever made. That transparency is a deliberate design goal: it is what lets anyone verify a protocol's solvency or a contract's behavior without trusting an intermediary. It is also a standing OpSec liability, because the same ledger that lets a user audit a protocol lets an adversary audit a team. Chain-analytics platforms such as Chainalysis Reactor, Arkham Intelligence, and Nansen cluster addresses by co-spend patterns, label them from exchange know-your-customer (KYC) data leaks and public disclosures, and can turn a single careless address reuse into a map of a team's entire treasury, personal holdings, and transaction timing. North Korea (Democratic People's Republic of Korea, DPRK)-linked actors used exactly this kind of reconnaissance ahead of the intrusions that made them responsible for \$1.34 billion of the \$2.2 billion in total cryptocurrency stolen in 2024, 61% of the year's total ([Chainalysis 2025 Crypto Crime Report](#)).

The practical response is not secrecy about the protocol, which would defeat the point of a public chain, but deliberate compartmentalization of what a given address reveals:

On-chain exposure by default	Mitigation
Treasury and admin wallet balances, visible to anyone	Fresh, single-purpose addresses; never fund an operational wallet directly from a KYC'd personal exchange account
Timing and gas patterns correlating a wallet to a time zone or individual	Batch or schedule transactions; avoid signing at the same time of day from the same wallet
Ethereum Name Service (ENS) or similar naming linking a wallet to a public identity	Use unlabelled addresses for treasury operations; reserve named addresses for public-facing donation or payment flows only
Counterparty graph exposed by every transfer	Route high-sensitivity transfers through processes reviewed for unintended linkage before signing

5.2 Decentralized Operations and Distributed Teams

A Web3 organization rarely has a network perimeter to defend, because it rarely has a single network. Core contributors, auditors, community moderators, and Decentralized Autonomous Organization (DAO) delegates work from personal devices, home and coworking networks, and their own choice of cloud accounts, spread across jurisdictions and time zones. Bring-your-own-device (BYOD) is the default operating mode, not an exception carved out for convenience. Every contributor's personal security hygiene becomes, in effect, part of the organization's attack surface, which is precisely the condition NIST Special Publication (SP) 800-207 Zero Trust Architecture was written for: identity and device posture, not network location, is what has to carry the trust decision ([NIST SP 800-207](#)).

The right controls depend on how the team is structured:

Coordination model	Primary OpSec risk	Baseline control
Fully doxxed core team	Individuals are identifiable, high-value targets for physical and social-engineering attacks	Personal threat modeling per role; travel and physical security controls (Chapter 10)
Pseudonymous contributors	Identity theft of the pseudonym; no reliable background check	Scoped access regardless of tenure; reputation and contribution history as a trust proxy, never a substitute for access control
Hybrid (doxxed leadership, pseudonymous contributors)	Impersonation of either group to reach the other	Out-of-band identity verification (9.1) before any access grant or fund request crosses the doxxed/pseudonymous boundary

5.3 Cryptocurrency and Digital Asset Security

Cryptographic keys in Web3 are bearer instruments: whoever can produce a valid signature controls the asset, permanently and irreversibly, with no chargeback and no help desk that can undo a transfer. Chainalysis attributes 43.8% of the \$2.2 billion stolen across the industry in 2024 to private key compromise, the single largest cause category tracked ([Chainalysis 2025 Crypto Crime Report](#)). Everything in this section exists to shrink that number for your organization specifically.

5.3.1 Cold vs. Hot and Key Compartmentalization

A hot wallet holds a private key on a network-connected device so it can sign transactions with minimal friction; a cold wallet keeps the key generated and stored on a device that never touches the internet, commonly a hardware wallet or an air-gapped machine, and signs by exporting an unsigned transaction, moving it out of band, and returning only the signature. Hot wallets trade security for availability and belong only where availability is the point: paying gas, funding a bot, settling small operational transfers. Cold storage trades availability for security and belongs wherever the loss of the asset would be organizationally significant.

Compartmentalization is the discipline that makes the hot/cold split actually work: one key, one purpose, and no key spans a trust boundary. A deployment key should never also be a treasury key. A testnet key should never share a seed with anything holding mainnet value. A departing contractor's access should map to keys that can be rotated in isolation, not a shared wallet the whole team uses.

Wallet tier	Typical balance ceiling	Storage medium	Signing cadence
Operational hot wallet	Days of gas and routine transfer volume	Software wallet on a dedicated machine (6.2)	Frequent, often automated within scoped limits
Working treasury	Weeks of runway	Multisig or MPC threshold wallet (5.3.2)	Scheduled, human-reviewed
Cold reserve	Majority of holdings	Air-gapped hardware wallet, geographically distributed backups	Rare, manual, dual-control

5.3.2 Multisig and Threshold Key Management

Multi-signature (multisig) wallets require M of N designated signers to approve a transaction on-chain, most commonly through the [Safe](#) smart contract standard. Threshold signature schemes (TSS) and multi-party computation (MPC) wallets achieve a similar M-of-N property off-chain: no single device ever assembles a complete private key, only a share of one, and a threshold of shares cooperates to produce a single signature. Both approaches remove the single point of failure a lone hot key represents, but both share an exploitable weak point: each signer still has to look at a proposed transaction and decide whether to approve it, and that decision happens on an ordinary computer running an ordinary browser.

The February 2025 Bybit incident is the largest cryptocurrency theft on record and the clearest illustration of that weak point. Attackers compromised infrastructure at Safe{Wallet}, the third-party interface Bybit's signers used to review and approve multisig transactions, and manipulated what those signers saw on screen while a different, malicious transaction was actually being signed. Bybit's signers approved what appeared to be a routine cold-wallet transfer; the transaction that executed reassigned control of the wallet and drained roughly 400,000 Ether worth \$1.4 to \$1.5 billion. The Federal Bureau of Investigation (FBI) attributed the intrusion to North Korea's Lazarus Group, tracked under the alias TraderTraitor ([Bybit hack summary](#), [Wikipedia](#)).

Radiant Capital suffered the same class of attack eight months earlier, at smaller scale but with the mechanism laid bare. The protocol used a 3-of-11 Safe multisig. Malware planted on three signers' devices intercepted the Safe front end so that it displayed the legitimate, intended transaction on screen while silently substituting a malicious transaction for the signers' hardware wallets to actually approve, exploiting the fact that hardware wallets often show only a truncated hash or a generic confirmation rather than full transaction details, a failure mode known as blind signing. The team had grown accustomed to occasional transaction failures as routine noise, which let the attacker collect signatures without triggering suspicion. The stolen signatures handed control of Radiant's Pool Provider contract to the attacker, who upgraded the lending pools to malicious versions and drained approximately \$53 million ([Halborn, Radiant Capital hack explained](#)).

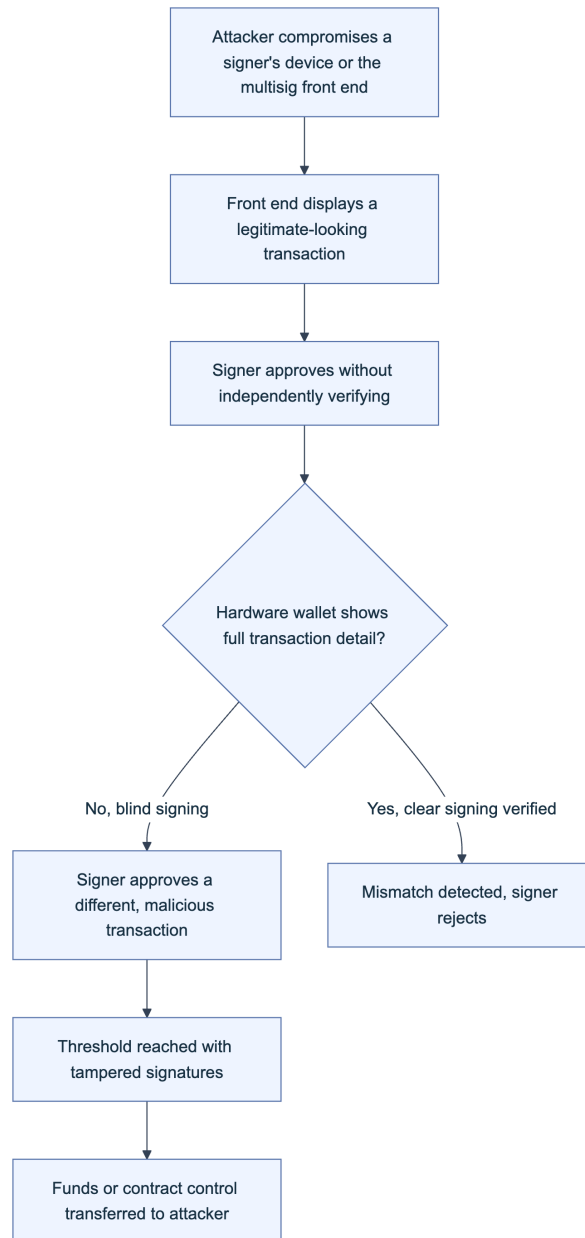


Figure 1. The blind-signing attack chain behind the Bybit and Radiant Capital losses. The single control that breaks the chain is a signer independently verifying the transaction on a device the attacker does not control, at the point marked D.

Defenses follow directly from where the chain breaks: require every signer to verify the recipient address, amount, and calldata on the hardware wallet's own display, never trusting the software front end alone; adopt clear-signing standards and independent transaction-simulation tools that decode calldata into human-readable terms rather than a raw hash; confirm the transaction hash through a channel the front end does not control; and treat any signer's report of "the transaction failed, trying again" as an incident-worthy anomaly rather than routine friction, exactly the assumption Radiant's team lost to.

5.4 Smart Contract and Protocol Exposure

A contract's code can pass a rigorous audit and remain one phished operator away from compromise if its administrative controls are not treated as OpSec-critical assets in their own right. Upgrade authority, pause functions, and parameter-setting roles are, from an operational security standpoint, just another set of keys, and every control this Part describes for treasury keys applies equally to them. The Smart Contract Security Verification Standard's access-control requirements define what "correct" administrative control looks like in code; this handbook is concerned with keeping the humans and machines that hold those admin keys from being the reason the code's guarantees fail in practice ([SCSVS](#)).

Operational controls that belong alongside the code-level ones:

- Admin keys held by a multisig or timelock, never a single externally owned account (EOA), with the same signer hygiene as 5.3.2.
- A timelock delay long enough for the community or an internal review process to react to a malicious or erroneous proposed change before it executes.
- Monitoring on every privileged function call, alerting the security team the moment an admin function fires, not after a post-incident review discovers it.
- Documented, rehearsed use of pause functionality, so the first time it is invoked is not during a live incident.
- Regular verification that on-chain admin roles match the organization's actual current personnel; a departed contractor's still-active admin key is a live vulnerability, not a paperwork gap (cross-reference Part 1, 3.2.3, offboarding and credential revocation).

5.5 Community and Open-Source Dynamics

Open-source code and public community channels are core to how Web3 projects build legitimacy, and both give an adversary a research environment most conventional targets never offer. Attackers can study a codebase's history for patterns before submitting a plausible pull request, observe how a support channel actually operates to craft a convincing impersonation, and identify which community moderators or "official" bot accounts hold elevated trust worth compromising. Discord and Telegram channels see impersonation of core developers, fake support accounts directing users to "verify your wallet" phishing sites, and compromised moderator accounts posting malicious "airdrop claim" links to an audience primed to trust them. On the code side, a patient, trust-building contribution history is the same playbook that produced the XZ Utils backdoor, where an attacker contributed to a project for more than two years to earn commit access before inserting a backdoor into an official release ([Trail of Bits on XZ Utils](#)).

Baseline controls: default new community members to a read-only role until a manual or time-based promotion; require at least two independent reviewers on any pull request touching deployment, signing, or dependency-manifest files, regardless of the contributor's history; never grant an external contributor merge rights into a branch that triggers a build or deploy pipeline;

and publish an official, cryptographically verifiable channel (a signed announcement key, a pinned message) for security-relevant communications so community members have a way to distinguish real alerts from impersonation.

5.6 Entry-Point Centralization (DNS, Front-Ends, RPC)

A protocol can be fully decentralized at the contract layer and still depend on a small number of centralized chokepoints for users to ever reach it: the Domain Name System (DNS) name in the address bar, the front-end application that renders the interface, and the Remote Procedure Call (RPC) endpoint that actually broadcasts a signed transaction to the network. Compromising any one of these lets an attacker manipulate what a user sees and signs without touching a single line of audited contract code, the exact pattern the OWASP Web3 Attack Vectors Top 15 tracks as supply-chain compromise (WA02) and User Interface and User Experience (UI/UX) deception (WA06) ([OWASP Web3 Attack Vectors Top 15](#)). Front-end DNS hijacks that redirect users toward a malicious clone site, and malicious or spoofed RPC endpoints that lie about balances or silently rewrite transaction data before a wallet ever sees it, have both recurred across DeFi protocols since 2022; the deep technical controls for the DNS and front-end layers are covered in the CDN and Front-End Supply Chain Handbook (01) and the DNS and Hosting Security Handbook (02).

From an OpSec standpoint, the discipline is treating every one of these chokepoints as a critical asset with a named owner, not an implementation detail:

Entry point	Single point of failure	OpSec control
Domain registrar account	Domain hijack redirects all users instantly	Registrar lock, dedicated hardened login, no shared credentials
Content delivery network (CDN) / hosting account	Malicious front-end served to every visitor	Multi-person approval for deploys, subresource integrity on scripts (Handbook 01)
RPC provider account and default endpoints	Transaction manipulation or censorship before broadcast	Verified default RPC list, monitoring for unexpected endpoint changes prompted in-wallet
Mobile app store listing	Fake or hijacked listing distributes malicious app	Verified publisher account, MFA on the developer console (Chapter 8)

6. Endpoint and Device Security

Every control in Chapter 5 ultimately executes on a physical device. This chapter sets the hardening baseline for those devices, then layers additional controls onto the subset of machines that sign or deploy anything of value.

6.1 Workstation Hardening

Baseline workstation hardening is not Web3-specific, but it is the floor every other control in this handbook stands on: a compromised operating system defeats a hardware wallet just as easily as it defeats a password manager. Apply full-disk encryption on every device that ever touches organizational data or keys; keep the operating system and browser on automatic security updates; run endpoint detection and response (EDR) tooling that alerts a central team rather than only the local user; disable USB auto-run and restrict removable media where feasible; remove local administrator rights from standard user accounts; and set a short, enforced screen-lock timeout. The [CIS Benchmarks](#) provide configuration baselines per operating system that a small team can adopt without building a hardening standard from scratch.

Workstation hardening checklist

- ☐ Full-disk encryption enabled and verified, not just assumed
- ☐ OS and browser patched automatically; no manually-deferred update queue
- ☐ EDR agent installed, reporting to a monitored console
- ☐ Local admin rights removed from daily-use accounts
- ☐ Screen lock at 5 minutes or less, biometric plus passphrase fallback
- ☐ USB auto-run disabled; unknown removable media never plugged in

6.2 Secure Development and Signing Machines

A machine that deploys contracts, holds infrastructure credentials, or signs a multisig transaction carries materially more risk than a daily-use laptop and deserves a materially higher trust tier, isolated from everyday browsing, email, and messaging. This is precisely the gap North Korea-aligned actors exploited against Radiant Capital: signers used devices that also handled Telegram and file attachments, and a malicious file delivered through that ordinary channel became the beachhead for a \$53 million theft (5.3.2). A dedicated signing machine, used for nothing else, removes the exposure a general-purpose device accumulates simply by existing on the internet.

Property	Daily-driver laptop	Dedicated signing machine
Email and messaging clients	Installed, actively used	None installed
Browser extensions	Whatever the user has accumulated	Only what signing requires, reviewed
Software installation	Unrestricted	Allowlisted, reviewed before install
Network exposure	Full internet access	Restricted to required endpoints where feasible
Session persistence	Long-lived, always logged in	Rebuilt from a clean image between sensitive sessions where operationally possible

Where a fully dedicated physical machine is not feasible for a smaller team, a hardened virtual machine reserved exclusively for signing, ideally rebuilt from a known-clean snapshot on a regular cadence, delivers most of the same isolation.

6.3 Browser Security and Browsing Safely

The browser is the execution environment for the wallet itself: extension-based wallets inject page-accessible interfaces, and a malicious or compromised extension, a malvertised search result, or a fake wallet-connection popup can intercept a transaction before it ever reaches the hardware wallet for confirmation. Malicious browser extensions that harvest clipboard contents (to swap a copied wallet address for an attacker's) or capture screenshots have repeatedly been distributed through official extension stores under innocuous-sounding names, which makes extension count and permission scope, not just extension source, a control worth enforcing.

Practical controls: maintain a dedicated browser profile used only for wallet interaction, with the minimum set of extensions the workflow actually requires; review each extension's requested permissions before installing and again on any update; verify URLs character by character before connecting a wallet, since homoglyph domains (substituting visually similar Unicode characters) remain a live phishing technique; and use a transaction-simulation or decoding tool that renders calldata in human terms before a signature is requested, so a malicious contract interaction is visible before approval rather than discovered afterward.

6.4 Hardware Wallet and Key Storage Hygiene

A hardware wallet keeps the private key inside a secure element that never exposes it to a network-connected computer, signing by displaying the transaction on its own screen and requiring a physical confirmation. That security model depends entirely on two things the operator controls: buying the device through a channel that guarantees it has not been tampered with, and actually reading what the device's screen shows before pressing confirm. Devices purchased secondhand or through unofficial resellers have been documented arriving with a pre-seeded recovery phrase, letting the seller drain funds the moment the “new” wallet is funded; buy only direct from the manufacturer or an authorized reseller, verify the tamper-evident packaging, and generate the seed on the device yourself rather than trusting a pre-printed card in the box.

The [Ledger Recover](#) key-backup service, announced in 2023, drew industry scrutiny precisely because it proposed extracting key shares from the secure element for an optional recovery service, prompting a public debate about what “the key never leaves the device” actually guarantees when a vendor can ship a firmware update changing that behavior. The lesson generalizes: a hardware wallet's security model rests on vendor firmware integrity as much as on the secure element itself, so verify firmware signatures before updating and never install firmware from a link received in an unsolicited message.

Key storage checklist

- ☐ Seed phrase recorded on a fire- and water-resistant metal backup, never photographed or stored in cloud notes
- ☐ Seed phrase split (Shamir's Secret Sharing or SLIP-39) or a hidden passphrase wallet used for the highest-value holdings
- ☐ PIN set to the device maximum length the workflow tolerates
- ☐ Device and its only backup never traveling together (10.2)
- ☐ Firmware updates verified against the vendor's published signature before installing

6.5 Separation of Hot and Cold Operations

Chapter 5's hot/cold split (5.3.1) only holds if the operational workflow enforces it every day, not just at wallet-creation time. The rule that makes it durable: a hot wallet is topped up from cold storage on a schedule or by manual review, never automatically, and no automation script, deployment pipeline, or bot ever holds a key with cold-tier value behind it.

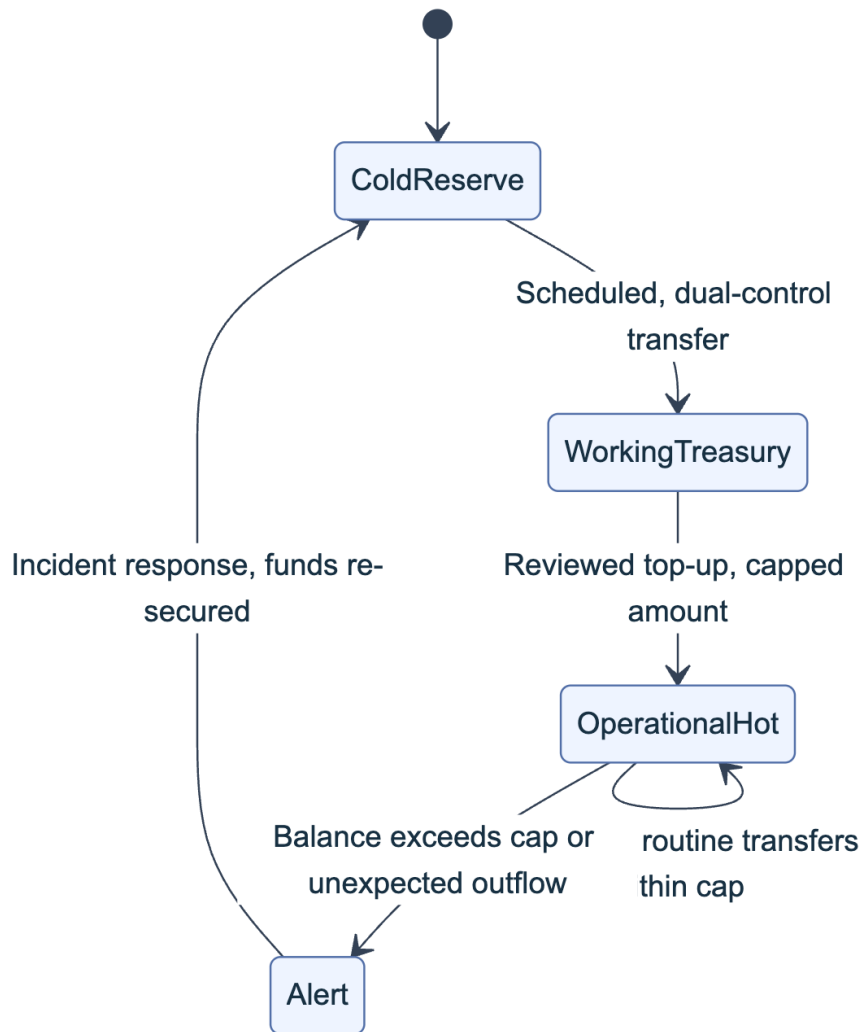


Figure 2. Fund flow across wallet tiers. Value moves down toward operational use through reviewed, capped steps; an alert on any breach of the operational cap routes the response back to cold reserve rather than compounding exposure in the hot tier.

7. Workspace and Productivity Security

Web3 teams run day-to-day operations, ironically, on the same Software-as-a-Service (SaaS) productivity stack every other organization uses. That stack is where an attacker often gets the first foothold, well before ever touching a wallet.

7.1 Google Workspace and Collaboration Platform Hardening

Google Workspace, or an equivalent SaaS identity platform such as Microsoft 365, functions as the organization's identity provider (IdP) for email, documents, and often single sign-on into other tools, which makes its admin console a single point of compromise worth defending at the tenant level. Enforce two-step verification for every account with no exceptions, enroll privileged administrators in Google's Advanced Protection Program for phishing-resistant hardware-key-only authentication, restrict third-party OAuth application access to an explicit allowlist so a malicious "connect your Google account" app cannot silently harvest Drive or Calendar data, and review the admin console's audit log on a fixed cadence rather than only after something looks wrong ([Google Workspace security checklist](#)).

Workspace admin hardening checklist

- ☐ Org-wide two-step verification enforced, no per-user opt-out
- ☐ Advanced Protection Program (or equivalent) enrolled for admins and treasury-adjacent roles
- ☐ Third-party OAuth apps allowlisted, not open by default
- ☐ Legacy protocol access (IMAP/POP, basic auth) disabled
- ☐ Admin console audit log reviewed on a fixed weekly or monthly cadence

7.2 Password Management and Robustness

A team password manager replaces the alternative every organization defaults to without one: password reuse across dozens of SaaS tools, and credentials shared over Slack or email in plaintext. A shared-vault model lets service credentials (a shared exchange sub-account login, a monitoring dashboard) be granted, rotated, and revoked per person without ever appearing in a chat log. [NIST SP 800-63-4](#), finalized on 31 July 2025, continues the direction of prior guidance: favor length over enforced complexity, screen new passwords against known-breach corpora, and drop mandatory periodic rotation absent specific evidence of compromise, since forced rotation on a fixed schedule measurably pushes users toward weaker, more predictable passwords.

Policy element	NIST SP 800-63-4-aligned guidance
Minimum length	At least 8 characters for user-chosen passwords, 15+ recommended for higher-assurance accounts
Complexity rules	Not required; length and breach screening matter more
Rotation	Only on evidence of compromise, not a fixed calendar
Storage	Password manager with vault-level access control, never plaintext or spreadsheet

7.3 Multi-Factor Authentication (MFA) Hardening

This section covers configuring MFA specifically within the productivity and collaboration stack; the standards-level comparison of MFA methods and their phishing resistance is in 8.1. Enforce a phishing-resistant method (FIDO2/WebAuthn security key or platform passkey) organization-wide inside the Google Workspace, Okta, or Azure AD admin console, and explicitly disable SMS or voice call as a fallback factor, since a fallback is only as strong as its weakest option regardless of what the primary factor is. Alert on any change to a user's registered MFA method: an attacker adding a new authenticator to a compromised account is a well-established persistence step, and catching it in near real time turns what would be a silent takeover into a contained incident.

8. Authentication and Access

Where Chapter 7 hardened the productivity stack specifically, this chapter defines the authentication and access-control principles that apply across every system a Web3 team touches, from cloud infrastructure to the signing tools covered in Chapter 6.

8.1 Multi-Factor Authentication (MFA)

Multi-factor authentication (MFA), sometimes called two-factor authentication (2FA), combines two or more independent factors: something the user knows (a password), something the user has (a security key or authenticator app), or something the user is (biometrics). Factors are not interchangeable in the security they provide. The Cybersecurity and Infrastructure Security Agency (CISA) ranks methods explicitly by phishing resistance and recommends FIDO2/WebAuthn hardware security keys and passkeys as the strongest widely available option, because the cryptographic challenge-response is bound to the specific origin requesting it and cannot be replayed against a look-alike site (CISA, [phishing-resistant MFA](#); [FIDO Alliance passkeys](#)).

Factor type	Phishing resistance	Primary weakness
FIDO2/WebAuthn security key or passkey	High	Device loss without a registered backup key
Time-based one-time password (TOTP) app	Medium	Code can be relayed in real time by a phishing proxy
Push notification approval	Medium-low	Push-bombing / MFA fatigue: repeated prompts until the user approves out of annoyance
SMS or voice call one-time password	Low	SIM swap, SS7 interception, no cryptographic binding to the site

8.2 Password and Credential Management

Where 7.2 covered human passwords for SaaS logins, this section covers the fuller credential lifecycle: service accounts, application programming interface (API) keys, CI/CD pipeline secrets, and cloud provider root credentials, none of which belong in a human-facing password manager alone. Route machine credentials through a secrets manager (a cloud provider's key management service, HashiCorp Vault, or an equivalent) with per-service scoping and access logging, never a shared root or admin login used by multiple humans or systems. Trigger rotation on role change and offboarding as a hard gate, not a best-effort follow-up (cross-reference Part 1, 3.2.3): a credential that outlives the person who needed it is a standing liability with no offsetting benefit.

8.3 Hardware-Backed Authentication

Hardware-backed authentication anchors a credential to a physical device's secure element rather than to something that can be copied, phished, or brute-forced remotely. FIDO2 security keys (YubiKey and equivalents) and passkeys, whether device-bound or synced across a vendor's ecosystem, are the user-facing form of this; for developers, the same principle extends to signing Secure Shell (SSH) and code-commit keys with a hardware-backed key rather than a plain file on disk.

```
# Generate an SSH key backed by a FIDO2 security key (resident, discoverable)
ssh-keygen -t ed25519-sk -O resident -O verify-required -C "signing-machine-key"

# Configure git to require a hardware-backed signature on every commit
git config --global gpg.format ssh
git config --global user.signingkey ~/.ssh/id_ed25519_sk.pub
git config --global commit.gpgsign true
```

A hardware-backed commit signature means a stolen laptop's disk image alone cannot forge a commit as a trusted maintainer; the physical key still has to be present and touched. This closes exactly the kind of maintainer-impersonation gap that made the XZ Utils backdoor possible over a long enough timeline (5.5).

8.4 Identity and Access Management (IAM) in Web3 Contexts

Traditional identity and access management (IAM), single sign-on (SSO) through Security Assertion Markup Language (SAML) or OpenID Connect (OIDC) into internal SaaS tools, gets a Web3-native counterpart in wallet-based authentication. Sign-In with Ethereum (EIP-4361) standardizes how an Ethereum account authenticates to an off-chain service by signing a structured message instead of submitting a password, replacing a phishable credential with a cryptographic proof of key possession. That trade shifts the entire trust anchor onto wallet-key hygiene: every control in Chapter 6 now doubles as an IAM control the moment a service accepts wallet-based login. Account-abstraction session keys (EIP-4337) extend the model further, letting a user grant a dApp a time-boxed, function-scoped signing authority instead of exposing the master key to every interaction, the Web3 analogue of a short-lived, least-privilege access token.

Traditional IAM control	Web3-native analogue
SSO via SAML/OIDC	Sign-In with Ethereum (EIP-4361)
Scoped OAuth access token	Account-abstraction session key (EIP-4337)
Role-based access control	Multisig threshold and per-role signer sets (5.3.2)
Just-in-time privilege elevation	Time-boxed session keys, revoked automatically on expiry

9. Communication and Collaboration Security

Communication channels are where most Web3 intrusions actually begin, well upstream of any wallet or contract. This chapter treats channel selection, key handling, and third-party integrations as first-class OpSec surfaces.

9.1 Secure Channels for Internal and External Comms

Match channel choice to sensitivity, and assume public community channels are fully adversary-monitored, because they are. A public Discord or Telegram server is a research environment for an attacker, not a private space; internal team channels on Slack or Signal are better but remain SaaS-hosted and phishable; the highest-sensitivity conversations, anything touching keys, signing, or fund movement, deserve an out-of-band, previously-established channel with disappearing messages, never the default team chat.

North Korea-aligned actors run a well-documented playbook against exactly this boundary, publicly tracked as Slow Pisces, Jade Sleet, or TraderTraitor: pose as a recruiter or collaborator on LinkedIn, move the conversation to a “coding assessment” hosted in a plausible-looking GitHub repository, or to a video call where the target is asked to “fix your camera or microphone driver” before the call can proceed, and use that pretext to deliver malware disguised as a routine software fix ([Unit 42](#), [Slow Pisces](#)).

Communication channel checklist

- ☐ Never act on a funds- or credential-related request received through a single channel; verify through a second, previously-established channel first
- ☐ Never install software, drivers, or “fixes” prompted mid-conversation or mid-call
- ☐ Treat unsolicited recruiter or collaboration outreach touching crypto work as a phishing vector by default until verified
- ☐ Reserve the highest-sensitivity channel for key and signing conversations exclusively

9.2 Handling of Keys, Mnemonics, and Sensitive Data

One rule sits above every other control in this section: seed phrases, private keys, and multisig approval requests are never transmitted through any chat, email, or file-sharing channel, encrypted or not. This is not a theoretical caution; it is the literal mechanism behind the \$53 million Radiant Capital loss, where a malicious file delivered as a routine “contractor deliverable” over Telegram was the entry point for the malware that ultimately hijacked signer approvals (5.3.2). A file transfer channel is not a secure enclave, and no amount of end-to-end encryption on the channel protects against a malicious payload the recipient is asked to open.

For any high-value transaction, apply a standing out-of-band verification protocol before signing: confirm the request through a second communication channel, verify the requester’s identity live rather than trusting a text or chat handle, and independently verify the transaction hash and calldata on the hardware wallet’s own display, never on the trust of a screen-shared or described transaction. This closes the exact gap the blind-signing diagram in 5.3.2 identifies as the attack’s decision point.

Data type	Acceptable channel	Never acceptable
Seed phrase, private key	Never transmitted; generated and stored offline only	Any chat, email, screenshot, or cloud note
Multisig transaction approval request	In-app notification plus independent hardware-wallet verification	A described or screen-shared transaction with no independent verification
Routine operational credentials	Team password manager, per-user scoped access	Plaintext in chat, shared documents

9.3 Collaboration Tools and Workspace Security

Beyond the identity-provider hardening in 7.1, the bot, webhook, and OAuth integration layer of Discord, Slack, Notion, and GitHub is its own attack surface: a third-party bot requesting broad OAuth scopes, a webhook URL accidentally committed to a public repository, or a GitHub Action with write access to protected branches or repository secrets all give an attacker a path that never touches a password. The tj-actions/changed-files compromise of March 2025, which altered a GitHub Action used across more than 23,000 repositories to dump CI runner memory into logs, is the general pattern this section defends against, covered in full in the CDN and Front-End Supply Chain Handbook (01).

Review and prune app and bot integrations on a quarterly cadence rather than leaving them to accumulate indefinitely; scope GitHub organization tokens to the minimum required repositories and permissions rather than an org-wide grant; and require branch protection with mandatory reviews on any repository that touches deployment scripts, signing tooling, or infrastructure-as-code, so a compromised or malicious contribution cannot reach production through a single approval.

10. Travel and Remote Work

Physical mobility introduces a threat model conventional OpSec programs often underweight: device seizure, jurisdiction-dependent search authority, and the fact that a stolen laptop belonging to a multisig signer is a direct path to theft, not merely a data-exposure incident.

10.1 Travel Security Overview

Not every traveler carries the same risk, and controls should scale with what a compromised device or seized traveler actually exposes.

Traveler risk tier	Exposure if device is seized or compromised	Baseline control
Non-signer employee	Internal documents, SaaS session tokens	Standard device hardening (6.1), session revocation on report
Multisig signer	Direct path to fund theft if signing credentials are reachable	Temporary signer-role delegation before travel; clean loaner device (10.2)
Founder or treasury-access role	Full organizational exposure, highest-value target	Dedicated travel device with zero standing production access; pre-cleared incident response plan

10.2 Pre-Travel Checklist and Device Hygiene

Preparation before departure matters more than any control applied at the border, because by the time a device is in an agent's hands, the only remaining question is what it contains.

Pre-travel checklist

- ☐ Travel with a clean loaner device wiped of production access, not the primary daily-use machine, wherever the role allows it
- ☐ Full-disk encryption with a strong passphrase, not biometric-unlock alone; a passphrase carries stronger legal protection against compelled disclosure in many jurisdictions than a fingerprint or face unlock ([EFF, Digital Privacy at the U.S. Border](#))
- ☐ Temporarily delegate or revoke standing multisig signing rights before departure if the role and threshold configuration allow it
- ☐ Never carry a hardware wallet and its only seed-phrase backup on the same trip
- ☐ Register device serial numbers with IT for remote-wipe capability before leaving
- ☐ Power down devices fully before reaching any checkpoint; a powered-off device resists a wider class of extraction techniques than one merely locked

10.3 Secure Access While Traveling

Public and hotel networks are hostile by default; a virtual private network (VPN) raises the bar but is not sufficient on its own, since it protects the network path without addressing a compromised endpoint or a malicious captive portal. Never sign a multisig transaction from an unmanaged network or an untrusted device, prefer a mobile hotspot on a carrier account not vulnerable to SIM-swap targeting over public wifi, and disable auto-join wifi and Bluetooth so a device does not silently associate with a spoofed access point mimicking a known network name.

10.4 Remote Work and Overseas Consultation Considerations

Remote hiring and overseas consulting arrangements are also where the DPRK IT-worker infiltration scheme most directly threatens a Web3 team. North Korean operatives have posed as remote contractors and full-time employees, using stolen, fabricated, or purchased identities and, in documented cases, real-time video manipulation, to gain legitimate insider access to codebases, infrastructure, and in some instances treasury-adjacent operations; Chainalysis links a portion of 2024's \$1.34 billion in DPRK-attributed cryptocurrency theft to exactly this infiltration vector ([Chainalysis 2025 Crypto Crime Report](#); [MITRE AADAPT](#)). The personnel-side depth on this threat, including hiring-pipeline controls, belongs to the Hiring, Remote Work, and Insider Threat Handbook (05); the OpSec-specific mitigation is scoping what a new remote hire or consultant can touch until trust is independently established.

Remote hire and consultant vetting checklist

- ☐ Live, unscripted video interview, not a resume and asynchronous messaging alone
- ☐ Identity verification beyond a self-reported name and LinkedIn profile where feasible
- ☐ Payment-method and banking geography checked for consistency with the claimed location
- ☐ No production, signing, or treasury-adjacent access granted at onboarding; access staged and expanded only as trust is demonstrated over time
- ☐ Pseudonymous contributors held to the same staged-access model as named hires, since a background check is unavailable for either group in practice

Key controls for Part 2

- Compartmentalize keys by purpose and value tier (5.3.1), and require independent, on-device transaction verification for every multisig or threshold signature (5.3.2) to close the blind-signing gap behind the Bybit and Radiant Capital losses.
- Isolate signing and deployment machines from daily-use browsing, email, and messaging (6.2), and buy hardware wallets only through verified channels with vendor-signed firmware (6.4).
- Enforce phishing-resistant MFA (FIDO2/WebAuthn) organization-wide, with SMS explicitly disabled as a fallback (7.3, 8.1).
- Never transmit seed phrases, private keys, or approval requests through any chat, email, or file channel, and verify high-value transactions through a second, previously-established channel (9.2).
- Treat DPRK-linked fake-recruiter and fake-contractor campaigns as a standing threat to communication channels and hiring pipelines alike (9.1, 10.4), and stage access for every new remote hire or consultant regardless of how the relationship began.

Part III: Control Domains

Part 1 fixed the vocabulary and Part 2 walked the Web3-specific attack surface. This part turns that surface into a control catalog, organized into four domains so that no protective measure floats without a home and no domain is defended while its neighbors are ignored. It closes with a repeatable process for choosing, deploying, and revising controls, because a catalog without a selection method just becomes a compliance checklist nobody re-examines.

11. Control Domains Overview

Every control a Web3 team deploys falls into one of four domains: organizational, personnel, physical, or technical. The split matters because each domain has a different owner, a different failure mode, and a different pace of change: organizational controls move at the speed of policy review, technical controls at the speed of a patch cycle. A team that hardens only technical controls (firewalls, hardware wallets, disk encryption) while leaving governance and personnel controls informal builds a house with a reinforced door and an unlocked window. The [NIST Cybersecurity Framework \(CSF\) 2.0](#) frames this same gap through its Govern function, elevated in the 2.0 revision to sit alongside Identify, Protect, Detect, Respond, and Recover precisely because governance gaps sit behind a disproportionate share of incidents the earlier, purely technical categories never touched.

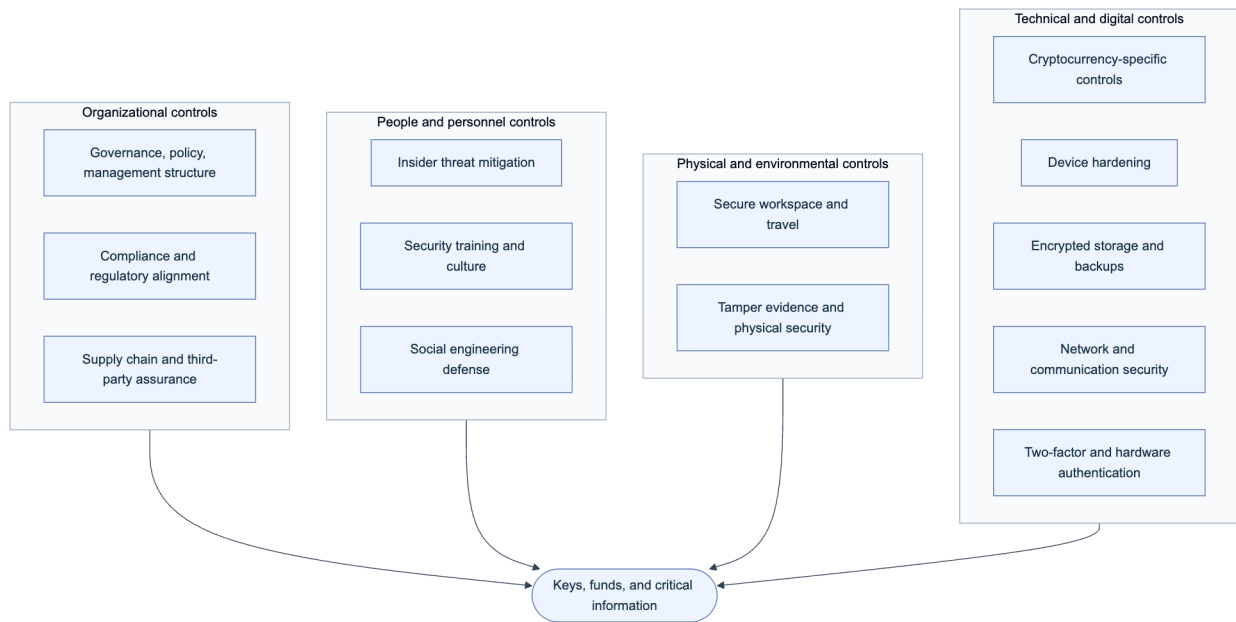


Figure 1. The four control domains, each protecting the same core assets from a different angle. A gap in one domain is not compensated by strength in another; the DPRK IT-worker infiltration pattern documented by the *FBI, U.S. Department of State, and Treasury* succeeded against organizations with strong technical controls because the failure sat in the personnel domain, at hiring.

The rest of this chapter works through each domain in turn. Chapter 12 then gives the process for turning the catalog into a program sized to a specific team’s risk and revisited on a cadence rather than assumed permanent.

11.1 Organizational Controls

Organizational controls are the policies, structures, and assurance processes that make every other domain durable rather than accidental. They answer the question “who decided this, and how do we know it still applies?” A Web3 team without them ends up with controls that exist because one engineer set them up once, with no record of why, no owner when that engineer leaves, and no mechanism to update them as the threat model changes.

11.1.1 Governance, Policies, and Management Structures

Governance is the decision-making structure that produces and maintains policy: who can approve a new custody arrangement, who owns the security budget, who signs off when a control is waived for a deadline. *NIST CSF 2.0*’s Govern (GV) function names this explicitly, covering organizational context, risk management strategy, roles and responsibilities, policy, and oversight as a first-class function rather than an implicit precondition. For a Web3 organization this usually means formalizing what a small founding team ran informally: a designated security owner (even part-time), a documented risk appetite statement, and a policy stack with clear precedence.

Structure the policy stack in layers so each document has a stable audience and update cadence: a short security policy stating intent and scope, approved by leadership and rarely changed; standards that set specific, testable requirements; procedures that describe exact steps; and

guidelines that recommend without mandating. Multisig or DAO-based approval for policy changes fits naturally here: treat a change to the custody standard the same way you would treat a treasury transaction, requiring the same threshold of signers.

Layer	Purpose	Example	Change cadence
Policy	States intent, scope, and accountability	"All production signing keys require hardware-backed MFA"	Annual, leadership-approved
Standard	Sets a specific, testable bar	"Treasury multisig threshold: minimum 3-of-5, no single jurisdiction majority"	Semi-annual
Procedure	Documents exact steps	Key-rotation runbook, offboarding checklist	As tooling changes
Guideline	Recommends without mandating	Suggested password manager, travel checklist	Ad hoc

11.1.2 Compliance and Regulatory Alignment

Regulatory exposure for a Web3 organization rarely maps onto a single jurisdiction, because contributors, users, and infrastructure are distributed. Several regimes still apply broadly enough to warrant a standing compliance function. The Financial Action Task Force's [Recommendation 16](#), the "Travel Rule," requires virtual asset service providers to collect and transmit originator and beneficiary information above defined thresholds. The European Union's [Markets in Crypto-Assets Regulation \(MiCA\)](#) sets licensing, disclosure, and operational-resilience requirements for crypto-asset service providers serving EU users. U.S. teams must screen counterparties and transactions against [Office of Foreign Assets Control \(OFAC\)](#) sanctions lists, a control with direct technical teeth after enforcement actions against mixing services and specific wallet addresses.

Compliance and security overlap more in Web3 than in traditional finance because the control that satisfies a regulator (transaction monitoring, sanctions screening, know-your-customer/anti-money-laundering (KYC/AML) verification) is frequently the same control that catches an active attack in progress. Treat compliance as a security partner, not a separate checkbox exercise: route sanctions-screening alerts and Travel Rule anomalies to the same triage process as security incidents, and maintain evidence, such as a SOC 2 report or [ISO/IEC 27001:2022](#) certification, that a counterparty or auditor will eventually request.

11.1.3 Supply Chain and Third-Party Assurance

A Web3 organization's supply chain extends past code dependencies (the subject of Handbook 01's CDN and front-end supply chain treatment) into every vendor with access to keys, infrastructure, or user data: custodians, oracle providers, bridge operators, cloud hosts, RPC (remote

procedure call) node providers, and the SaaS tools the team collaborates in. Each is a third party whose compromise becomes the organization's incident, so assurance means collecting evidence before trust is extended and re-verifying it on a schedule, not once at selection.

Require a minimum evidence package scaled to the vendor's access level: a recent SOC 2 Type II or ISO 27001 report for anyone touching production infrastructure, a published smart contract audit and bug bounty history for any on-chain dependency, and a software bill of materials (SBOM) in [CycloneDX](#) or [SPDX](#) format plus build provenance under the [Supply-chain Levels for Software Artifacts \(SLSA\)](#) framework for anyone shipping code into your build or production path.

Vendor category	Minimum evidence required	Re-verification cadence
Custodian / MPC provider	SOC 2 Type II, insurance attestation, key-ceremony documentation	Annual
Smart contract dependency	Public audit reports, bug bounty status, upgrade governance	Per version bump
Cloud / infrastructure host	SOC 2 or ISO 27001, incident notification SLA	Annual
RPC / oracle provider	Uptime SLA, decentralization of data sources, incident history	Annual
SaaS collaboration tools	SSO/SCIM support, data residency, breach notification terms	Annual

11.2 People and Personnel Controls

Technical controls constrain what a system permits; personnel controls constrain what a human is tricked or coerced into permitting. In Web3, where a single approved transaction can move an entire treasury and teams hire pseudonymously across borders, the personnel domain carries a disproportionate share of realized risk. Handbook 03 (Employee Lifecycle Security) and Handbook 05 (Hiring, Remote Work, and Insider Threat) go deep on hiring and lifecycle mechanics; this section covers the controls inside day-to-day operational security.

11.2.1 Insider Threat Mitigation

Insider threat covers three distinct actors, each needing a different control. A malicious insider deliberately abuses legitimate access; a negligent insider creates risk through carelessness, such as reusing credentials or approving a transaction without reading it; a compromised insider is a legitimate account now controlled by an external attacker, functionally indistinguishable from a malicious insider until investigated. The [DPRK IT-worker scheme](#), documented in repeated U.S. Department of Justice and Treasury actions, institutionalized a fourth pattern specific to this industry: operatives obtain legitimate remote developer roles at crypto companies using fabricated identities, generating a salary stream for a sanctioned regime and, in a subset of cases, direct access to code and infrastructure.

Actor type	Detection signal	Primary mitigation
Malicious insider	Access outside role scope, off-hours privileged actions, data staged before departure	Least privilege, separation of duties, exit revocation SLA
Negligent insider	Repeated policy exceptions, credential reuse, unreviewed approvals	Training, blind-signing bans, mandatory transaction simulation
Compromised insider	Impossible-travel logins, new device without MFA re-enrollment, unusual approval timing	Device attestation, session anomaly detection, hardware-key-bound sessions
DPRK/state-actor infiltration	Inconsistent identity documents, reluctance to appear on video, payment routed through mixing or exchange chains flagged by Chainalysis or TRM Labs reporting	Identity verification at hiring, video interview requirement, code contribution review, wallet-address screening for payroll

Layer detection with prevention: least-privilege access limits what any single insider can reach, mandatory two-person review on treasury actions removes single points of betrayal, and off-boarding automation (Handbook 03) closes the window between a role change and access revocation.

11.2.2 Security Training and Culture

Training that runs once at onboarding decays within months against an evolving threat landscape; culture is what persists between formal sessions. Build a program around three cadences: onboarding (mandatory before credential issuance, covering the organization’s custody model, approval workflows, and reporting channel), quarterly refreshers (rotating topics: phishing recognition, deepfake voice awareness, hardware wallet hygiene), and just-in-time training triggered by a near-miss or a new threat pattern observed in the industry.

Measure culture, not just completion. Track phishing simulation click-through rates over time (a healthy program trends down, not a permanently low number achieved by testing the same three people), track how quickly staff report suspicious contact versus how often security discovers it independently, and track whether staff use a “pause and verify” option without fear of looking slow. A culture where an engineer feels safe stopping a plausible \$2 million transfer to ask “does this look right to you?” outperforms any single technical control, because it catches attacks specifically designed to defeat those controls.

11.2.3 Social Engineering Defense

Social engineering in Web3 has moved past generic phishing into targeted, well-resourced campaigns: fake recruiter outreach carrying malware-laden “coding challenges,” voice-cloned executives requesting urgent wire or signing actions, and impersonated hardware wallet or exchange support staff. The defining defense is out-of-band verification: never confirm an unusual, high-stakes request using a channel the request itself supplied.

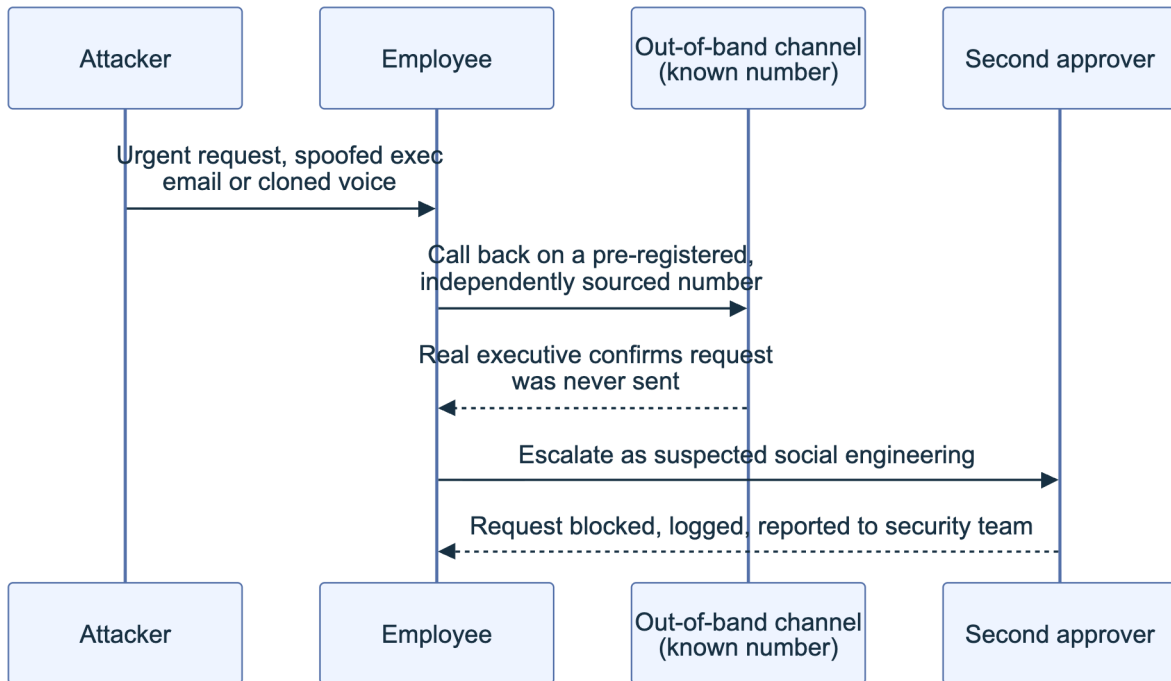


Figure 2. Out-of-band verification defeats spoofed or voice-cloned executive requests because the attacker cannot control a channel they were never given.

Formalize the callback list in advance (pre-registered numbers for every executive and every approver, distributed through a channel the attacker cannot intercept) and pair it with a code-word or shared-secret convention for voice confirmation, since generative voice cloning has closed the gap that “I’d recognize their voice” used to cover. Report every attempt, successful or not, to build a pattern library; isolated near-misses often turn out to be reconnaissance for a coordinated campaign once compared across the team.

11.3 Physical and Environmental Controls

Physical security is easy to deprioritize in a domain that talks constantly about cryptography, but a stolen unlocked laptop or a compromised hardware wallet in transit defeats every cryptographic control behind it. Web3 teams are also unusually mobile and unusually likely to carry high-value physical key material (hardware wallets, seed phrase backups) across borders, which raises the stakes of this domain relative to a typical enterprise.

11.3.1 Secure Workspace and Travel

A secure workspace starts with the basics that remote-first Web3 teams routinely skip: screen locks enforced by policy, visitors and roommates excluded from a view of the signing screen, and a clean-desk habit for any printed recovery material. Travel raises the threat model further: border crossings can compel device search or seizure in ways a home office never faces, public networks invite adversary-in-the-middle interception, and a lost bag can mean a lost key.

Travel operational security checklist

- Carry a dedicated travel device with minimal data, not your primary signing device, wherever the trip risk profile allows it.
- Move hardware wallets and seed backups in carry-on luggage, never checked baggage, splitting custody across separate bags or travelers when threshold custody allows it.
- Disable biometric unlock before crossing a border where compelled unlocking is a documented risk, reverting to a passphrase.
- Use a trusted, paid VPN on any public or hotel network; never sign a transaction or enter a recovery phrase on unmanaged Wi-Fi.
- Pre-register a check-in cadence with a designated contact for high-risk travel, with a defined escalation trigger.
- Avoid publicizing travel or conference plans in advance; targeted physical attacks against known key holders correlate with conference schedules.

11.3.2 Tamper Evidence and Physical Security

Tamper evidence lets you detect interference with a device even when the interference was too skilled to leave functional traces. Hardware wallet vendors ship devices in tamper-evident packaging specifically so a purchaser can detect pre-shipment interdiction; verify the seal against the vendor's published reference images before first use, and treat any mismatch as a reason to discard the unit rather than debug it. Never accept a hardware wallet as a gift or pre-configured unit from an unverified source, and never restore a seed phrase you did not personally generate onto hardware you did not personally initialize.

Extend tamper evidence past the device itself: use numbered, void-indicating tamper-evident bags for any device or backup material in transit or storage, log serial and seal numbers at each handoff, and store master recovery backups in physically secured, rated locations (a bank safe deposit box, a UL/TL-rated commercial safe) distributed geographically so a single physical event cannot recover a full threshold. Document a chain-of-custody log for any physical key material, mirroring the evidentiary discipline forensic teams use, so any gap in custody is visible rather than assumed away.



Figure. Tamper-evident tape leaves a visible void pattern once peeled, the same mechanism a numbered security bag uses to reveal interference with a hardware wallet or backup shard in transit. A missing or mismatched void pattern at handoff is a chain-of-custody failure, not a cosmetic detail. Source: [Wikimedia Commons](#), CC BY-SA 4.0.

11.4 Technical and Digital Controls

Technical controls are where most Web3 security investment concentrates, and for good reason: they are the last line standing when organizational, personnel, and physical controls all fail simultaneously. They are also the domain most specific to Web3's core function, custody of cryptographic key material that directly authorizes value transfer with no reversal mechanism.

11.4.1 Cryptocurrency-Specific Controls

Custody architecture is the central technical decision a Web3 organization makes, and it trades off liquidity against blast radius. A single-signature hot wallet is fastest to use and worst to lose; a fully offline, air-gapped cold wallet is safest at rest and slowest to operate; multisignature and multi-party computation (MPC) schemes sit between the two, distributing signing authority so no single compromised key or device authorizes a transfer alone.

Custody model	Typical use	Compromise blast radius	Operational friction
Hot wallet, single key	Operational float, gas reserves	Full loss of wallet balance	Low
Multisig (for example, Safe , 2-of-3 or 3-of-5)	Treasury, protocol admin	Requires compromising threshold of signers	Medium

Custody model	Typical use	Compromise blast radius	Operational friction
MPC custody	Institutional treasury, exchange-grade custody	Depends on provider's key-share distribution and policy engine	Medium
Cold storage, air-gapped	Long-term reserves	Requires physical access plus PIN/passphrase	High

Two controls belong on every custody tier regardless of model: address allowlisting (restricting routine flows to pre-approved destinations) and transaction simulation before signing, using tooling that renders the actual state change a pending transaction will cause rather than trusting the signing interface's display. The February 2025 [Bybit](#) incident, in which attackers compromised a Safe{Wallet} front-end and presented signers with a transaction display that did not match its actual on-chain effect, is the canonical argument for simulation independent of the signing UI: every signer approved what they saw on screen, and what they saw was not what executed. Align custody practice to the [CryptoCurrency Security Standard \(CCSS\)](#) where an external, auditable baseline is useful for counterparties or investors.

11.4.2 Device Hardening

Every device that touches a key, a signing session, or privileged infrastructure access needs a hardened baseline, not the vendor default configuration. Start from a published benchmark: the [Center for Internet Security \(CIS\) Benchmarks](#) provide OS-specific hardening guides, and [NIST SP 800-53](#) System and Communications Protection and System and Information Integrity control families map hardening requirements to the same catalog governance uses elsewhere in this handbook.

```
# macOS baseline spot-check before issuing a device for signing use
fdesetup status           # FileVault full-disk encryption: On
sudo spctl --status       # Gatekeeper: enabled
csrutil status            # System Integrity Protection: enabled
softwareupdate --list     # no pending critical updates
```

Minimum baseline for any device in the signing or privileged-access path: full-disk encryption enforced by policy, endpoint detection and response (EDR) tooling reporting to a central console, automatic updates with a short patch SLA for critical vulnerabilities, application allowlisting or at minimum a deny-list for known-malicious installer patterns (the fake recruiter “coding challenge” malware family is a recurring Web3-specific vector), and a dedicated, single-purpose device for high-value signing that never runs a general-purpose browser or email client.

11.4.3 Encrypted Storage and Backups

Every piece of key material needs a backup, and every backup is a second copy of the thing an attacker wants most. Resolve that tension with cryptographic splitting rather than plaintext duplication. [SLIP-0039](#) (Shamir's Secret Sharing for Mnemonic Codes) splits a seed into multiple

shares with a defined threshold, so that, for example, any 3 of 5 shares reconstruct the seed while any 2 reveal nothing. This is materially stronger than the single **BIP-39** mnemonic phrase it is often layered on top of: loss of one share is not catastrophic, and theft of one share is not sufficient.

Backup method	Threshold model	Primary risk	Best fit
Single BIP-39 phrase, paper	None (single point)	Fire, water, single-location theft	Not recommended for material value
BIP-39 phrase, metal backup	None (single point, fire/water resistant)	Single-location theft	Personal wallets, low-to-moderate value
SLIP-39 shares, geographically split	k-of-n	Requires coordinating k custodians for recovery	Treasury, organizational custody
Encrypted digital backup (age, GPG) offline	Depends on key management	Passphrase or key compromise	Supplementary, never sole backup

Encrypt any digital backup at rest with a well-reviewed tool (`age` or GPG with a strong passphrase) and store the encrypted artifact and the decryption key under separate custody so a single breach does not yield both halves. Test recovery on a schedule: an unverified backup is a hypothesis, not a control.

11.4.4 Network and Communication Security

Network controls protect the path between a device and the services it depends on; communication controls protect the content moving over that path. Route all traffic from devices that touch signing or infrastructure access through a managed VPN or zero-trust gateway rather than trusting network location, consistent with the zero-trust posture Part 1 anchors in **NIST SP 800-207**. Configure DNS-over-HTTPS or DNS-over-TLS to reduce DNS-based interception, and block known-malicious or newly registered domains at the resolver.

```
# WireGuard peer fragment: force all device traffic through the tunnel
[Interface]
PrivateKey = <device-private-key>
DNS = 10.10.0.1
[Peer]
Endpoint = vpn.example-org.internal:51820
AllowedIPs = 0.0.0.0/0, ::/0
```

For communication, standardize on end-to-end encrypted tooling with forward secrecy for anything discussing keys, incidents, or unreleased code: the **Signal Protocol** is the reference implementation this handbook recommends by default, with disappearing messages enabled on sensitive channels so a compromised device has a bounded blast radius into history. Ban unmanaged personal messaging apps for security discussions, and provision or de-provision any group chat used for approvals through the same lifecycle process as any other access grant.

11.4.5 Two-Factor and Hardware Authentication

Not every second factor resists the same attacker. SMS one-time passcodes (OTP) are vulnerable to SIM-swap fraud, a technique the [FBI's Internet Crime Complaint Center \(IC3\)](#) has repeatedly flagged in public service announcements as a growth vector specifically because it defeats an entire authentication factor without touching the victim's device. Push-based approval improves on SMS but remains vulnerable to "MFA fatigue" attacks, where an attacker spams approval requests until a tired or distracted user taps accept. Hardware security keys implementing the [FIDO2/WebAuthn](#) standard are the strongest widely deployed option because the cryptographic challenge is bound to the origin requesting it, which makes phishing a key-protected login effectively impossible even against a perfect visual clone of the real site.

Factor	Phishing resistance	SIM-swap resistance	NIST SP 800-63B AAL fit
SMS OTP	None	None	AAL1, discouraged for new deployments
TOTP app (authenticator)	Low (proxyable in real time)	High	AAL2
Push notification	Low-medium (fatigue attacks)	High	AAL2
FIDO2 hardware key (WebAuthn)	High (origin-bound)	High	AAL3



Figure. A FIDO2/WebAuthn hardware security key: the form factor behind the AAL3 mandate in this section. The cryptographic challenge it answers is bound to the requesting origin, which is what makes it resistant to phishing in a way a TOTP code or push approval is not. Source: [Wikimedia Commons](#), CC BY-SA 4.0.

Mandate hardware-key MFA (per [NIST SP 800-63B](#) Authenticator Assurance Level 3, AAL3) for every account that can approve a transaction, alter a custody configuration, or access source repositories with write permission, and register at least two physical keys per person so a lost key does not lock the owner out into an insecure recovery flow. Retire SMS as a fallback factor entirely wherever the identity provider allows it; a strong primary factor with a weak recovery path is only as strong as the recovery path.

12. Control Selection and Implementation

A control catalog is a menu, not a mandate: no organization deploys every control in Chapter 11 on day one, and few should try. This chapter gives the process for turning the catalog into a program sized to a team's risk and revisited on a schedule.



Figure 3. Control selection is a cycle, not a project. Chapter 14 (Continuous Improvement) picks up where 12.5 feeds back into 12.1 at the program level.

12.1 Identifying Requirements

Requirements come from three sources that rarely agree on priority, so name each explicitly rather than blending them into a vague “best practice” mandate. Regulatory and contractual requirements are non-negotiable floors: a Travel Rule obligation or a SOC 2 customer commitment exists whether or not the team agrees it is the most urgent risk. Threat-model-derived requirements come from the asset-identification and threat-analysis process in Part 1, translating “the treasury multisig is a target” into “the multisig needs hardware-key-bound signers and transaction simulation.” Business requirements set the friction ceiling: a five-person team cannot run the same approval chain as a two-hundred-person exchange, and forcing that mismatch produces workarounds worse than the risk the control targeted.

Source	Example requirement	Output artifact
Regulatory / contractual	FATF Travel Rule data transmission for qualifying transfers	Compliance control mapping
Threat-model-derived	Treasury signers must resist a \$10M targeted phishing campaign	Control requirement statement tied to a named asset
Business / operational	Approval latency must stay under 4 hours for routine payroll	Friction budget per control

Document requirements as testable statements, not aspirations: “signers use hardware keys” is testable; “signers should be security-conscious” is not.

12.2 Assessing Current State and Gaps

Assessment starts by mapping what is actually deployed against the requirement list from 12.1, and the honest version of this exercise usually surprises the team running it: a control assumed universal often has holes at exactly the devices added most recently or held by the most senior, least-supervised staff. Use a maturity reference to avoid a binary pass/fail that hides partial implementation: [NIST CSF 2.0 Tiers](#) (Partial, Risk Informed, Repeatable, Adaptive) or the Department of Energy’s [Cybersecurity Capability Maturity Model \(C2M2\)](#) give a graduated scale that shows progress between audits rather than a recurring zero.

Gap assessment matrix (excerpt)

Control	Current state	Gap	Priority
Hardware-key MFA on all signer accounts	6 of 9 signers enrolled	3 signers on TOTP	High
SLIP-39 backup of treasury seed	Single paper backup, one location	No threshold split	Critical
SOC 2 report from MPC custody vendor	Requested, not received	Evidence gap	Medium

Run this assessment against evidence, not self-report: pull actual MFA enrollment data from the identity provider rather than asking each signer to confirm.

12.3 Selecting Appropriate Controls

Selection is where risk appetite becomes concrete, and the common failure mode runs in both directions: under-controlling leaves the gaps found in 12.2 open, while over-controlling burns the team's limited attention on low-value friction and breeds the workaround culture that defeats the controls that matter. Weigh each candidate on three axes: effectiveness against the specific threat, implementation cost, and operational friction. Map candidates against the STRIDE-style threat categories from Part 1 so selection traces back to a named threat rather than a generic “more security” instinct.

Threat	Candidate control	Effectiveness	Decision
Phished signer approves malicious transaction	Transaction simulation before signing	High	Adopt, mandatory
SIM-swapped signer bypasses MFA	Retire SMS fallback, hardware-key only	High	Adopt, phased rollout
Insider exfiltrates treasury via legitimate access	Two-person approval on treasury moves	High	Adopt for treasury only

Reference [NIST SP 800-53's](#) control catalog and CSF 2.0's Protect function when a team needs an externally recognized control ID for an auditor or investor, but do not let the catalog substitute for the threat-driven reasoning above it; a control selected only because it appears on a checklist is one nobody maintains once the audit passes.

12.4 Systematic Deployment and Monitoring

Deploy selected controls in phases, not as a single cutover: pilot with a small group, fix the friction points the pilot surfaces, then roll out organization-wide with a fixed deadline and an executive-communicated mandate so stragglers cannot indefinitely opt out. A hardware-key MFA rollout, for example, benefits from provisioning two keys per person during the pilot phase, catching the “what happens when I lose my only key” failure mode before it hits the whole company at once.

Monitoring turns “we deployed the control” into “we know the control is working.” Track leading indicators that predict failure before an incident does: MFA enrollment rate and factor type distribution pulled from the identity provider (not self-reported), phishing simulation click-through and report rates trended over quarters, patch latency for critical vulnerabilities on devices in the signing path, and time from offboarding trigger to full access revocation. Feed these into a central dashboard reviewed at the deployment plan’s cadence, and treat a metric trending the wrong way as an incident-adjacent signal worth investigating, not a number to revisit next quarter.

12.5 Continuous Evaluation and Improvement

A control program is never finished; the threat model that justified it keeps moving. Set a fixed cadence for re-evaluation rather than relying on an incident to trigger it, because by the time an incident forces the review, the gap has already been exploited.

Cadence	Activity
Weekly	Review monitoring dashboard for trending metrics and open alerts
Monthly	Vendor assurance evidence spot-check, offboarding SLA audit
Quarterly	Phishing simulation, policy exception review, gap-matrix refresh
Semi-annual	Tabletop exercise against a current threat scenario (Part 5, Appendix C)
Annual	Full control catalog re-selection against updated threat model, third-party penetration test, policy re-approval

Close the loop explicitly back to 12.1: every tabletop exercise, near-miss, and finding from the Incident Response Handbook (06)’s postmortem process is a new input to “identifying requirements,” not a one-off remediation. [NIST CSF 2.0](#)’s Respond and Recover functions both build in an improvement step that feeds lessons learned forward into future activity; treat every control review the same way, whether or not an incident triggered it. Part 4 integrates this cadence with the organization’s wider security program, including how it interacts with external audits and the incident response lifecycle.

Key controls for Part 3

- Organize every control under one of four domains (organizational, personnel, physical, technical) so governance, people, physical security, and technology are assessed as a set, not in isolation.
- Layer the policy stack (policy, standard, procedure, guideline) and require multisig-equivalent approval for changes to security-critical standards.

- Screen and monitor for state-actor and insider infiltration specifically, including DPRK IT-worker infiltration patterns, alongside conventional negligent- and malicious-insider risk.
- Require out-of-band verification for any unusual, high-stakes request, with a pre-registered callback list distributed outside the channel an attacker could intercept.
- Mandate hardware-key (FIDO2/WebAuthn) MFA at AAL3 for every account that can approve a transaction or write to production, and retire SMS as a fallback wherever possible.
- Split key material with SLIP-39 threshold backups stored under separate, geographically distributed custody, and verify tamper-evident packaging on every hardware wallet before first use.
- Run the control lifecycle (identify, assess, select, deploy, evaluate) as a recurring cycle with fixed cadences, not a one-time project closed out after the first audit.

Part IV: Integration and Continuous Improvement

Operational security (OpSec) fails as a program the moment it becomes an island: a binder of policies that never touches the incident responders, the infrastructure team's monitoring stack, or the organization's legal obligations. This part connects the controls built in Parts II and III to the rest of the security program and to the standards this handbook is built on, then sets a cadence for revisiting all of it. A control deployed once and never reviewed decays into paperwork; the practices in Chapter 14 are what keep the program current as the team, the threat landscape, and the regulatory environment change under it.

13. Integration with Other Frameworks

OpSec does not run in isolation from verification standards, incident response, infrastructure monitoring, or privacy law. Each of the following sections names a specific seam and the handoff that has to work across it.

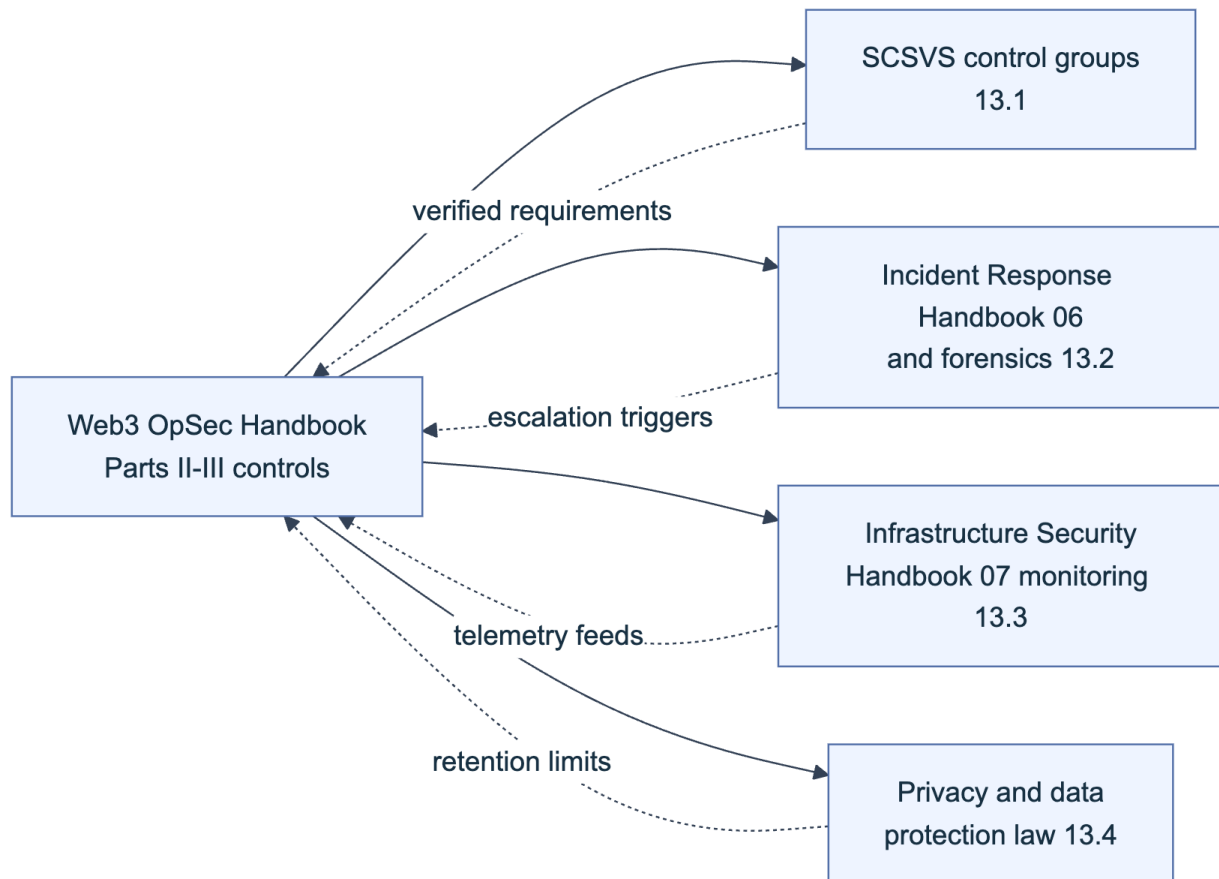


Figure 1. Chapter 13 maps four integration seams. Each arrow out of OpSec represents a control this handbook produces; each dashed arrow back represents a requirement the other framework imposes on how that control is built.

13.1 Mapping to SCSVS Control Groups

The **Smart Contract Security Verification Standard (SCSVS)** states verifiable requirements a deployed smart contract system must satisfy: correct access control logic, sound cryptographic usage, safe external communication, and defensible business logic and governance. Every one of those requirement families assumes a human and process layer behind it that the code itself cannot verify, and that layer is exactly what this handbook's Parts II and III build. A contract can implement textbook-correct multi-signature access control and still be compromised the moment one signer's laptop is unmanaged, because SCSVS checks the contract's access-control logic, not the operational hygiene of the key that logic trusts.

Treat the mapping as a checklist for the security lead who owns both the contract audit and the OpSec program, so that a passing SCSVS review is not mistaken for a complete security posture:

SCSVS control area	What SCSVS verifies in code	OpSec dependency (this handbook)
Access control and authentication	Role checks, multi-signature thresholds, admin function gating	Key custody, hardware authenticator enrollment, access review cadence (Chapters 8, 12)

SCSVS control area	What SCSVS verifies in code	OpSec dependency (this handbook)
Cryptographic implementation	Correct signature schemes, randomness, key derivation	Signing device hardening, seed and key storage, endpoint integrity (Chapters 6, 8)
Secure communication	Oracle and cross-contract message validation	Communications platform hardening, verified out-of-band channels (Chapter 9)
Architecture and threat modeling	Design-level attack surface review	Threat actor and asset identification process (Part I, Chapter 4)
Business logic and governance	Economic invariants, upgrade and pause authority	Governance key custody, multi-party approval workflow (Chapter 12)

A finding on either side of this table should trigger a look at the other. If an SCSVS review flags an admin function with a single-signer threshold, the OpSec response is not just “raise the threshold in code,” it is also “verify every signer’s device and key storage meet Chapter 6 and 8 standards,” because a higher threshold only helps if each additional signer is independently hard to compromise.

13.2 Alignment with Incident Response and Forensics

OpSec is preventive; incident response (IR) is reactive; forensics reconstructs what happened after the fact. The three disciplines share data and hand off responsibility at defined points, and *NIST SP 800-61 Revision 3, Incident Response Recommendations and Considerations for Cybersecurity Risk Management: A CSF 2.0 Community Profile (April 2025)*, frames this explicitly: incident response is not a standalone function but a set of activities threaded through every NIST Cybersecurity Framework (CSF) 2.0 function, from Govern through Recover. The Incident Response Handbook (06) owns the escalation runbooks, containment procedures, and evidence-handling chain of custody; this handbook’s job is to make sure the inputs IR needs already exist before an incident starts.

Three handoffs matter most for a Web3 team. First, the asset and key inventory this handbook’s controls maintain (Chapters 6 and 8) is what lets an IR lead scope an incident in minutes instead of days: which signer, which device, which key, when it was last rotated. Second, the out-of-band communication channel established under Chapter 9, verified and pre-tested, becomes the IR team’s assumption of last resort when the primary channel (email, the usual chat workspace) is itself suspected of compromise. Third, when the incident involves fund movement, IR hands off to the EVM Forensics and DeFi Recovery Handbook (04) for on-chain tracing, and the OpSec-maintained wallet and signer inventory is the starting point for that trace.

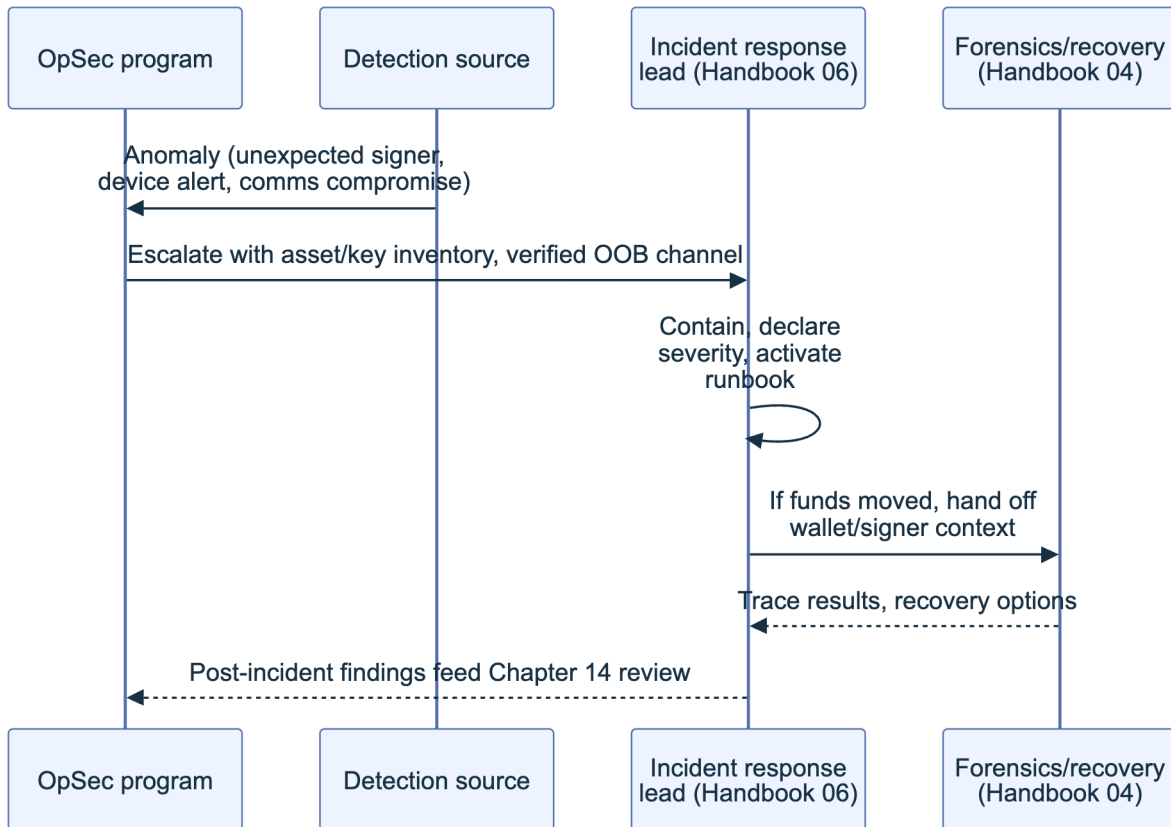


Figure 2. The OpSec-to-IR handoff. OpSec supplies the inventory and channel IR needs at declaration time; IR returns findings that feed the continuous improvement loop in Chapter 14.

13.3 Infrastructure and Monitoring Integration

Every OpSec control in Parts II and III generates signal: a device enrolls or is deprovisioned, a hardware key is registered or revoked, a secrets vault is accessed, a comms platform admin role changes. That signal is worthless sitting in the console of the tool that produced it. The Infrastructure Security Handbook (07) owns the organization’s monitoring stack, and the integration point is straightforward but frequently skipped: OpSec-owned identity, device, and access events must flow into the same correlation layer as infrastructure telemetry, because an attacker who compromises a laptop and an attacker who compromises a cloud role are executing the same kill chain from different starting points. *NIST SP 800-207, Zero Trust Architecture*, frames this as continuous, centralized policy evaluation: a Zero Trust Architecture (ZTA) is only as good as the telemetry feeding its policy decision point, and identity and device signals from OpSec controls are core inputs to that decision, not a side channel.

OpSec event source	What it emits	Where it should land
Identity provider / single sign-on (SSO)	Login, MFA challenge, session anomaly	Central log pipeline, correlated with device posture
Hardware authenticator (FIDO2/ YubiKey) registry	Enrollment, revocation, backup-key use	Central log pipeline, alert on out-of-policy revocation

OpSec event source	What it emits	Where it should land
Endpoint detection and response (EDR) agent	Process anomaly, unmanaged device flag	Same alerting queue as infrastructure EDR/cloud alerts
Secrets manager / key management service (KMS)	Access, rotation, permission change	Central log pipeline, alert on off-hours or new-principal access
Comms platform admin audit log	Role change, integration added, export event	Central log pipeline, weekly review minimum

Building this integration is mostly a routing problem, not a tooling problem: point each source's audit-log export at the same collector the infrastructure team already runs, tag events with an OpSec-specific source field so they are queryable independently, and agree with the infrastructure team on which OpSec event types page someone immediately versus which land in a daily digest.

13.4 Privacy and Data Protection Overlap

The OpSec program itself is a data controller. Device inventories, travel schedules, hardware key serial numbers, background-check results referenced from the Hiring, Remote Work, and Insider Threat Handbook (05), and even the metadata of who has access to which signing key, are personal data about identifiable team members. That data is now a target in its own right: an attacker who obtains the travel calendar of a treasury signer has a head start on timing a physical or SIM-swap attack, and a leak of the device inventory tells an attacker exactly which laptop model and operating system to build an exploit for.

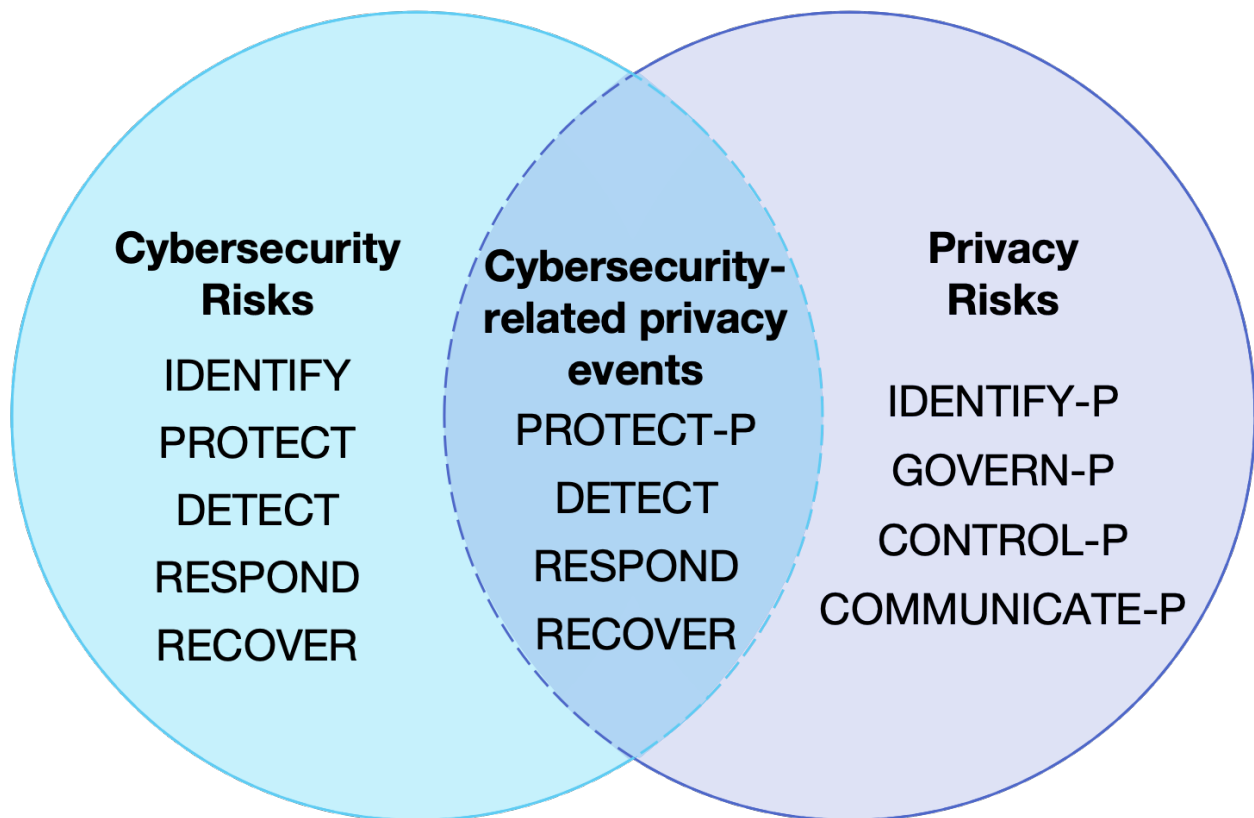


Figure. Privacy and cybersecurity outcomes overlap but are not interchangeable. Security controls can protect an OpSec data store while the collection itself still creates privacy risk; the Privacy Framework adds explicit Control-P and Communicate-P outcomes for how data is processed and explained. Source: *NIST, Privacy Framework and Cybersecurity Framework Functions*, U.S. government work, public domain.

Two regulatory regimes set the floor. Under the EU’s General Data Protection Regulation (GDPR), [Article 33](#) requires notifying the competent supervisory authority “without undue delay and, where feasible, not later than 72 hours” after becoming aware of a personal data breach, which means an OpSec data store (the travel log, the device inventory) has the same 72-hour clock as customer data the moment it holds EU personal data. US state laws, principally the California Consumer Privacy Act as amended by the California Privacy Rights Act (CCPA/CPRA), impose parallel obligations on any organization handling California residents’ data, including employees’. Treat OpSec records with the same classification discipline as customer data:

- **Minimize collection:** log the fact that a device passed a compliance check, not the full device fingerprint, unless the fingerprint is operationally required.
- **Minimize retention:** travel itineraries and access logs older than the incident-response and audit window (commonly six to eighteen months) should be purged, not archived indefinitely.
- **Restrict access:** the OpSec data store itself needs the access controls described in Chapter 8, not just the assets it protects.
- **Encrypt at rest and in transit:** no exception for “internal” tooling.

This is not a compliance afterthought bolted onto Chapter 13; it is the reason the data-minimization habits taught throughout Part III exist in the first place.

14. Continuous Improvement

A control set frozen at first deployment is a snapshot of yesterday's threat model. This chapter turns the one-time process from Part I, Chapter 4 (identify assets, analyze threats, assess vulnerabilities, evaluate risk, deploy controls) into a loop that runs continuously against a changing team, changing tooling, and a changing adversary.

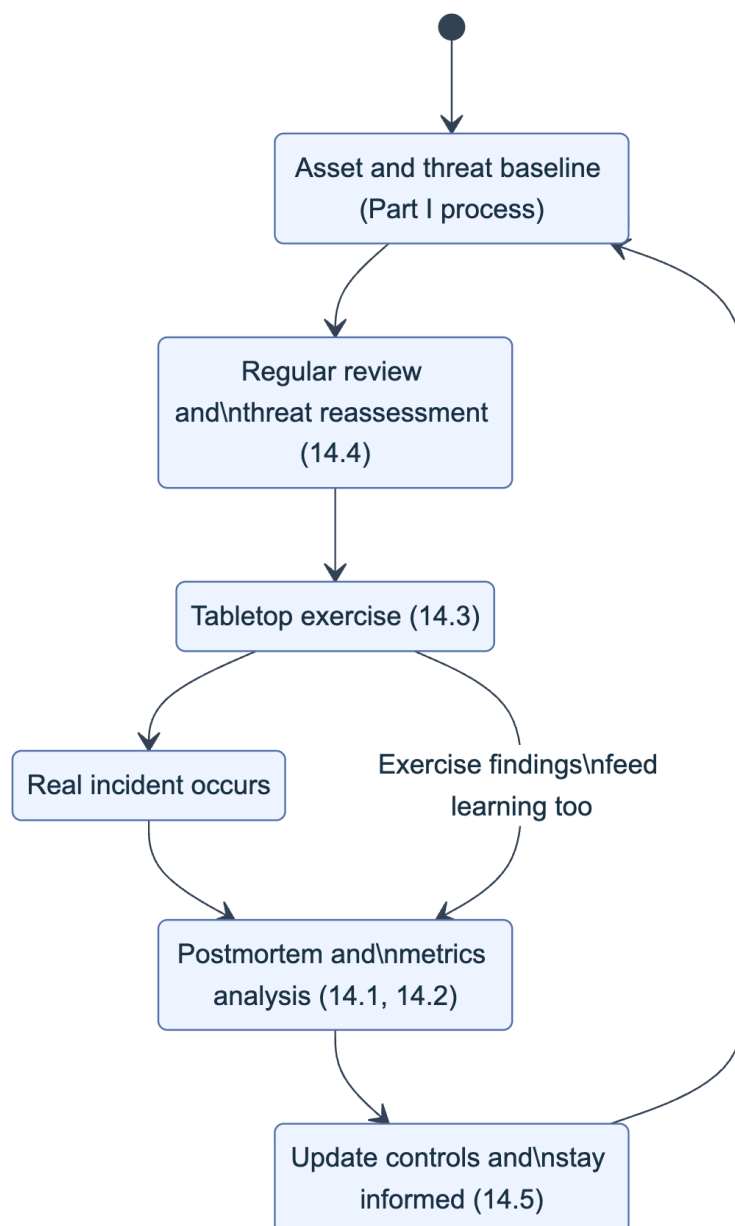


Figure 3. The continuous improvement loop. Learning enters from two paths, real incidents and rehearsed tabletop exercises, and both feed the same control-update step before the baseline is reassessed.

14.1 Learning from Incidents

Every incident, whether it lands on this team or a peer organization, is a free lesson if the postmortem process captures it. Adopt a blameless postmortem discipline: the [Google Site Reliability Engineering postmortem culture guidance](#) is written for infrastructure outages but the principle transfers directly to key-compromise and social-engineering incidents, that a postmortem “assumes everyone involved had good intentions and did the right thing with the information they had,” and that fixing systems, not blaming people, is what prevents recurrence. A postmortem that ends with “the signer should have been more careful” produces no control change; one that ends with “the signing workflow allowed a blind-signed transaction to reach a hardware wallet with no simulation step” produces one.

External incidents deserve the same rigor, run as a desk review rather than a live postmortem. The February 21, 2025 Bybit exchange theft, roughly \$1.5 billion in Ether, remains the largest cryptocurrency theft on record: attackers compromised the workstation of a developer at Safe{Wallet}, the third-party multisignature wallet interface Bybit’s signers used, and manipulated the transaction displayed in that interface so that signers approved a transfer of upgrade authority while believing they were approving a routine transfer. US federal investigators and blockchain analytics firms attributed the operation to North Korea’s Lazarus Group, operating under the “TraderTraitor” cluster name ([Bybit incident summary](#)). A team that never touches Safe{Wallet} still learns from this: the failure mode is blind signing on a compromised front end, not a Bybit-specific bug, and the applicable defenses (hardware wallet transaction simulation, independent verification of calldata before signing, treating the signing workstation itself as in scope for endpoint hardening under Chapter 6) apply to any multisignature workflow.

```
## Postmortem: [incident name], [date]
**Severity**: [Critical/High/Medium/Low]
**Duration**: [detection to resolution]
**Impact**: [funds, data, availability, quantified]

### Timeline
- [UTC timestamp]: [event]

### Root cause
[System and process failure, not individual blame]

### What went well
[Detection, containment, or communication that worked]

### What went wrong
[Gaps, with no names attached to blame]

### Action items
| Action | Owner | Due | Chapter/control affected |
|---|---|---|---|
```

14.2 Security Metrics and Effectiveness of Controls

A control nobody measures is a control nobody can prove is working. Split metrics into leading indicators, which predict future exposure, and lagging indicators, which count past failures. NIST CSF 2.0's Govern function ties an organization's risk management strategy directly to measurable outcomes rather than checklist completion, which is the framework-level argument for building this section's metrics program instead of treating Chapter 6 through 12's controls as a one-time deployment ([NIST CSF 2.0](#)).

Metric	Type	What it signals	Review cadence
Hardware authenticator enrollment rate	Leading	Fraction of privileged accounts still on phishable auth	Monthly
Phishing simulation click-through rate	Leading	Susceptibility trend across the team	Quarterly
Key/credential rotation SLA adherence	Leading	Whether rotation policy is followed, not just written	Monthly
Mean time to detect (MTTD)	Lagging	Monitoring pipeline effectiveness (ties to 13.3)	Per incident, trended quarterly
Mean time to respond/contain (MTTR)	Lagging	IR handoff effectiveness (ties to 13.2)	Per incident, trended quarterly
Device compliance rate (patched, MDM-enrolled)	Leading	Endpoint fleet exposure (Chapter 6)	Monthly
Access review completion rate	Leading	Whether Chapter 12 reviews actually happen on schedule	Per review cycle

Report these to whoever owns risk decisions, not just to the security team. A metric that only the people who produce it ever see cannot change budget or priority.

14.3 Tabletop Exercises and Scenario Walk-Throughs

A tabletop exercise rehearses the incident response process against a realistic scenario without a real incident's cost, and it is the single highest-leverage way to find gaps in the OpSec-to-IR handoff described in 13.2 before an adversary finds them first. The Cybersecurity and Infrastructure Security Agency (CISA) publishes structured [Tabletop Exercise Packages \(CTEPs\)](#) covering ransomware, insider threat, and supply chain scenarios with discussion guides and injects; adapt them rather than writing from a blank page, then layer in the scenarios specific to a Web3 team's threat model.

Run at minimum these four Web3-specific scenarios annually, each scripted with timed injects and a designated facilitator who is not also a scenario participant:

Scenario	Tests	Key question to answer
Signer device compromise mid-transaction	Endpoint hardening (Ch. 6), signing workflow, IR handoff	Does anyone notice before the signature is broadcast?
SIM-swap of a treasury or governance signer	Authentication (Ch. 8), out-of-band verification	Can the team reach the signer through a channel the attacker doesn't control?
Deepfake video call requesting emergency key access	Communications security (Ch. 9), verification culture	Does the requested action get independently verified before anyone acts?
Malicious insider attempts unauthorized fund movement	Access control (Ch. 12), multi-party approval	Does the approval workflow actually require a second independent party?

Full scripts, injects, and facilitator notes for each of these scenarios are collected in Appendix C (Part V). Debrief every exercise with the same blameless format used for real incidents (14.1); a tabletop that surfaces a gap and produces no action item was a rehearsal with no payoff.

14.4 Regular Reviews and Threat Assessment

The five-step process from Part I, Chapter 4, asset identification, threat actor analysis, vulnerability assessment, risk evaluation, and control deployment, is not a project with an end date. Re-run it on a fixed cadence and, separately, whenever a defined trigger event occurs, because calendar-only reviews miss the risk that shows up between scheduled dates.

Calendar cadence: - Access and key inventory review: quarterly (feeds the metric in 14.2) - Full threat model refresh: semi-annual - Complete OpSec program audit, including a walkthrough of every chapter's controls in Parts II and III: annual

Trigger-based review, run outside the calendar the moment any of these occur: - A new chain, product line, or custody model launches (new asset class, new threat surface) - Personnel change involving anyone with signing authority or admin access (departure, role change, onboarding) - A merger, acquisition, or major vendor change (new inherited risk) - Any incident above a defined severity threshold (feeds directly from 14.1) - A named incident at a peer organization involving a control this team also relies on (as with the Bybit case in 14.1)

Document every review's outcome even when it produces no change; "reviewed, no change required, evidence attached" is itself the audit trail that proves the loop is running.

14.5 Staying Informed and Updating Practices

Threat intelligence that never reaches the person who can act on it is not intelligence, it is archive material. Build a short, recurring ritual, not an open-ended subscription list nobody reads. A practical monthly digest pulls from a small set of sources chosen for signal density over volume:

- **MITRE ATT&CK**, updated on a semi-annual release cycle, for technique-level adversary behavior across the broader threat landscape.

- **MITRE AADAPT** (Adversarial Actions in Digital Asset Payment Technologies), the ATT&CK-style knowledge base built specifically for cryptocurrency and digital-asset threat behavior, for techniques ATT&CK's general IT scope does not cover.
- CISA advisories and alerts, for infrastructure-level and nation-state activity with direct relevance to Chapter 13.3's monitoring integration.
- OWASP Smart Contract Security (SCS) project updates, for changes to SCSVS, the Smart Contract Weakness Enumeration (SCWE), and the **Web3 Attack Vectors Top 15**, which tracks the supply-chain and social-engineering vectors most relevant to this handbook.
- Hardware wallet and custody vendor advisories (Ledger, Trezor, Safe, and any custody provider actually in use), for vulnerability disclosures affecting the specific tools the team's signers rely on.

Assign the digest to a named owner with a rotation, not to “the security team” collectively, because unowned recurring work is the first thing that lapses under deadline pressure. Each digest item gets one of three dispositions logged: no action needed, tracked for the next scheduled review (14.4), or escalated immediately.

14.6 Log Management and Visibility for OpSec

None of the review, metrics, or learning processes in this chapter work without a log record to review, measure, or learn from. **NIST SP 800-92, *Guide to Computer Security Log Management***, lays out the lifecycle this section applies to OpSec-specific sources: generate, transmit, store, analyze, and dispose, with retention and access decisions made deliberately rather than by whatever the default happens to be in each tool.

The minimum OpSec-relevant log inventory, cross-referenced to where each source is discussed elsewhere in this handbook:

```
opsec_log_sources:
- source: identity_provider_sso
  events: [login, mfa_challenge, session_anomaly, admin_role_change]
  retention_days: 400
  reference: "Chapter 8, Section 13.3"
- source: hardware_authenticator_registry
  events: [enrollment, revocation, backup_key_use]
  retention_days: 730
  reference: "Chapter 8"
- source: endpoint_edr
  events: [process_anomaly, unmanaged_device_flag, compliance_drift]
  retention_days: 400
  reference: "Chapter 6, Section 13.3"
- source: secrets_manager_kms
  events: [access, rotation, permission_change]
  retention_days: 730
  reference: "Chapter 8, Section 13.2"
- source: comms_platform_admin_audit
  events: [role_change, integration_added, export_event]
  retention_days: 400
  reference: "Chapter 9"
- source: travel_and_device_inventory
  events: [record_created, record_accessed, record_purged]
  retention_days: 550
  reference: "Section 13.4 (privacy-restricted)"
```

Retention figures above are illustrative starting points, not a fixed mandate: set them against the incident-response and forensic window the organization actually needs (Section 13.2) and against the minimization obligation from Section 13.4, and document the reasoning either way. The travel and device inventory row deliberately shows the tension between the two sections directly: it needs to exist long enough to support an investigation, and no longer, with access restricted to the smallest group that can justify needing it.

Key controls for Part 4

- Maintain a mapping between SCSVS control groups and the OpSec chapters that support them; review it whenever an SCSVS finding lands.
- Feed OpSec identity, device, and access events into the same monitoring pipeline the infrastructure team runs, tagged for independent query.
- Give the OpSec-to-IR handoff a rehearsed, verified out-of-band channel before an incident, not during one.
- Classify and minimize OpSec's own records (travel, device inventory, access logs) as personal data subject to GDPR Article 33 and equivalent state law.
- Run blameless postmortems on internal incidents and structured desk reviews on major external ones (Bybit, Ledger Connect Kit) with a documented action-item owner and due date.

- Rehearse at least four Web3-specific tabletop scenarios annually and debrief every one the same way as a real incident.
- Re-run the five-step OpSec process on both a fixed calendar and a defined trigger list; log every review's outcome, including "no change required."

Part V: Appendices

This part is the handbook's working reference layer, built to be pulled up mid-incident or mid-onboarding rather than read start to finish. It fixes the vocabulary from Parts 1 through 4 in a single glossary, packages the control set into copy-ready policy templates and printable checklists, and grounds the abstract controls in two named cryptocurrency thefts and a physical-coercion trend so a reader can rehearse the failure before it happens to their own team.

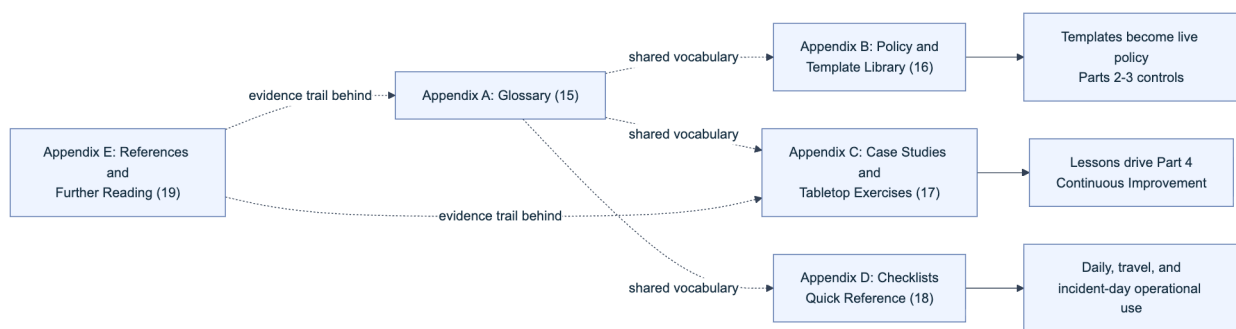


Figure 1. How the appendices connect. The glossary is a lookup table for everything below it; the policy library turns Part 2 and Part 3 controls into documents you can adopt directly; the case studies feed the Part 4 continuous-improvement cycle; the checklists are the daily operational output; the reference list is the evidence trail behind all of it.

15. Appendix A: Glossary

Each entry below is defined the way this handbook uses it, with a pointer to the part and section where the term does its operational work. Acronyms are expanded on first use. Where a term maps to a named external standard or framework, the citation is included so you can verify the definition against the primary source rather than this summary.

AADAPT. MITRE's Adversarial Actions in Digital Asset Payment Technologies knowledge base, an ATT&CK-style catalog of adversary tactics specific to blockchain and digital-asset theft, launched in 2024 as a companion to the enterprise ATT&CK matrix (aadapt.mitre.org; Part 4, Section 13.2).

Blind signing. Approving a transaction from a rendered summary without independently verifying the underlying calldata, the root failure mode behind the February 2025 Bybit theft (Appendix C, Section 17.2).

Cold wallet / cold storage. Key custody kept fully offline with no network-connected signing path, the highest-assurance tier for treasury reserves (Part 2, Sections 5.3.1, 6.5).

Compartmentalization. Splitting systems, accounts, and knowledge into isolated domains so compromising one does not automatically expose the rest: network segmentation, dev/test/prod isolation, separate accounts per security domain (Part 1, Section 3.4).

Critical asset. Anything whose loss or exposure directly threatens funds or operational continuity, signing keys foremost, mapped and prioritized in the five-step operational process (Part 1, Section 4.1).

Defense in depth. Layering independent controls across physical, technical, administrative, and human dimensions so no single failure is fatal (Part 1, Section 3.1).

Enhanced Identity Governance (EIG). A Zero Trust component tying every access decision to a verified, continuously reassessed identity rather than a network location (Part 1, Section 2.3.1).

Entry-point centralization. Concentration of user trust onto a few chokepoints, a front-end domain, a DNS zone, an RPC (Remote Procedure Call) endpoint, that an attacker who compromises one path can use to reach every user regardless of how decentralized the underlying protocol is (Part 2, Section 5.6).

Ephemeral / just-in-time (JIT) credential. Access granted for a bounded window and automatically revoked afterward, replacing standing privileged access with access that exists only while actively needed (Part 1, Section 2.3.4).

FIDO2 / passkey. A phishing-resistant authentication standard using public-key cryptography bound to the legitimate origin, preferred over one-time-code apps for any treasury-adjacent account ([FIDO Alliance](#); Part 2, Section 8.3).

Hardware wallet. A dedicated device that generates and stores private key material off a general-purpose, internet-connected computer; its independent display is the last line of defense against a compromised signing interface (Part 2, Section 6.4).

Hot wallet. An internet-connected wallet for operational flows such as gas top-ups, carrying the tightest balance caps of any custody tier precisely because it is the most exposed (Part 2, Section 5.3.1).

Identity and Access Management (IAM). The systems and processes that grant, review, and revoke access by verified identity rather than by shared credential, applied across Web3-specific surfaces such as multisig signer lists and RPC provider consoles (Part 2, Section 8.4).

Identity, Credential, and Access Management (ICAM). Proving who a user is, issuing the credential that represents that identity, and governing what the credential can access, the backbone of Zero Trust identity governance (Part 1, Section 2.3.1).

Insider threat. Risk from a person with legitimate access who uses it, or fails to safeguard it, against the organization's interest, treated in full in the Hiring, Remote Work, and Insider Threat Handbook (05) (Part 3, Section 11.2.1).

Key ceremony. A witnessed, documented procedure for generating a signer's key material and independently cross-checking the resulting address before it joins a multisig or deploy configuration (Part 2, Section 5.3.2).

Laptop farm. A scheme, most publicly linked to Democratic People's Republic of Korea (DPRK) state actors, in which a domestic accomplice hosts company-issued equipment on behalf of a fraudulently hired remote worker to hide their true location, covered in Handbook 05 (Part 3, Section 11.2.1).

Least privilege (PoLP). The rule that access matches current, actual need and nothing more, applied through role-based grants and just-in-time elevation rather than standing broad access (Part 1, Section 3.2).

Log management and visibility. Collection, retention, and monitoring of security-relevant events across identity, endpoint, and signing infrastructure, the evidence base for continuous improvement and incident response (Part 4, Section 14.6).

Microsegmentation. Fine-grained network isolation, down to the individual workload, limiting how far an attacker who lands on one system can move laterally (Part 1, Section 2.3.2).

Multi-Factor Authentication (MFA). Authentication requiring more than one verification factor; this handbook recommends a phishing-resistant form (FIDO2/passkey) over a one-time-code app for any Tier 1 or Tier 2 account (Part 2, Section 8.1).

Mnemonic / seed phrase. The human-readable word sequence encoding a wallet's master private key, a value never typed into a networked device, photographed, or stored in plaintext (Part 2, Section 9.2).

Multi-Party Computation (MPC). Threshold cryptography that splits key generation and signing across multiple parties so no single party ever holds the complete private key, an alternative to on-chain multisig (Part 2, Section 5.3.2).

Multisignature wallet (multisig). A wallet, most commonly built on **Safe** (formerly Gnosis Safe), that executes only once a defined threshold of designated signers approves it on-chain, eliminating unilateral control (Part 2, Section 5.3.2).

Need-to-know. Restriction of sensitive information to only those whose role requires it, layered on top of least privilege (Part 1, Section 3.3).

NIST Cybersecurity Framework (CSF) 2.0. NIST’s voluntary framework, published February 26, 2024, structured around six functions: Govern, Identify, Protect, Detect, Respond, Recover ([NIST CSF 2.0](#); Part 1, Section 2.2).

RPC (Remote Procedure Call) endpoint. The node or gateway a wallet or dApp queries to read chain state and broadcast transactions; a centralized or spoofable endpoint is an entry-point-centralization risk regardless of how decentralized the chain itself is (Part 2, Section 5.6).

SASE (Secure Access Service Edge). A converged network-and-security architecture delivering identity-aware access from the cloud edge rather than a fixed office perimeter, a natural fit for distributed Web3 teams (Part 1, Section 2.3.3).

Signing machine. A workstation dedicated to preparing and approving transactions, hardened and, where risk warrants, kept separate from general browsing and email (Part 2, Section 6.2).

SDP (Software-Defined Perimeter). A Zero Trust control that cloaks internal services from unauthenticated network discovery, granting connectivity only after identity and device posture are verified (Part 1, Section 2.3.2).

Tabletop exercise. A discussion-based walkthrough of a simulated incident, run without touching production systems, used to expose gaps in a response plan before a real event forces the same decisions under pressure (Appendix C, Section 17.4; Part 4, Section 14.3).

Threat actor. An individual or group with the capability and intent to cause harm, profiled by objective and typical technique to prioritize which controls matter most (Part 1, Section 4.2).

Threshold. The minimum number of a multisig’s designated signers whose approval is required before a transaction executes (Part 2, Section 5.3.2).

Wrench attack. Physical coercion of a keyholder to force a signature, key handover, or fund transfer, a risk that multisig thresholds and signer diversity are specifically designed to blunt (Appendix C, Section 17.3).

Zero Trust Architecture (ZTA). A model built on “never trust, always verify,” in which no user, device, or location is trusted by default and every request is authenticated, authorized, and continuously evaluated, defined in [NIST SP 800-207](#) and expanded with implementation guidance in [SP 1800-35](#), finalized June 10, 2025 (Part 1, Section 2.1).

16. Appendix B: Policy and Template Library

The templates below turn the control language from Parts 2 and 3 into documents a security lead can adopt with minimal editing. Each is a skeleton, not a finished policy: fill every bracketed field, route the draft through legal and leadership review, and version it so a later audit shows which policy was in force on a given date. None replace the jurisdiction-specific legal review a real deployment requires.

16.1 Web3 Operational Security Policy Template

The top-level document the other four templates sit under. It states scope, authority, and the classification tiers everything downstream refers back to.

```
# [Organization Name] Web3 Operational Security Policy
Version: [X.X] Effective date: [YYYY-MM-DD] Owner: [Security Lead / Role]
Review cadence: [e.g., every 6 months, or after any Critical/High incident]

## 1. Purpose and Scope
This policy governs operational security for all personnel, contractors, and
systems with access to [organization]'s critical assets, defined in Section 2.

## 2. Asset Classification
| Tier | Examples | Minimum controls |
|-----|-----|-----|
| Tier 1 (Critical) | Deploy keys, multisig signer keys, treasury cold storage | Hard-
ware-backed auth, dedicated signing machine, dual control |
| Tier 2 (Sensitive) | Admin/IAM consoles, CI/CD secrets, customer PII | Phishing-res-
istant MFA, logged access, least privilege |
| Tier 3 (Standard) | General SaaS, internal docs | Organization-wide MFA baseline |

## 3. Roles and Authority
- Security Lead: [name/role] - policy ownership, exception approval
- Asset Owners: [named per Tier 1/2 system] - day-to-day control operation
- All personnel: report suspected compromise within [X hours] per Part 4, Section 14.1

## 4. Governing Sub-Policies
- Key and Wallet Custody Policy (Section 16.2)
- Device and Endpoint Hardening Baseline (Section 16.3)
- Communication and Sensitive Data Handling Policy (Section 16.4)
- Travel and Remote Work Security Policy (Section 16.5)

## 5. Exceptions
Any deviation requires written Security Lead approval, a stated expiry date,
and a compensating control. Undocumented exceptions are policy violations.

## 6. Enforcement and Review
Non-compliance is handled per [HR/disciplinary process reference].
This policy is reviewed on the cadence above and after every Medium+ incident.
```

16.2 Key and Wallet Custody Policy Template

Operationalizes the cold/hot/multisig split from Part 2, Section 5.3, and the key-ceremony requirement from the glossary. This is what a signer reads before joining a multisig, not after.

```
# Key and Wallet Custody Policy
## Custody tiers
| Wallet type | Max balance | Signers required | Storage |
|-----|-----|-----|-----|
| Hot (operational) | [$ cap] | 1, with monitoring | Hardware-backed, network-connected |
| Working capital | [$ cap] | 2-of-[N] multisig | Hardware wallets, geographically distributed |
| Treasury reserve | No cap / [$ cap] | [M]-of-[N] multisig | Air-gapped cold storage |

## Key ceremony requirements
- New signer key generated on an offline, single-purpose device
- Resulting address verified independently by [role] over a pre-agreed out-of-band channel before being added to any multisig
- Ceremony witnessed and logged: date, participants, resulting address

## Signing session requirements
- Never approve a transaction from rendered summary alone; decode and verify raw calldata on a device independent of the proposing interface
- Dual control: proposer and approver are different people for any Tier 1 transfer
- Any signer request received via chat/email is independently verified by voice or video before action, per Section 16.4

## Rotation triggers
- Signer departure (Employee Lifecycle Handbook 03)
- Suspected device compromise
- Every [interval, e.g., 12 months] as routine hygiene
```

16.3 Device and Endpoint Hardening Baseline Template

Applies to any device that touches Tier 1 or Tier 2 assets, per Part 2, Section 6.1.

```
# Endpoint Hardening Baseline
- Full-disk encryption enabled and verified: [Y/N]
- EDR (Endpoint Detection and Response) agent installed and reporting: [Y/N]
- OS and browser patched within [X days] of vendor release
- Screen lock: automatic after [X minutes] idle
- Browser extensions restricted to an approved allowlist; no unreviewed wallet-adjacent extensions
- Signing machines (Section 6.2): no general browsing, no personal email, no unreviewed software installation
- Device enrolled in MDM (Mobile Device Management) with remote-wipe capability
- Local admin rights: default denied, granted only via [JIT process]
```

16.4 Communication and Sensitive Data Handling Policy Template

Governs where Tier 1/Tier 2 information (per Section 16.1) may be discussed, per Part 2, Section 9.

```
# Communication and Sensitive Data Handling Policy
## Approved channels by sensitivity
| Data type | Approved channel | Prohibited channel |
|-----|-----|-----|
| Mnemonics / seed phrases | Never transmitted electronically in any form | Chat, email, SMS, screenshare, password manager notes |
| Signing requests | Voice/video call to a pre-known contact, or in-person | Unverified chat message alone |
| Secrets / API keys | Secrets manager, one-time self-destructing link | Persistent chat history, email |
| General business | Company-approved collaboration platform | Personal messaging apps for company business |

## Verification rule
Any request to move funds, change a signer, or share a credential that arrives over chat, email, or a phone call is independently verified through a second, pre-agreed channel before action, regardless of apparent urgency or seniority of the requester.
```

16.5 Travel and Remote Work Security Policy Template

Supports Part 2, Section 10 and the physical-coercion lessons in Appendix C, Section 17.3.

```
# Travel and Remote Work Security Policy
## Before travel
- Loaner device issued for high-risk destinations; no Tier 1 credentials carried on personal or primary devices
- Signer status and holdings not disclosed in travel-related communications or public itineraries
- Emergency/duress contact and check-in schedule confirmed (Appendix C, 17.3)

## During travel
- No signing sessions performed over untrusted or hotel Wi-Fi without a managed VPN or SASE (Secure Access Service Edge) connection
- Duress protocol (Appendix D, Section 18.4) reviewed and understood before departure

## On return
- Loaner device wiped and re-imaged before reissue, per NIST SP 800-88 Rev. 1 media sanitization guidance
- Any anomalous device behavior during travel reported before reconnecting to production systems
```

17. Appendix C: Case Studies and Tabletop Exercises

The two incidents below are the largest cryptocurrency thefts publicly attributed to a named nation-state actor as of this writing, and both trace back to an operational security failure rather than a cryptographic one. The physical-coercion trend that follows is a reminder that the strongest key-management design still assumes the keyholder is not under duress. Each case study closes with detection signals and mitigations rather than attack detail beyond what a defender needs to threat-model the failure.

17.1 Case Study: Ronin Network Validator Compromise (March 2022)

On March 23, 2022, attackers drained approximately 173,600 Ether and 25.5 million USDC (roughly \$625 million at the time) from the Ronin bridge underlying the Axie Infinity game, a theft undetected for six days until a user's failed withdrawal surfaced it ([Ronin Network](#), [Wikipedia summary](#)). The bridge required five of nine validator signatures. Attackers compromised four validators run directly by Sky Mavis, the game's developer, then used a still-active emergency permission, granted to Sky Mavis months earlier by the Axie DAO to help process a load spike and never revoked, to sign as a fifth validator without compromising it individually. On April 14, 2022, the FBI attributed the theft to the Lazarus Group and APT38, both linked to the Democratic People's Republic of Korea (DPRK).

Detection signals: a stale, broad-scope permission (the Axie DAO delegation) with no expiry or periodic re-justification; four of nine validator keys concentrated under one operator instead of independently run; and a six-day detection gap showing no monitoring watched bridge outflows against expected volume.

Mitigations: expire every emergency or temporary delegation by default and require active re-approval (Part 1, Section 3.2.3); enforce genuine signer diversity so compromising one operator cannot silently satisfy multiple threshold slots (Part 2, Section 5.3.2); alert on outbound transfers against a modeled baseline rather than waiting on a user to notice (Part 4, Section 14.6).

17.2 Case Study: Bybit Multisig Blind-Signing Compromise (February 2025)

On February 21, 2025, approximately 400,000 Ether (roughly \$1.4 to \$1.5 billion) was stolen from Bybit's cold wallet, the largest cryptocurrency exchange theft on record, attributed by the FBI to DPRK's "TraderTraitor" operation ([Bybit](#), [Wikipedia summary](#); [FBI/IC3 PSA250226](#)). Attackers compromised infrastructure belonging to Safe{Wallet}, the third-party multisig interface Bybit's signers used, and manipulated what the interface displayed. Signers working from hardware wallets approved what the interface rendered as a routine transfer; the calldata they actually signed reassigned control of the cold wallet's contract logic to an attacker-controlled address.

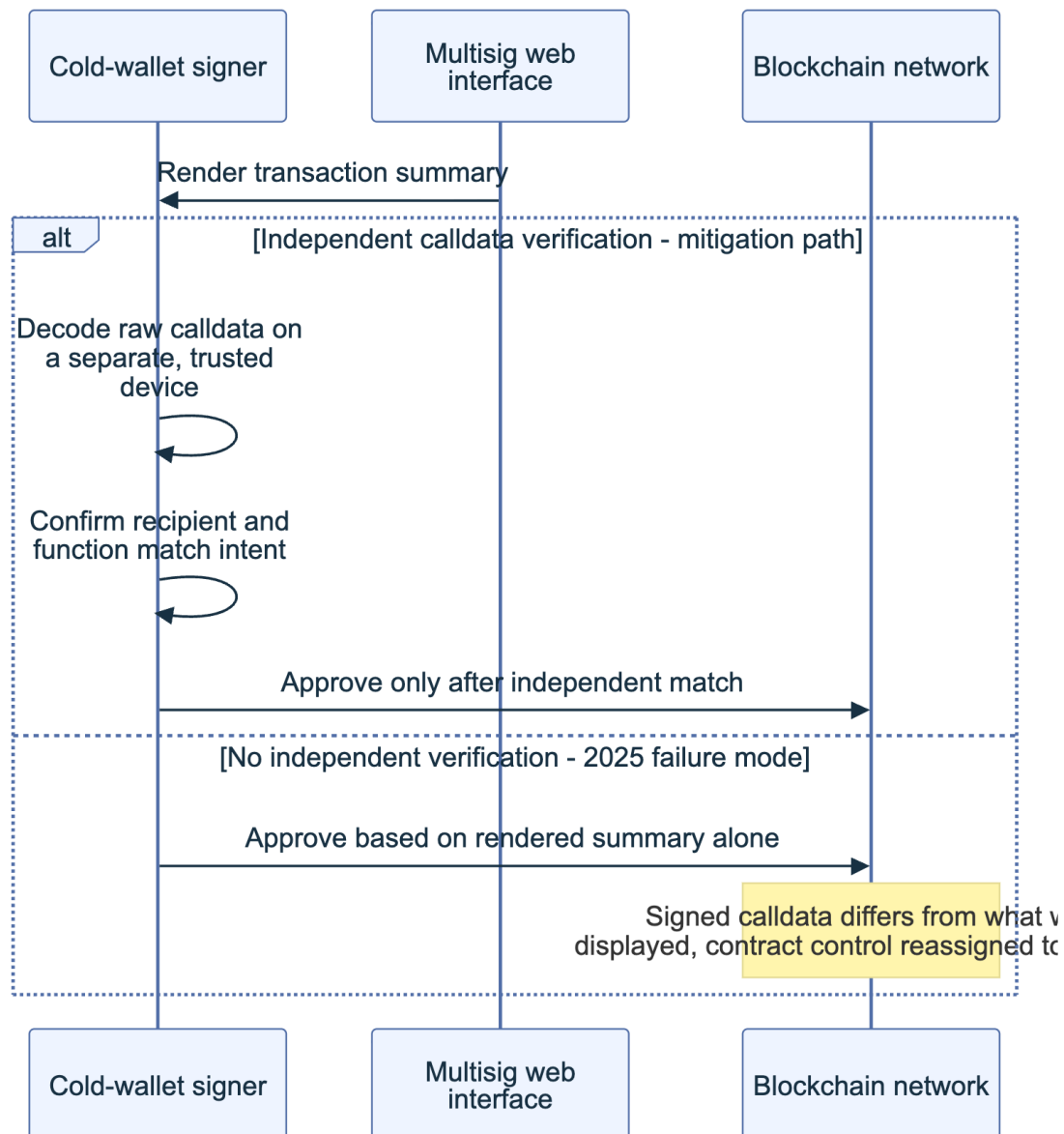


Figure 2. The blind-signing failure path versus the independent-verification mitigation. The hardware wallet correctly signed what it was told to sign; the compromise was in what the signer was shown, not in the cryptography.

Detection signals: signers had no independent, out-of-band way to confirm the rendered summary matched the raw calldata their hardware device was about to sign, and the compromise lived entirely in third-party infrastructure the victim did not directly control or monitor.

Mitigations: decode and verify raw calldata on a device independent of the proposing interface before every Tier 1 approval (Appendix B, Section 16.2); treat a multisig UI provider as part of your trust boundary and monitor its advisories like a direct vendor (Part 3, Section 11.1.3); require the proposer and final approver to use genuinely separate infrastructure, not merely separate accounts on the same interface.

17.3 Case Study: Physical Coercion and Wrench Attacks (2025)

Reporting through 2025 documented a rising pattern of physical attacks, kidnapping, coercion, and extortion targeting individuals publicly known or suspected to hold significant cryptocurrency, including keyholders and executives at crypto firms, a trend the Associated Press covered under the informal industry term “wrench attack” ([origin and reporting summarized on Wikipedia](#)). Unlike the two cases above, no cryptographic or software control stops a wrench attack: the adversary’s leverage is the keyholder’s physical safety, not a technical gap.

Detection signals: public disclosure of holdings, signer status, or travel plans (speaker bios, social media, leaked directories) that lets an adversary locate a target; and no pre-agreed duress signal, so a coerced approval looks identical to a voluntary one.

Mitigations: minimize public association between an identity and its signer or custody role; set multisig thresholds so no single coerced signer can move funds alone, a cryptographic control that addresses the physical threat by making coercion of one person insufficient; establish a duress protocol (Appendix D, Section 18.4) so a coerced keyholder can signal it without the coercer knowing.

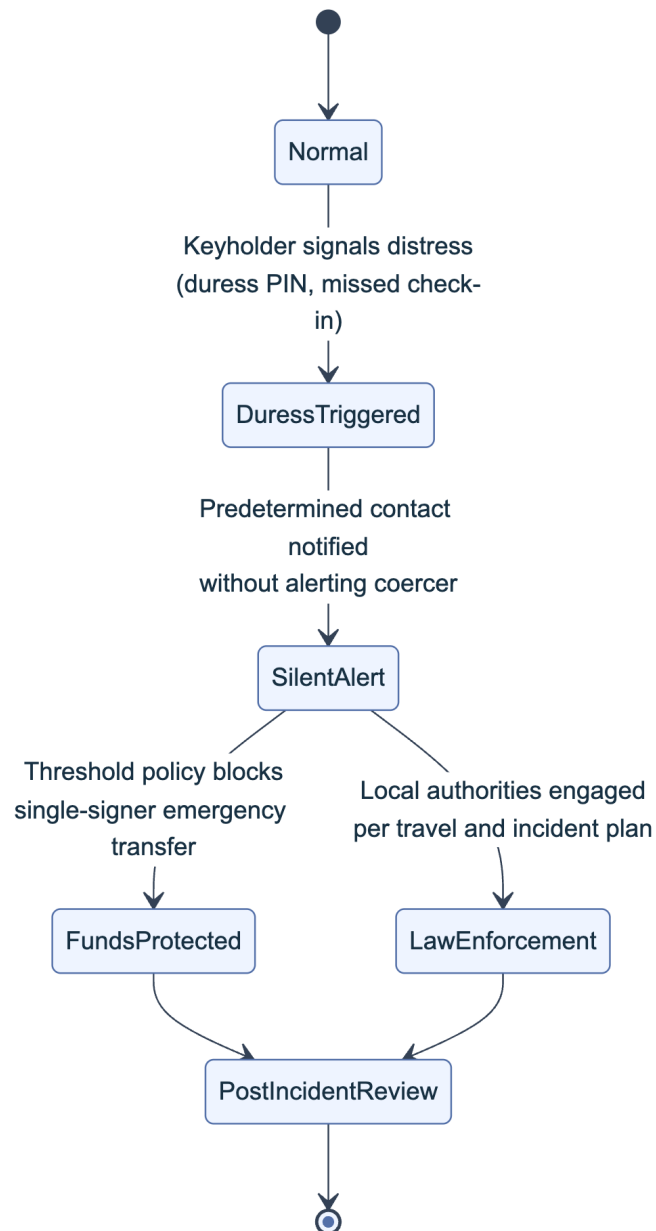


Figure 3. A duress-response protocol. The design goal is that a coerced signer can trigger this path without the coercer being able to tell the difference from a normal approval in progress.

17.4 Tabletop Exercise Scripts

Run each script as a discussion-based exercise per Part 4, Section 14.3: no production system is touched, and the facilitator introduces each inject verbally or in writing, then pauses for the team's decision before revealing the next inject.

Scenario	Objective	Key injects (in order)	Success criteria
Validator/engineer targeting (Ronin-pattern)	Catch a social-engineering approach before it reaches infrastructure access	1. Engineer reports an unsolicited, unusually generous remote-role offer with an attachment. 2. Facilitator reveals the attachment requested unusual permissions when "opened." 3. What standing permissions could this role already reach?	Team names the report-and-verify step (Part 2, Section 9.1) and gives a real answer on standing permissions, not "we'd have to check"
Multisig session compromise (Bybit-pattern)	Test whether signers verify calldata independently under time pressure	1. Routine multisig transfer proposed with a plausible deadline. 2. Facilitator reveals the rendered summary and a separately decoded calldata sample that disagree. 3. Where in your actual process would this be caught?	Team names the specific control (Appendix B, Section 16.2) that catches the mismatch and confirms it is practiced, not just documented
Keyholder duress (wrench-attack pattern)	Test the duress protocol without depending on the coerced person acting normally	1. Signer misses a scheduled check-in while traveling. 2. Facilitator reveals a signing request arrives shortly after, apparently from that signer, under time pressure. 3. Who is authorized to invoke the duress protocol, and on what evidence?	Team names the authorized decision-maker and the threshold control (Appendix C, Section 17.3) that limits damage even if invoked late

18. Appendix D: Checklists (Quick Reference)

These are deliberately short: each is meant to fit on a single printed page or a single pinned message in an operations channel. For the full narrative behind any line, follow the section reference back into Parts 2 through 4.

18.1 Endpoint and Signing Machine Quick Checklist

- ☐ Full-disk encryption verified on this device (Part 2, 6.1)
- ☐ EDR agent installed and reporting (Appendix B, 16.3)
- ☐ OS and browser fully patched (Part 2, 6.1)
- ☐ No unreviewed browser extensions installed (Part 2, 6.3)
- ☐ Signing machine used for signing only, no general browsing (Part 2, 6.2)
- ☐ Hardware wallet firmware current and sourced from vendor directly (Part 2, 6.4)

18.2 Authentication and Access Quick Checklist

- ☐ Phishing-resistant MFA (FIDO2/passkey) enabled on every Tier 1/2 account (Part 2, 8.1, 8.3)
- ☐ No shared or reused credentials across systems (Part 2, 8.2)
- ☐ Standing access reviewed against actual current need this quarter (Part 1, 3.2)
- ☐ Just-in-time elevation used for any non-routine privileged action (Part 1, 2.3.4)

18.3 Communication and Key-Handling Quick Checklist

- ☐ No mnemonic or seed phrase has ever been typed into a networked device or chat (Appendix B, 16.4)
- ☐ Any signing or fund-movement request received via chat/email verified over a second channel (Appendix B, 16.4)
- ☐ Secrets shared only through the approved secrets manager or a self-destructing link (Part 2, 9.2)

18.4 Travel and Duress Quick Checklist

- ☐ Loaner device issued for high-risk destinations; no Tier 1 credentials on personal devices (Appendix B, 16.5)
- ☐ Duress signal and check-in schedule confirmed with a named contact before departure (Appendix C, 17.3)
- ☐ VPN/SASE required for any signing-adjacent activity on untrusted networks (Part 1, 2.3.3)
- ☐ Signer status and travel itinerary not disclosed publicly (Appendix C, 17.3)

18.5 Control Domain Quick-Reference Matrix

Use this table to sanity-check that a new control lands in the domain it belongs to, per the four-domain structure in Part 3, Section 11.

Domain	Example control	Typical owner
Organizational	Written OpSec policy, third-party assurance review	Security Lead
People and personnel	Insider-threat vetting, security training cadence	HR + Security
Physical and environmental	Tamper-evident hardware storage, travel protocol	Facilities + Security
Technical and digital	MFA, endpoint hardening, encrypted backups	Engineering + Security

19. Appendix E: References and Further Reading

The full, deduplicated bibliography for this handbook lives in a separate file, `references.md`, maintained independently of the narrative parts so sources can be updated as incident reporting and standards revisions mature. Every inline citation across Parts 1 through 4 and the appendices above points to a primary source: a standard's publication page, a government attribution statement, or a named incident's own disclosure.

Category	What it holds	Representative sources
Standards and Frameworks	NIST and MITRE publications this handbook's controls trace back to	NIST SP 800-207, NIST SP 1800-35, NIST CSF 2.0, NIST SP 800-53 Rev. 5, NIST SP 800-88 Rev. 1, MITRE ATT&CK, MITRE AADAPT
Incidents and Case Studies	Named thefts and physical-security incidents cited as evidence for a control	Ronin Network (March 2022), Bybit (February 2025), the 2025 wrench-attack reporting trend
Tools	Vendor and project documentation for named tools referenced in templates	Safe (formerly Gnosis Safe), FIDO Alliance passkey documentation
Further Reading	Cross-references to companion OWASP SCS handbooks and standards	OWASP SCSVS, SCSTG, SCWE, Web3 Attack Vectors Top 15, Handbooks 03, 05, and 06

Treat `references.md` as the place to verify a claim before repeating it externally, particularly for the incident case studies in Appendix C: public and forensic reporting on both thefts was still being refined after initial disclosure, and figures cited here reflect the sources linked at the point of each claim.

Key controls for Part 5

- Keep the glossary within reach for any reader joining mid-process, new to the security function or running their first key ceremony.
- Adopt the Appendix B templates as living documents: version them, route exceptions through written approval, re-review after any Medium-or-higher incident.
- Run at least one Appendix C tabletop script per year per scenario type; a team's inability to answer the facilitator's question is the finding, not a failed test.
- Use the Appendix D checklists as the actual pre-flight gate for signing sessions and travel, not retrospective documentation.
- Verify a claim against `references.md` before repeating it outside the handbook, especially incident details that may be updated as investigations continue.

This is the final part of the OWASP SCS Web3 Operational Security Handbook.

Appendix C: References and Further Reading

The sources below are grouped by category and cited throughout the handbook's parts; each is listed once even where multiple sections reference it.

Standards and Frameworks

- NIST Cybersecurity Framework (CSF) 2.0
- NIST Cybersecurity Framework (CSF) 2.0 overview
- NIST Cybersecurity Framework, CSF 2.0 release (February 2024)
- NIST SP 800-207, Zero Trust Architecture
- NIST SP 1800-35, Implementing a Zero Trust Architecture
- NIST SP 800-53 Revision 5, Security and Privacy Controls for Information Systems and Organizations
- NIST SP 800-61 Revision 3, Incident Response Recommendations and Considerations for Cybersecurity Risk Management
- NIST SP 800-63-4, Digital Identity Guidelines
- NIST SP 800-63B, Authentication and Lifecycle Management
- NIST SP 800-92, Guide to Computer Security Log Management
- ISO/IEC 27001:2022
- Center for Internet Security (CIS) Benchmarks
- Cybersecurity Capability Maturity Model (C2M2)
- CISA Zero Trust Maturity Model
- Cloud Security Alliance, Software-Defined Perimeter
- Gartner, Secure Access Service Edge (SASE)
- OWASP Smart Contract Security (SCS) / Smart Contract Security Verification Standard (SCSVS)
- OWASP Web3 Attack Vectors Top 15
- MITRE ATT&CK
- MITRE AADAPT (Adversarial Actions in Digital Asset Payment Technologies)
- Supply-chain Levels for Software Artifacts (SLSA)
- CycloneDX
- SPDX
- FIDO Alliance, FIDO2/WebAuthn
- FIDO Alliance, Passkeys
- EIP-4337, Account Abstraction
- EIP-4361, Sign-In with Ethereum

- BIP-39, Mnemonic Code for Generating Deterministic Keys
- SLIP-0039, Shamir's Secret-Sharing for Mnemonic Codes
- CryptoCurrency Security Standard (CCSS)
- Markets in Crypto-Assets Regulation (MiCA)
- FATF Recommendation 16, the "Travel Rule"
- GDPR Article 33, Notification of a Personal Data Breach
- National Security Decision Directive 298 (1988)
- idmanagement.gov, Federal Identity, Credential, and Access Management

Incidents and Case Studies

- Bybit hack summary, Wikipedia
- Bybit official statement, X/Twitter
- Ronin Network incident summary, Wikipedia
- Halborn, Radiant Capital Hack Explained (October 2024)
- Trail of Bits, Supply-Chain Attacks Are Exploiting Our Assumptions (XZ Utils)
- Uber, September 2022 Security Incident Update
- Unit 42, Slow Pisces: New Custom Malware
- FBI Statement on Attribution of Malicious Cyber Activity to Lazarus Group and APT38
- FBI, CISA, and Treasury Joint Advisory AA22-108A, North Korean Cyber Actors Targeting Blockchain Companies
- FBI/IC3 Public Service Announcement PSA250226
- DOJ, Court-Authorized Disruption of North Korean Remote IT Worker Fraud

Tools and Documentation

- Chainalysis
- Chainalysis 2025 Crypto Crime Report
- TRM Labs
- Ledger, Ledger Recover
- Safe
- Signal Protocol documentation
- CISA, homepage and layered-security guidance
- CISA, Phishing-Resistant MFA Fact Sheet
- CISA, Tabletop Exercise Packages (CTEPs)
- Google Workspace, Security Checklist for Medium and Large Businesses
- Office of Foreign Assets Control (OFAC)
- FBI Internet Crime Complaint Center (IC3)
- FBI, Counterintelligence

Further Reading

- [Google Site Reliability Engineering, Postmortem Culture](#)
- [EFF, Digital Privacy at the U.S. Border \(2017\)](#)
- [Rubber-hose cryptanalysis, Wikipedia](#)